

SKALA

2) AI의 서비스화데이터분석 및 AIOps

1. 데이터 분석을 위한 Python 이해

2026.6

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

[Python Summary]

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

변수와 데이터 타입 – 변수

변수 이름 규칙 + a

변수 이름 설정 시 지켜야 할 몇 가지 원칙들

- 사용 가능한 문자: 알파벳 (소문자 **a-z**, 대문자 **A-Z**), 숫자 (**0-9**), 언더스코어 (**_**)
- 첫 글자 규칙: 변수명은 숫자로 시작할 수 없으며, 문자나 언더스코어로 시작
- 대소문자 구분 (예: **myVar**와 **MyVar**는 다른 변수)
- 예약어 사용 금지: **Python**의 예약어(키워드)는 사용 불가
- 의미 있는 이름 사용: 데이터와 관련 있는 의미 있는 변수명을 사용하는 것이 좋음
- 특수문자 사용 제한: 언더스코어(**_**)를 제외한 특수문자는 사용 불가
- 명명 규칙: 소문자 사용하되,
여러 단어로 구성된 경우 **snake_case** 또는 **camelCase**
- 언더스코어로 시작하는 이름 주의: 특별한 의미를 가질 수 있어서
일반 변수에는 피하는 것이 좋음

예약어 확인

```
import keyword
print(keyword.kwlist)
```

1. 변수?

- 데이터를 저장하는 공간
- 이름을 통해 값을 참조 가능

2. 변수 선언과 할당

예시

```
x = 10 # 정수형 변수
name = "Alice" # 문자열 변수
is_active = True # 불리언 변수
```

Python

간결하고 가독성이 높은 문법을 가진 인터프리터 기반의 프로그래밍 언어

특징

- 문법이 쉽고 간결함 → 초보자도 배우기 쉬움
- 인터프리터 언어 → 코드를 한 줄씩 실행하며 테스트 가능
- 다양한 라이브러리 제공 → 데이터 분석, 인공지능, 웹 개발 등 활용 가능
- 객체 지향 프로그래밍(OOP) 지원
- 멀티 패러다임 지원 → 절차적, 객체 지향, 함수형 프로그래밍 가능
- 플랫폼 독립적 → **Windows, Mac, Linux** 등 다양한 환경에서 실행 가능

활용분야

- 웹 개발: **Django, Flask**
- 데이터 분석: **Pandas, NumPy, Matplotlib**
- 인공지능(AI) 및 머신러닝: **TensorFlow, PyTorch, Scikit-Learn**
- 자동화 및 스크립트: 업무 자동화, 웹 스크래핑
- 게임 개발: **Pygame**
- 네트워크 프로그래밍: **socket** 모듈

Python

간결하고 가독성이 높은 문법을 가진 인터프리터 기반의 프로그래밍 언어이지만.....

활용 불가(?)

- 운영체제(OS) 및 저수준 시스템 프로그래밍
 - 이유: Python은 인터프리터 언어로, 메모리 관리나 하드웨어 직접 제어가 어려움
- 모바일 앱 개발 (iOS, Android 네이티브 지원이 어려움)
- 고성능 게임 개발 (CPU 사용 ↑)
- 실시간 시스템 및 하드웨어 제어
 - Garbage Collection (사용하지 않는 객체/메모리 정리)으로 인한 오류 발생 가능 多 (Global Interpreter Lock 이 걸릴 때...)
- 블록체인 개발 (높은 보안성, 효율성 요구...)
- ERP 시스템 등 대형 Enterprise System.
- multithreading 활용 기반 고성능 Application
 - 멀티프로세스는 가능하지만 multithreading 성능은 낮음

Python 문법 Summary

요약 정리

- **Python**은 다른 프로그래밍 언어와 마찬가지로 다양한 수 데이터(정수, 실수 등)를 처리하기 위한 문법을 제공
- 변수에 값을 할당할 때는 등호(=)를 사용
- 정수 데이터의 경우 양의 정수, 0, 음의 정수로 구성되며, 소수점 이하 값이 없다.
- **Python**은 컴퓨터의 메모리가 허용하는 한, 정수 데이터에 크기 제한이 없다.
- 실수 데이터는 소수점 이하 값을 포함하는 데이터
- 실수 데이터와 정수 데이터 사이에서 연산을 수행하는 경우, 데이터의 형 변환(정수-->실수)이 일어난다.
- 사칙연산 : +, -, *, /
- 몫 연산자 : //
- 나머지 연산자 : %
- 제곱 연산자 : **
- 문자열을 사용할 때는 큰 따옴표나 작은 따옴표를 사용
- 문자열끼리 덧셈을 수행하면, 단순히 두 문자열이 연결된다.

Python 문법 Summary

요약 정리

- 문자열 안에서 큰 따옴표나 작은 따옴표를 출력해야 한다면 역슬래시(\) 사용
- 문자열 인덱싱의 경우 첫번째 문자는 인덱스 0에 해당한다. `aa[:5]`, `bb[3:]`
- 리스트 자료형은 대괄호 안에 원소들을 쉼표로 구분하여 넣을 수 있다.
- 일반적으로 리스트의 각 원소는 같은 자료형을 쓰지만, 서로 다른 자료형의 데이터가 들어갈 수 있다.
- 리스트에 대해 인덱싱과 슬라이싱을 사용할 수 있다.
- 리스트끼리 더할 수 있는데, 이 때는 단순히 두 리스트를 이어 붙인 결과가 반환된다.
- 리스트는 다중(2중, 3중 등)으로 중첩하여 사용할 수 있다.
- 문자열은 불변(**Immutable**) 객체이지만, 리스트는 가변(**mutable**) 객체이다.
- 리스트 대상 **insert**, **append**, **remove**, **sort** 메서드 사용 가능
- 리스트 초기화 시 리스트 컴프리헨션(**list comprehension**)을 사용하면 효율적이고 간결함 증대
- 튜플 자료형은 리스트와 유사하지만, 대괄호 대신 소괄호()을 사용하고 리스트는 값을 변경할 수 있지만, 튜플은 값을 변경할 수 없다. 값이 변경되면 안 되는 경우, 튜플을 사용하면 효과적이다.

Python 문법 Summary

요약 정리

- 사전 자료형과 집합 자료형의 경우 특정한 원소의 존재 여부를 빠르게 확인하고 추가/삭제할 수 있다.
- 사전(Dictionary) 자료형의 경우, 데이터를 **Key-Value** 쌍의 형태로 저장할 때 사용할 수 있다.
 - ✓ **{key: value}** 형식으로 데이터를 저장하며, 각 키는 고유해야 함.
 - ✓ 값을 중복될 수 있지만, 키는 중복될 수 없음.
 - ✓ **Python 3.7**부터는 삽입 순서를 보장
- 집합(Set) 자료형의 경우, 데이터의 중복을 허용하지 않고, 순서와 상관없을 때 사용하는 자료형
 - ✓ 중복되지 않는 고유한 요소들을 집합으로 저장하는 자료형
 - ✓ {요소1, 요소2, ...} 형식으로 데이터 저장
 - ✓ 집합은 순서가 없으며, 인덱스를 통해 접근 불가
 - ✓ 중복 없는 값의 수집, 집합 연산(교집합, 합집합 등)을 수행할 필요가 있을 때 사용
- **Bool** 자료형
 - ✓ 거짓(false) : 0, 비어있는 리스트/튜플/사전, None 등
 - ✓ 참(true) : 이외의 실제 의미 있는 값들

Python 문법 Summary

요약 정리

- 일반적으로 표준 입출력을 사용하는 프로그램은 키보드(입력장치)로부터 입력을 받아 모니터(출력장치)로 결과를 내보낸다.
- 파일 객체를 만들 때는 `open()`, 파일 객체를 닫을 때는 `close()` 함수를 사용한다.
- 파일 모드 : `r(read)`, `w(write)`, `a(append)`
- 파일에 데이터를 쓸 때는 `write()` 함수를 사용할 수 있다.
- 파일의 모든 줄을 한꺼번에 읽을 때는 `readlines()` 함수를 사용할 수 있으며, 기본적으로 각 줄을 그대로 읽기 때문에 줄바꿈 기호(`\n`)가 포함되어 있다. 줄바꿈 기호를 제거하기 위해 `strip()`을 사용할 수 있다.
- `readline()` 함수를 이용하면 파일의 데이터를 한 줄 씩 읽을 수 있다.
- `read()` 함수를 사용하면 파일의 전체 내용을 하나의 문자열로 반환한다.

파일 쓰기 예제

```
with open('example.txt', 'w') as file:  
    file.write("Hello, World!\n")  
    file.write("This is a sample text file.\n")  
    file.write("Goodbye!")
```

Python 문법 Summary

요약 정리

파일 읽기 예제

```
with open('example.txt', 'r') as file:  
    contents = file.read() # 파일의 모든 내용을 읽기  
    print(contents)
```

여러 줄을 파일에 쓰기 예제

```
lines = ["First line\n", "Second line\n", "Third line\n"]  
  
with open('lines_example.txt', 'w') as file:  
    file.writelines(lines) # 리스트의 각 요소를 파일에 쓰기
```

Python 문법 Summary

요약 정리

여러 줄을 읽기 예제

```
with open('lines_example.txt', 'r') as file:
```

```
    lines = file.readlines() # 파일의 모든 줄을 읽어 리스트로 반환
```

각 줄 출력

```
for line in lines:
```

```
    print(line.strip()) # 줄 끝의 개행 문자 제거하고 출력
```

기존 파일에 추가 쓰기 예제

```
with open('example.txt', 'a') as file:
```

```
    file.write("\nThis line is added later.")
```

Python 문법 Summary

요약 정리

- 전체 소스코드를 기능별로 분리하여 작성하는 것이 효율적이며, 함수를 이용해 소스 코드를 기능별로 분리할 수 있다.

```
def greet(name):  
    """주어진 이름을 가진 사람에게 인사합니다."""  
    return f"Hello, {name}!"  
  
# 함수 호출  
message = greet("Alice")  
print(message)  # 출력: Hello, Alice!
```

Python 문법 Summary

요약 정리

- 클래스 : 봉어빵 틀에 비유
- 인스턴스 : 봉어빵 틀에서 생성된 봉어빵에 비유

```
class Person:
    def __init__(self, name, age):
        self.name = name # 인스턴스 변수 name 초기화
        self.age = age   # 인스턴스 변수 age 초기화

    def introduce(self):
        return f"Hello, my name is {self.name} and I am {self.age} years old."

# 객체 생성
person1 = Person("Alice", 30)
person2 = Person("Bob", 25)

# 메서드 호출
print(person1.introduce()) # 출력: Hello, my name is Alice and I am 30 years old.
print(person2.introduce()) # 출력: Hello, my name is Bob and I am 25 years old.
```

Python 문법 Summary

요약 정리

- 하기와 같이 예외처리

```
try:
    number = int(input("숫자를 입력하세요: "))
    result = 10 / number
except ValueError:
    print("유효하지 않은 숫자입니다.")
except ZeroDivisionError:
    print("0으로 나눌 수 없습니다.")
else:
    print(f"결과는 {result}입니다.") # 예외가 발생하지 않았을 때 실행
```

- 
- 1.** 실행 구조와 개발 환경
 - 2.** 자료구조와 컴프리헨션
 - 3.** 함수·파일·예외 처리
 - 4.** 타입 힌트와 데이터 모델링
 - 5.** 코드 품질과 기초 테스트
 - 6.** 비동기와 병렬 처리 입문
 - 7.** 종합 실습
- 데이터 수집 미니 파이프라인

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

1. 실행 구조와 개발 환경

- 인터프리터 · AST · 바이트코드 · PVM · venv

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

왜 데이터 분석의 표준 언어인가?

인터프리터 언어

- ✓ 소스코드를 한 줄씩 읽어 즉시 실행
- ✓ 대화형(REPL) 탐색이 가능해 데이터 분석에 최적

방대한 생태계

- ✓ **NumPy, Pandas, scikit-learn, Polars, DuckDB**
- ✓ 데이터 분석에 필요한 모든 것이 **pip** 로

2026 AI/ML 표준

- ✓ **TensorFlow·PyTorch·LangChain·RAG**
파이프라인까지 **Python** 중심
- ✓ 전체 커리큘럼 기반 언어

읽기 쉬운 문법

- ✓ 들여쓰기 기반 블록 구조
- ✓ 분석 코드를 비전공자도 읽을 수 있을 만큼 간결

Python 실행 구조

Python은 인터프리터 언어다

개념

- 파이썬은 컴파일 언어가 아닌 인터프리터 언어
- 사용자가 .py 파일로 작성한 코드를 한 줄씩 읽고 바로 실행
- 그러나 단순한 한 줄씩 실행 구조가 아니라, 중간에 바이트코드(Bytecode)라는 중간 표현 단계 존재

인터프리터 역할

- 소스코드
 - 파서(Parser)
 - **AST(Abstract Syntax Tree)**
 - 바이트코드 생성기
 - **Python Virtual Machine(PVM)**

```
import dis

def add(x, y):
    return x + y

dis.dis(add) # 바이트코드 출력
```

Python 실행 흐름

소스코드에서 결과까지



CPython은 소스를 직접 기계어로 변환하지 않고 바이트코드를 거친다.

→ 이식성과 재사용성 확보

AST와 dis

실행 구조를 눈으로 확인하기

핵심 내용

- ✓ **AST**: 소스코드를 트리 구조로 추상화
- ✓ 린터·포매터·트랜스파일러가 **AST**를 사용
- ✓ **dis** 모듈: 함수가 어떤 바이트코드로 컴파일되는지 출력
- ✓ **LOAD_FAST·BINARY_OP·RETURN_VALUE** 명령어 구조
- ✓ 컴프리헨션이 **for** 루프보다 빠른 이유를 바이트코드로 확인

【 코드 예시 】

```
# dis 모듈 실습
import ast, dis

# AST 분석
tree = ast.parse('x = a + b')
print(ast.dump(tree, indent=2))

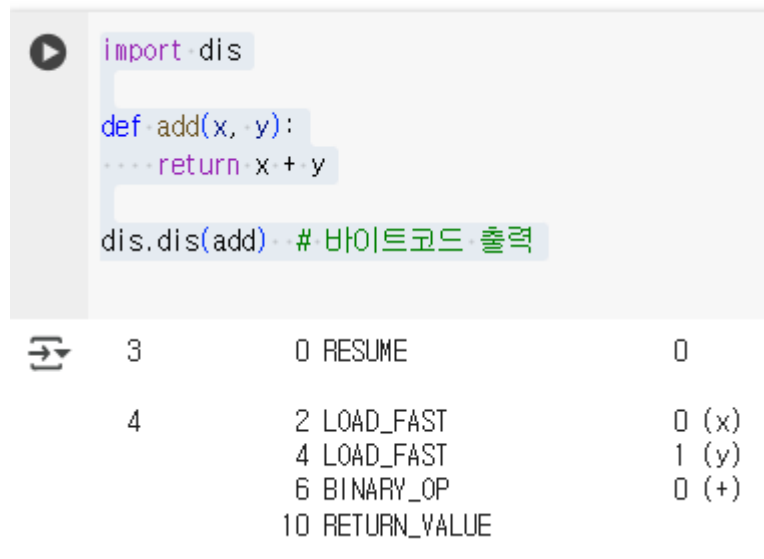
# 바이트코드 분석
def add(x, y): return x + y
dis.dis(add)

# 출력:
# LOAD_FAST 0 (x)
# LOAD_FAST 1 (y)
# BINARY_OP 0 (+)
# RETURN_VALUE
```

Python 실행 구조

Python은 컴파일 + 인터프리트 방식으로 바이트코드(Python VM 실행 중간코드)로 재사용성, 이식성이 높음

【 바이트코드 명령어 】



The image shows a Python code snippet in a text editor with a play button icon. The code defines a function `add(x, y)` that returns `x + y`, and then calls `dis.dis(add)` with a comment `# 바이트코드 출력`. Below the code, the output of the `dis` module is shown, displaying the byte code for the `add` function. The output is a table with four columns: instruction number, instruction, argument, and comment.

줄번호	바이트오프셋	명령어	인자	의미
3	0	RESUME	0	Python 3.11 이후 도입된 실행 초기화 명령어 (디버깅/프레임 시작용)
4	2	LOAD_FAST	0(x)	지역 변수 x를 스택에 Push
	4	LOAD_FAST	1(y)	지역 변수 y를 스택에 push
	6	BINARY_OP	0(+)	스택에서 두 값을 pop 한 후 + 연산 수행, 결과를 Push
	10	RETURN_VALUE	-	연산결과 반환

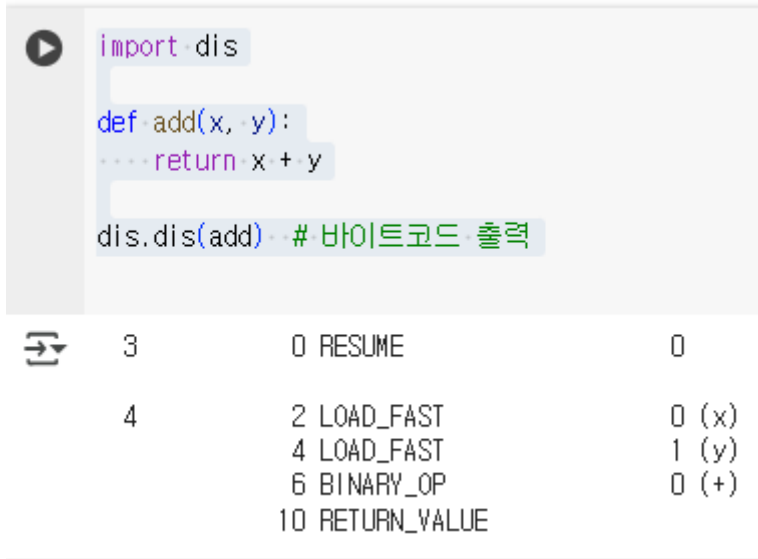
줄번호	바이트오프셋	명령어	인자	의미
3	0	RESUME	0	Python 3.11 이후 도입된 실행 초기화 명령어 (디버깅/프레임 시작용)
4	2	LOAD_FAST	0(x)	지역 변수 x를 스택에 Push
	4	LOAD_FAST	1(y)	지역 변수 y를 스택에 push
	6	BINARY_OP	0(+)	스택에서 두 값을 pop 한 후 + 연산 수행, 결과를 Push
	10	RETURN_VALUE	-	연산결과 반환

→ Python의 바이트코드 분석 결과
특정 함수의 내부 실행 흐름(dis모듈 출력)

x와 y를 스택에 올리고 x + y 연산을 수행한 뒤
결과를 반환(return)하는 구조

Python 실행 구조

Python은 컴파일 + 인터프리트 방식으로 바이트코드(Python VM 실행 중간코드)로 재사용성, 이식성이 높음



```

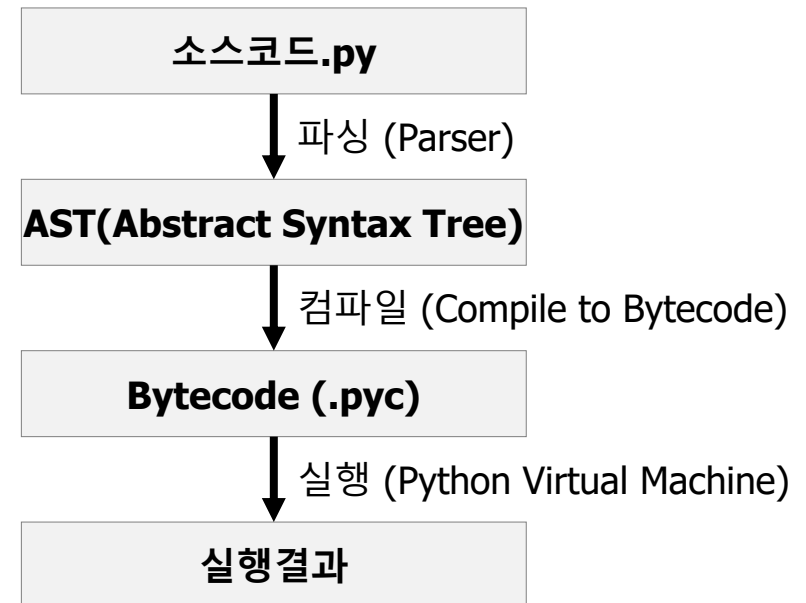
import dis

def add(x, y):
    ...return x + y

dis.dis(add) # 바이트코드 출력
  
```

3	0 RESUME	0
4	2 LOAD_FAST	0 (x)
	4 LOAD_FAST	1 (y)
	6 BINARY_OP	0 (+)
	10 RETURN_VALUE	

파이썬의 실행 구조



[참고] AST(Abstract Syntax Tree)

【 AST 사용 목적 】

코드 분석 및 이해

- 코드의 구조를 명확하게 보여줌
 - 컴파일러, 인터프리터가 코드 의미 분석 및 오류 검증

코드 변환 및 최적화

- 코드 구조를 트리 형태로 조작
 - 더 효율적인 코드로 최적화
 - 다른 버전의 언어로 쉽게 변환
(최신 JAVASCRIPT 문법을 구형 문법으로 변환)

다양한 개발도구 활용

- 수많은 개발도구들을 활용하여 코드분석 및 내부조작
 - 린터(Linter) : 코드 문법 오류 수정
 - 포매터(Formatter) : 코드 스타일 통일
 - 트랜스파일러(Transpiler) : 최신 문법 변환

“우리가 만든 코드를 기계가
실행할 수 있는 형태로 바꾸는
과정(컴파일), 코드에 문제가
없는지 분석하는 과정에서
AST가 핵심적인 역할”

- 코드의 의미/구조를 컴퓨터가
이해하기 쉽게 문법적인
핵심 요소만 남겨 추상화한 자료 구조
- 노드(변수 선언, 연산, 함수 호출 등)
코드의 핵심적 의미 요소만 남아
계층적 트리 구조 구성

[참고] AST(Abstract Syntax Tree)

자바스크립트
const a = 10 + 5;

AST

- VariableDeclaration (변수 선언)
 - kind: const (상수)
 - declarations:
 - VariableDeclarator (변수 선언자)
 - id: Identifier (식별자)
 - name: "a"
 - init: BinaryExpression (이항 표현식)
 - left: Literal (리터럴)
 - value: 10
 - operator: "+"
 - right: Literal (리터럴)
 - value: 5

【 바이트코드 개념/장점 】

- 파이썬 소스코드가 파싱 후 생성하는 중간 표현(**Intermediate Representation**)
- **pyc** 파일에 저장되며, **PVM**에서 실행
- 실행 속도 향상 (한 번 생성 후 재사용 가능)
- 추상화된 명령어 수준으로 **OS**와 독립적으로 동작

확인 예제

```
python -m py_compile hello.py
```

```
__pycache__/hello.cpython-311.pyc 생성됨
```

Python_Virtual Machine (PVM)

【 역할 】

- 바이트코드를 하나씩 읽어 해석하고 실행하는 엔진
- 일종의 스택 기반 가상 머신

【 특징 】

- 각 명령은 스택 기반으로 동작
- 바이트코드는 opcode + 인자(argument) 구조

```
import dis

def add(x, y):
    ...return x + y

dis.dis(add) # 바이트코드 출력
```

↔	3	0 RESUME	0
	4	2 LOAD_FAST	0 (x)
		4 LOAD_FAST	1 (y)
		6 BINARY_OP	0 (+)
		10 RETURN_VALUE	

【 실행 파이프라인 】

- 소스코드를 바이트코드로 컴파일한 뒤 **Python Virtual Machine(PVM)**에서 실행하는 하이브리드 방식
- 이 과정을 이해하면 **Python**의 성능 특성과 **__pycache__**가 왜 생성되는지에 대한 이유를 확인 가능

단계	처리 내용	관련 모듈 / 처리내용
1. 소스 코드	py 파일 → Lexer (어휘 분석) → Token 스트림	개발자가 만든 py
2. 파싱	Token → AST() 생성, 문법 오류 감지	ast 모듈
3. 컴파일	AST → 바이트코드(.pyc) 생성, __pycache__ 저장	py_compile 모듈
4. PVM 실행	바이트코드를 opcode 단위로 해석/실행 (스택 기반)	Cpython 인터프리터
5. 결과 반환	실행 결과 출력, 예외 처리	프로그램 종료

Python 메모리 모델

【 메모리 관리 구조 】

- Stack: 함수 호출, 지역 변수 등
- Heap: 객체, 클래스 인스턴스, 리스트 등
- Reference Counting + Garbage Collector:
메모리 자동 해제

【 Python 객체 구조 】

- 모든 값은 객체(object) 로 표현됨
- 각 객체는 메타정보를 포함
 - type (자료형 정보)
 - refcount (참조 수)
 - value (데이터 값)

【 전략 】

전략	설명
Reference Counting	객체마다 참조 수 저장, 0이 되면 삭제
Cycle Detector	순환 참조 탐지 후 수동 해제
GC모듈	gc.collect() 등으로 수동 개입 가능

【 tip 】

개념	설명
__pycache__	바이트코드 캐싱 디렉토리
sys.getrefcount()	참조 수 확인
gc 모듈	가비지 컬렉터 제어
dis 모듈	바이트코드 분석

macOS + VS Code 개발 환경 설정 — 단계별 실습

macOS 기준 전체 환경 구성 흐름. 이 환경이 2일간 모든 실습의 기반

【 코드 예시 】

```
# 1. Python 3.11 설치 확인 (Homebrew)
$ brew install python@3.11
$ python3.11 --version      # Python 3.11.x

# 2. 프로젝트 폴더 생성 + venv
$ mkdir data-project && cd data-project
$ python3.11 -m venv .venv
$ source .venv/bin/activate # 활성화
(.venv) $                  # 프롬프트 변경 확인

# 3. 분석 패키지 한 번에 설치
→ .venv/ 는 .gitignore에 추가, requirements.txt는 반드시 커밋
```

실행 구조 이해의 목적

패키지 설치 오류 해결

- pip install 실패
- ModuleNotFoundError
 - venv 활성화 여부,
인터프리터 경로를 알아야 즉시 해결

에러 메시지 독해력

- Traceback은 PVM 실행 스택을 보여준다
- 스택 구조를 알면 오류 원인을
파일·라인·표현식 단위로 찾을 수 있음

성능 최적화의 출발점

- 바이트코드를 이해하면
왜 컴프리헨션이 빠른지
왜 Polars가 Pandas보다 빠른지
직관적으로 납득

후속 과목 연결

- LangChain·RAG·AI Agent는 모두 Python
런타임 위에서 동작. 환경 격리(venv)와
의존성 관리가 기본

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

- 제어, 연산, 반복, list · dict · set · deque · Counter · 컴프리헨션 · 제너레이터

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

제어문 (Control Flow Statements)

조건문 (if, elif, else)

```
if condition1:
    # 실행 코드
elif condition2:
    # 다른 조건
else:
    # 위 조건들 모두 거짓일 때 실행
```

- 조건식은 **bool** 평가 가능 객체로 자동 변환
- **None, 0, "", [], {}** 는 모두 **False**로 평가
- 조건문 안에서는 **Short-circuit Evaluation** 발생

-- Short-circuit Evaluation 사례

```
x = None
if x and x > 0:
    # x가 None이면 두 번째 조건은 평가 안 됨
    print("양수입니다.")
```

[참고] Short-circuit Evaluation

【 개념 】

- 논리 연산자 **and**, **or**를 사용 시
왼쪽 피연산자만 보고
전체 결과를 결정할 수 있으면,
오른쪽은 평가하지 않고 건너뛰는 동작



Python 평가 방식

연산자	동작 방식
and	왼쪽이 False ⇒ 전체는 무조건 False (오른쪽 생략)
or	왼쪽이 True ⇒ 전체는 무조건 True (오른쪽 생략)

```
def left():  
    print("left")  
    return False
```

```
def right():  
    print("right")  
    return True
```

```
print(left() and right()) # 'right' 출력 안 됨
```

left
False

- **left()**가 **False**니까 **and** 연산 전체도 **False**!
- **right()**는 호출조차 안 됨!

[참고] Short-circuit Evaluation_실제 활용

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

조건 + 안전한 연산 결합

```
if user and user.is_active:
```

```
# user가 None이면 .is_active는 AttributeError 발생하므로 단락 평가 필수
```

성능 최적화

```
if expensive_check() and cheap_check():
```

```
# cheap_check() 먼저 놓으면 성능 손해!
```

None 방지

```
value = config.get("key") or "default"
```

[참고] Short-circuit Evaluation_주의!

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

【 주의사항 】

- **and/or**는 항상 **bool**이 아닌 원래 값을 반환

```
print([] or [1])  # [1]  
print([] and [1]) # []
```

→ 이 때문에 **or**을 이용한 디폴트값 처리가 자주 사용

조건문 (if)

✓ 1. 조건문(if)?

- 특정 조건에 따라 코드 블록을 실행하는 데 사용
- 특정 조건이 참(True)일 때만 실행되는 코드이며 만약 조건이 거짓(False)이면 실행되지 않음

✓ 2. 주의사항

- 콜론(:)으로 조건문을 마무리
- 들여쓰기로 코드 블록을 구분
- 논리 연산자(and, or, not)를 사용하여 복잡한 조건 사용 가능
- in 키워드를 사용하여 포함 여부 확인 가능



기본 조건문

```
if 조건:
    실행할 코드
```

- if 뒤에는 조건을 작성
- 조건이 참(True)이면 아래 코드가 실행
- 조건이 거짓(False)이면 코드가 실행되지 않음

조건문 (if)

다양한 조건문(if)

· 가장 기본적인 if 문

```
age = 20 # 나이가 20살

if age >= 18: # 만약 나이가 18살 이상이라면
    print("성인입니다.") # 실행됨
```

· 실행되지 않는 경우

```
age = 16 # 나이가 16살

if age >= 18: # 나이가 18살 이상이 아니므로
    print("성인입니다.") # 실행되지 않음
```

· if-else 문 (조건이 거짓일 때 실행되는 코드 추가하기)

```
age = 16

if age >= 18:
    print("성인입니다.")
else:
    print("미성년자입니다.")
```

· if-elif-else 문 (여러 개의 조건 처리하기)

```
age = 10

if age >= 18:
    print("성인입니다.")
elif age >= 13:
    print("청소년입니다.")
else:
    print("어린이입니다.")
```

조건문 (if)

다양한 조건문(if)

• if 문에서 여러 조건 사용 (and, or, not)

```
age = 25
money = 10000
if age >= 18 and money >= 5000:
    print("영화를 볼 수 있습니다.") # 실행됨
```

```
age = 16
student_card = True

if age >= 18 or student_card:
    print("학생 할인 가능합니다.") # 실행됨
```

```
vip = False

if not vip:
    print("VIP가 아닙니다.") # 실행됨
```

• in과 not in 사용하기

```
fruits = ["apple", "banana", "cherry"]
if "banana" in fruits:
    print("바나나가 있습니다.") # 실행됨
if "grape" not in fruits:
    print("포도가 없습니다.") # 실행됨
```

• if 문 한 줄로 쓰기 (조건부 표현식)

```
age = 20
status = "성인" if age >= 18 else "미성년자"
print(status) # 출력: 성인
```

• pass 문 (아무것도 실행하지 않음)

```
age = 20
if age >= 18:
    pass # 아직 코드를 작성하지 않았지만 오류 방지
```


조건문(if)

다양한 조건문(if)

• if 문 중첩 사용하기

```
age = 25
money = 10000
if age >= 18:
    if money >= 5000:
        print("영화를 볼 수 있습니다.") # 실행됨
    else:
        print("돈이 부족합니다.")
else:
    print("나이가 너무 어려요.")
```

조건문	설명
if	• 조건이 참(True)일 때 실행
if-else	• 조건이 거짓(False)일 때 실행할 코드 포함
if-elif-else	• 여러 개의 조건을 처리할 때 사용
and, or, not	• 여러 조건을 조합할 때 사용
in, not in	• 리스트, 문자열에서 값이 포함되어 있는지 확인
if 한 줄 쓰기	• value = "값" if 조건 else "다른 값"
pass	• 실행할 코드가 없을 때 사용
중첩 if	• if 문 안에 또 다른 if 문 작성 가능

연산자

구분	연산자	설명
산술 연산자	+	• 더하기
	-	• 빼기;
	*	• 곱하기
	/	• 나누기
	//	• 몫
	%	• 나머지
	**	• 제곱
비교 연산자	==	• 같음
	!=	• 다름
	>	• 초과
	<	• 미만
	>=	• 이상
	<=	• 이하
논리 연산자	and	• 두 조건이 모두 참이면 true
	or	• 둘 중 하나라도 참이면 true
	not	• 참이면 거짓, 거짓이면 참

- **a == b**는 값 비교, **a is b**는 객체 ID 비교
- 비트 연산은 메모리 구조/하드웨어 다를 때 중요
- 정수는 내부적으로 **immutable**, 연산시 새 객체 생성

```
a = 10
b = a
print(a is b) # True
a += 1
print(a is b) # False (새 객체)
```

반복문 (Loops)

【 for 】

for 변수 in iterable:
반복 코드

【 while 】

while 조건:
반복 코드

사용 방법

- 리스트, 튜플, 문자열, range 객체, zip 객체, enumerate

```
for i, val in enumerate(["A", "B", "C"]):  
    print(i, val)
```

- 조건이 False가 될 때까지 반복break, continue, else 문장도 사용 가능

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)  
else:  
    print("정상 종료") # break 없이 끝나야 실행
```

반복문(for, while)

✓ 1. 반복문?

- 같은 코드를 여러 번 실행할 때 사용
- for, while이 대표적



사용 방법

```
for 변수 in 반복할_데이터:  
    실행할 코드
```

```
while 조건:  
    실행할 코드
```

반복문(for)

사용 방법 및 예시

• 기본 예제

```
fruits = ["사과", "바나나", "체리"]

for fruit in fruits:
    print(fruit)
```

→ 리스트(**fruits**)의 요소를 하나씩 꺼내서 **fruit**에 저장하고 출력

• range()를 이용한 숫자 반복

```
for i in range(5): # 0부터 4까지 반복 (5번)
    print(i)
```

```
for i in range(2, 6): # 2부터 5까지
    print(i)
```

```
for i in range(1, 10, 2): # 1부터 9까지, 2씩 증가
    print(i)
```

• for 문을 사용한 합 구하기

```
total = 0

for i in range(1, 6): # 1부터 5까지 합 구하기
    total += i

print("합:", total)
```

→ **total += i**는 **total = total + i**와 동일

• for 문과 enumerate()

```
fruits = ["사과", "바나나", "체리"]

for index, fruit in enumerate(fruits):
    print(index, fruit)
```

→ **enumerate()** : 인덱스와 값을 동시에 가져올 수 있음

반복문(while)

사용 방법 및 예시

• 기본 예제

```
count = 0

while count < 5:
    print(count)
    count += 1
```

→ **count**가 5보다 작을 동안 실행
count += 1이 없으면 무한 루프에 빠질 수 있음

• while 문을 사용한 합 구하기

```
total = 0
num = 1

while num <= 5:
    total += num
    num += 1

print("합:", total)
```

→ 결과: 합: 15
for 문과 동일한 결과지만,
while 문은 조건을 만족할 때까지 반복

반복문 제어 (break, continue)

break (반복문 강제 종료)

- **break**를 만나면 반복문이 즉시 종료

```
for i in range(10):  
    if i == 5:  
        break # i가 5일 때 반복문 종료  
    print(i)
```

continue (특정 조건에서만 건너뛰기)

- **continue**를 만나면 해당 반복만 건너뛰고 계속 실행

```
for i in range(5):  
    if i == 2:  
        continue # i가 2일 때만 건너뛰고 계속 진행  
    print(i)
```

반복문 (for, while) – 정리

개념	설명
for 문	• 리스트, 문자열, range() 등을 순회할 때 사용
while 문	• 특정 조건이 참일 동안 반복
break	• 반복문을 강제로 종료
continue	• 특정 조건에서만 반복 건너뛰기
range(n)	• 0부터 n-1까지 숫자 생성
enumerate()	• 리스트에서 인덱스와 값을 동시에 가져옴

【 리스트 컴프리헨션 】

- 반복문 축약

```
squares = [x*x for x in range(10) if x % 2 == 0]
```

【 제너레이터 기반 반복 (yield) 】

- yield는 단순한 return과 다르게
"제너레이터(generator)"라는 특별한 객체를
만드는 핵심 개념
- 함수의 실행을 잠시 멈췄다가, 나중에 다시
이어서 실행할 수 있게 만드는 키워드
⇒ 함수 ↔ 반복 가능한 객체로 변경

```
def fib(n):  
    a, b = 0, 1  
    for _ in range(n):  
        yield a  
        a, b = b, a + b
```

[참고] List와 Dictionary Comprehension 활용 반복문 간소화

리스트 컴프리헨션 (List Comprehension)

· 기본 for 문 vs 리스트 컴프리헨션

```
numbers = []
for i in range(1, 6):
    numbers.append(i)
print(numbers) # [1, 2, 3, 4, 5]
```

```
numbers = [i for i in range(1, 6)]
print(numbers) # [1, 2, 3, 4, 5]
```

· 조건 추가하기

```
even_numbers = [i for i in range(10) if i % 2 == 0]
print(even_numbers) # [0, 2, 4, 6, 8]
```

```
result = ["짝수" if i % 2 == 0 else "홀수" for i in range(1, 6)]
print(result) # ['홀수', '짝수', '홀수', '짝수', '홀수']
```

· 리스트 요소 변환하기

```
words = ["apple", "banana", "cherry"]
upper_words = [word.upper() for word in words]
print(upper_words) # ['APPLE', 'BANANA', 'CHERRY']
```

· 중첩 for 문 (이중 리스트)

```
pairs = [(x, y) for x in range(1, 3) for y in range(3, 5)]
print(pairs) # [(1, 3), (1, 4), (2, 3), (2, 4)]
```

[참고] List와 Dictionary Comprehension 활용 반복문 간소화

딕셔너리 컴프리헨션 (Dictionary Comprehension)

• 기본 for 문 vs 딕셔너리 컴프리헨션

```
squares = {}
for i in range(1, 6):
    squares[i] = i ** 2
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
squares = {i: i ** 2 for i in range(1, 6)}
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

• 조건 추가하기

```
even_squares = {i: i ** 2 for i in range(1, 6) if i % 2 == 0}
print(even_squares) # {2: 4, 4: 16}
```

```
num_type = {i: "짝수" if i % 2 == 0 else "홀수" for i in
range(1, 6)}
print(num_type) # {1: '홀수', 2: '짝수', 3: '홀수', 4: '짝수', 5: '홀수'}
```

• 기존 딕셔너리 변형하기

```
fruit_prices = {"apple": 1000, "banana": 500, "cherry": 2000}
discount_prices = {fruit: price * 0.9 for fruit, price in
fruit_prices.items()}
print(discount_prices) # {'apple': 900.0, 'banana': 450.0, 'cherry': 1800.0}
```

• 두 리스트를 딕셔너리로 변환하기

```
keys = ["name", "age", "city"]
values = ["Alice", 25, "Seoul"]

person = {k: v for k, v in zip(keys, values)}
print(person) # {'name': 'Alice', 'age': 25, 'city': 'Seoul'}
```

집합 컴프리헨션 (Set Comprehension)

• 리스트나 딕셔너리와 유사하지만, 중복을 자동으로 제거

```
numbers = [1, 2, 2, 3, 3, 4, 5]
unique_squares = {num ** 2 for num in numbers}
print(unique_squares) # {1, 4, 9, 16, 25}
```

리스트 vs 딕셔너리 컴프리헨션 – 정리

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

개념	예제	설명
리스트 생성	• <code>[i for i in range(5)]</code>	• <code>[0, 1, 2, 3, 4]</code>
리스트 + 조건	• <code>[i for i in range(10) if i % 2 == 0]</code>	• <code>[0, 2, 4, 6, 8]</code>
리스트 + 변환	• <code>[word.upper() for word in words]</code>	• 모든 단어 대문자로 변환
딕셔너리 생성	• <code>{i: i**2 for i in range(5)}</code>	• <code>{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}</code>
딕셔너리 + 조건	• <code>{i: i**2 for i in range(5) if i % 2 == 0}</code>	• 짝수만 포함
딕셔너리 변환	• <code>{k: v*2 for k, v in dict.items()}</code>	• 값 2배 증가

list – 가변 순차 자료형

List

- 특징
 - 순서 있음
 - 중복 허용
 - 인덱싱 및 슬라이싱 가능
 - 요소 수정 가능
- 내부구조: 동적 배열 (array list)
- 시간 복잡도
 - 접근: $O(1)$
 - 삽입/삭제
 - $O(n)$ (중간 삽입시)
 - $O(1)$ (맨 뒤에 삽입시, amortized)

예시코드

```
a = [1, 2, 3]
a.append(4)    # O(1)
a.insert(1, 100) # O(n)
print(a[2])    # O(1)
```

tuple – 불변 순차 자료형

tuple

- 특징
 - 리스트와 유사하지만 불변 (immutable)
- 사용처: 해싱 가능한 데이터로 사용 (dict 키나 set 요소)
- 성능상 장점: 메모리 사용량 적고, 리스트보다 빠름시간 복잡도

예시코드

```
b = (1, 2, 3)
# b[0] = 10 # 오류 발생
```

set – 중복 불가, 순서 없는 집합

tuple

- 특징
 - 중복 제거, 해시 기반 자료구조 (hash table)
- 주요 연산: 합집합 $|$, 교집합 $&$, 차집합 $-$
- 시간복잡도: 삽입/삭제/탐색 $O(1)$

예시코드

```
s = {1, 2, 3}
s.add(4)      # O(1)
s.remove(2)   # O(1)
print(3 in s) # True
```

dict – 키-값 쌍 해시맵

dict

- 특징
 - 키는 해시 가능 객체 (immutable)만 가능
- 내부 구조: 해시 테이블 기반
- Python 3.7+부터 삽입 순서 보장

예시코드

```
d = {'a': 1, 'b': 2}
d['c'] = 3
print(d['a'])    # O(1)
```


deque (from collections) – 양방향 큐

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

deque

- 특징
 - 리스트보다 양 끝 삽입/삭제가 빠름
- 사용처: 큐(Queue), 스택(Stack), 슬라이딩 윈도우 등
- 구현: **Doubly Linked List**

예시코드

```
from collections import deque

dq = deque([1, 2, 3])
dq.append(4)    # 오른쪽에 추가
dq.appendleft(0) # 왼쪽에 추가
dq.pop()        # 오른쪽 제거
dq.popleft()    # 왼쪽 제거
```

[참고] Doubly Linked List

deque는 “내부적으로 Doubly Linked List”라는 의미는 **C로 구현된 양방향 블록 연결 리스트**라는 뜻

【 차이점 】

항목	Doubly Linked List	Python deque 내부 구현
구조	node가 앞뒤로 연결	고정 크기의 배열 블록들을 앞뒤로 연결
구현	python 직접 구현 시 사용	python 내부 : C로 구현된 deque object에서 실행
장점	삽입/삭제는 빠르나 접근은 느림	양쪽 O(1) 삽입/삭제 + 블록 기반으로 접근도 효율적
사용처	커스텀 자료 구조 학습용	실전 성능 최적화된 Queue/Stack 용도

- deque는 단순한 Doubly Linked List가 아님
- 실제로는 "linked list of arrays" 구조
 - 각 블록이 일정 크기의 배열이며, 이 블록들이 양방향으로 연결됨
 - 이를 통해 cache-friendly 하면서도 O(1) 성능을 보장함.

고급 자료구조 (Python Advanced Data Structures)

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

✓ 1. 기본 자료 구조

- 리스트(List), 튜플(Tuple),
- 딕셔너리(Dictionary), 집합(Set)

더 빠르고 효율적인 데이터 관리를 위해 고급 자료구조를 활용

자료 구조

모듈들

- ✓ Deque
- ✓ Counter
- ✓ OrderedDict
- ✓ defaultdict

collections 모듈 활용

collections 모듈에는 기본 자료구조보다 더 효율적인 기능을 제공하는 클래스들이 포함

deque (빠른 삽입/삭제가 가능한 리스트)

- **list.append()/list.pop()**은 **O(1)**
리스트 앞쪽에서 삽입/삭제 시 **O(n)**의 성능 저하 발생
- 앞쪽에 삽입/삭제 하려면 전체를 뒤로 밀거나 앞으로 당겨야 함

- **deque(덱)**은 양쪽 끝에서 삽입/삭제가 **O(1)**로 매우 빠름

```
from collections import deque
```

- ```
queue = deque([1, 2, 3])
queue.append(4) # 오른쪽 추가
queue.appendleft(0) # 왼쪽 추가
print(queue) # deque([0, 1, 2, 3, 4])

queue.pop() # 오른쪽 요소 제거
queue.popleft() # 왼쪽 요소 제거
print(queue) # deque([1, 2, 3])
```
- **사용 사례:** 웹 브라우저 뒤로 가기 기능

## Counter (요소 개수 세기)

- 리스트, 튜플, 문자열에서 각 요소의 개수를 자동으로 세어주는 딕셔너리

```
from collections import Counter
```

```
text = "hello world"
count = Counter(text)
print(count) # {'h':1, 'e':1, 'l':3, 'o':2, ' ':1, 'w':1, 'r':1, 'd':1}
```

- **사용 사례:** 단어 빈도수 분석, 로그 데이터 분석

# collections 모듈 활용

**collections** 모듈에는 기본 자료구조보다 더 효율적인 기능을 제공하는 클래스들이 포함

## OrderedDict (순서 유지하는 딕셔너리)

- **Python 3.6** 이상에서는 기본 딕셔너리도 순서를 유지하지만, **OrderedDict**는 모든 버전에서 순서 유지

```
from collections import OrderedDict
```

```
ordered_dict = OrderedDict()
ordered_dict['apple'] = 5
ordered_dict['banana'] = 2
ordered_dict['orange'] = 3
```

```
print(ordered_dict) # OrderedDict([('apple', 5),
('banana', 2), ('orange', 3)])
```

- **JSON** 데이터 저장, 데이터 정렬

## defaultdict (기본값을 자동으로 설정하는 딕셔너리)

- 존재하지 않는 키를 조회할 때 자동으로 기본값을 설정

```
from collections import defaultdict
```

```
word_count = defaultdict(int) # 기본값 0
word_count["apple"] += 1
word_count["banana"] += 2
print(word_count) # {'apple': 1, 'banana': 2}
```

- 단어 빈도수 카운팅, 그래프 알고리즘

# heapq (힙: 우선순위 큐)

## heapq

- 최소/최대 값을 빠르게 찾는 자료구조
- `heapq`는 기본적으로 최소 힙(min-heap, 최소값이 루트)을 지원
- `heapq.heappush()`와 `heapq.heappop()`은  $O(\log N)$  (입력 크기  $N$ 이 늘어나도 연산회수가 조금씩만 증가)

### 최소 힙

```
import heapq

heap = []
heapq.heappush(heap, 5)
heapq.heappush(heap, 2)
heapq.heappush(heap, 8)
heapq.heappush(heap, 1)

print(heapq.heappop(heap)) # 1 (가장 작은 값이 제거됨)
print(heapq.heappop(heap)) # 2
```

다익스트라 알고리즘 (최단 경로), 우선순위 작업 스케줄링

### 최대 힙 (내림차순)

```
import heapq

max_heap = []
heapq.heappush(max_heap, -5)
heapq.heappush(max_heap, -1)
heapq.heappush(max_heap, -10)

print(-heapq.heappop(max_heap)) # 10
print(-heapq.heappop(max_heap)) # 5
```

가장 높은 점수 찾기, 데이터 정렬

# bisect (이진 탐색을 활용한 정렬된 리스트 관리)

## bisect

- 정렬된 리스트에서 특정 값을 빠르게 찾거나 삽입하는 라이브러리
- $O(\log N)$ 의 성능을 가지며, 매우 빠른 탐색이 가능

### 최소 힙

```
import bisect

numbers = [1, 3, 5, 7, 9]
pos = bisect.bisect_left(numbers, 6) # 6이 들어갈 위치 찾기
print(pos) # 3 (index)

bisect.insort(numbers, 6) # 리스트에 삽입 후 자동 정렬
print(numbers) # [1, 3, 5, 6, 7, 9]
```

정렬된 리스트에서 빠른 검색, 자동 정렬 데이터 저장

# 고급 자료 구조 비교표

## 2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

| 자료구조               | 모듈            | 특징                              | 사용 예제         |
|--------------------|---------------|---------------------------------|---------------|
| <b>deque</b>       | • collections | • 빠른 삽입/삭제 ( $O(1)$ )           | • 웹 브라우저 기록   |
| <b>Counter</b>     | • collections | • 요소 개수 카운팅                     | • 단어 빈도 분석    |
| <b>OrderedDict</b> | • collections | • 삽입 순서 유지                      | • JSON 데이터 처리 |
| <b>defaultdict</b> | • collections | • 기본값 자동 설정                     | • 그래프 알고리즘    |
| <b>heapq</b>       | • heapq       | • 우선순위 큐 ( $O(\log N)$ )        | • 다익스트라 알고리즘  |
| <b>bisect</b>      | • bisect      | • 정렬된 리스트 이진 탐색 ( $O(\log N)$ ) | • 빠른 검색       |
| <b>set</b>         | • 내장          | • 중복 제거, 집합 연산                  | • 데이터 필터링     |



# defaultdict (from collections) – 초기값을 자동 제공하는 딕셔너리

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

## defaultdict

- 특징
  - 존재하지 않는 키 접근 시 KeyError 대신 기본값 생성
- 사용처: 그룹핑, 카운팅 등에 유용

## 예시코드

```
from collections import defaultdict

dd = defaultdict(list) # list() → 기본값은 빈 리스트 []
dd['a'].append(1) # 존재하지 않는 'a'를 읽으려고 하므로,
 # 자동으로 dd['a'] = [] 후 append
dd['a'].append(2) # dd['a']는 이제 [1], 거기에 2 추가
 # → [1, 2]

print(dd) # 결과: defaultdict(<class 'list'>, {'a': [1, 2]})
```

# Counter (from collections) – 요소 개수 세기

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

## Counter

- 특징
  - dict 기반 빈도 수 계산
- 사용처: 통계, 자연어 처리 등

## 예시코드

```
from collections import Counter

cnt = Counter(['a', 'b', 'a', 'c', 'b', 'a'])
print(cnt) # Counter({'a': 3, 'b': 2, 'c': 1})
print(cnt['a']) # 3
print(cnt.most_common(2)) # [('a', 3), ('b', 2)]
```

- **most\_common(n)** 메서드는 가장 많이 등장한 n개의 요소를 (요소, 개수) 튜플의 리스트로 반환

# 자료구조별 요약 비교

【 자료 구조별 요약 비교 】

| 구조                 | 순서 | 중복 | 변경 가능 | 주요 장점           | 시간복잡도(탐색) |
|--------------------|----|----|-------|-----------------|-----------|
| <b>list</b>        | O  | O  | O     | 인덱싱/슬라이싱        | O(1)      |
| <b>tuple</b>       | O  | O  | X     | 해시 가능, 불변성      | O(1)      |
| <b>set</b>         | X  | X  | O     | 중복 제거, 집합 연산 빠름 | O(1)      |
| <b>dict</b>        | O* | X  | O     | 키-값 쌍 지정        | O(1)      |
| <b>deque</b>       | O  | O  | O     | 양방향 삽입/삭제 O(1)  | O(n)      |
| <b>defaultdict</b> | O* | X  | O     | 자동 초기화 딕셔너리     | O(1)      |
| <b>Counter</b>     | O  | O  | O     | 빠른 빈도 계산        | O(1)      |

일반적으로는 O(1)이지만, 해시 충돌이나 리사이징 등으로 인해 최악의 경우 성능이 저하될 수 있음을 의미  
(예: O(n)이 될 수도 있음)

# [Why?] 리스트의 앞쪽에서 삽입/삭제 시 $O(n)$ 의 성능 저하가 발생

리스트(list)는 동적 배열(Dynamic Array) 구조를 사용하므로 끝에서 추가하거나 제거하는 것은 매우 빠름 ( $O(1)$ )

## 리스트 끝에서 추가/삭제 ( $O(1)$ )

```
my_list = [1, 2, 3]

my_list.append(4) # 리스트의 끝에 4 추가 ($O(1)$)
print(my_list) # [1, 2, 3, 4]

my_list.pop() # 리스트의 끝 요소 제거 ($O(1)$)
print(my_list) # [1, 2, 3]
```

- 리스트의 끝에 새로운 요소를 추가하는 **append()**는 배열 끝에 바로 저장하므로  **$O(1)$**
- **pop()**도 마지막 요소를 제거하는 것이라서  **$O(1)$**

## 리스트의 앞쪽에서 추가/삭제 ( $O(n)$ )

```
my_list = [1, 2, 3]

my_list.insert(0, 0) # 리스트의 맨 앞에 0 삽입 ($O(n)$)
print(my_list) # [0, 1, 2, 3]

my_list.pop(0) # 리스트의 맨 앞 요소 제거 ($O(n)$)
print(my_list) # [1, 2, 3]
```

- 첫 번째 요소를 삭제하면, 모든 요소가 한 칸씩 앞으로 이동해야 함
- 요소가 많을수록 이동해야 하는 연산이 많아져서 성능이 나빠짐

\*  **$O(1)$ 의 의미:** "상수 시간(Constant Time) 연산으로 연산이 수행되는 시간이 입력 크기( $n$ )에 관계없이 일정하다는 의미

## [참고] deque는 양쪽 끝에서 삽입/삭제가 $O(1)$ 이다!

`collections.deque`(덱)는 "이중 연결 리스트(Double Linked List)" 형태, 양쪽 끝에서 삽입/삭제 시  $O(1)$ 의 성능을 유지

### 리스트 끝에서 추가/삭제 ( $O(1)$ )

```
from collections import deque

dq = deque([1, 2, 3])

dq.append(4) # 오른쪽 끝에 추가 ($O(1)$)
print(dq) # deque([1, 2, 3, 4])

dq.appendleft(0) # 왼쪽 끝에 추가 ($O(1)$)
print(dq) # deque([0, 1, 2, 3, 4])

dq.pop() # 오른쪽 끝 요소 제거 ($O(1)$)
print(dq) # deque([0, 1, 2, 3])

dq.popleft() # 왼쪽 끝 요소 제거 ($O(1)$)
print(dq) # deque([1, 2, 3])
```

- 연결 리스트 구조이므로 요소를 추가/삭제할 때 다른 요소를 이동시킬 필요가 없음
- 양쪽 끝에서 추가/삭제하는 연산은 항상  $O(1)$

### ★ $O(1)$ vs. $O(n)$ 비교

| 연산                      | 수행시간     | 설명                         |
|-------------------------|----------|----------------------------|
| <b>append()</b>         | • $O(1)$ | • 항상 리스트 끝에 추가하므로 실행시간 일정  |
| <b>pop()</b>            | • $O(1)$ | • 항상 리스트 끝에서 삭제하므로 실행시간 일정 |
| <b>insert(0, value)</b> | • $O(n)$ | • 리스트 앞에 추가하면 모든 요소가 이동    |
| <b>pop(0)</b>           | • $O(n)$ | • 리스트 앞에서 삭제하면 모든 요소가 이동   |

# [참고] $O(\log N)$

데이터의 크기( $N$ )가 증가해도 연산 시간이 아주 천천히 증가하는 알고리즘으로 데이터가 많아질수록 실행 시간이 조금씩만 증가하는 효율적인 알고리즘

## $O(\log N)$

- $O(\log N)$ 의 핵심 개념
  - 데이터를 한 번에 하나씩 처리하는 게 아니라 반씩 줄여가면서 처리
  - 즉, 매 단계마다 데이터 크기가 절반으로 감소
  - 예시: 이진 탐색(Binary Search), 힙(heap), 균형 이진 트리(BST)

### 이진 탐색 구현

```
def binary_search(arr, target):
 left, right = 0, len(arr) - 1
 while left <= right:
 mid = (left + right) // 2 # 중간 값 찾기
 if arr[mid] == target:
 return mid # 찾으면 위치 반환
 elif arr[mid] < target:
 left = mid + 1 # 오른쪽 탐색 (데이터 절반 제거)
 else:
 right = mid - 1 # 왼쪽 탐색 (데이터 절반 제거)
 return -1 # 값이 없으면 -1 반환

정렬된 리스트에서 7의 위치 찾기
numbers = [1, 3, 5, 7, 9, 11, 13]
print(binary_search(numbers, 7)) # 3 (index)
```

데이터가 커져도 연산 횟수가 크게 증가하지 않음

# [참고] zip()

## ✓ 1. zip()?

- 여러 개의 리스트(또는 튜플)를 하나의 튜플 묶음으로 반환



## 사용 방법

- 두 리스트 묶기 : 각 인덱스별로 (이름, 나이) 형태의 튜플로 묶임

```
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]

zipped = zip(names, ages)
print(list(zipped)) # [('Alice', 25), ('Bob', 30), ('Charlie', 35)]
```

- for 문과 함께 사용 : 여러 개의 리스트를 한 번에 순회 가능

```
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]

for name, age in zip(names, ages):
 print(f"{name}의 나이는 {age}살입니다.")
```

# [참고] zip()을 활용한 다양한 예제

## 예제

### • zip()을 이용한 딕셔너리 생성

```
keys = ["name", "age", "city"]
values = ["Alice", 25, "Seoul"]

person = dict(zip(keys, values))
print(person) # {'name': 'Alice', 'age': 25, 'city': 'Seoul'}
```

→ 두 개의 리스트를 딕셔너리로 변환

### • zip()을 활용한 리스트 언패킹

```
pairs = [("Alice", 25), ("Bob", 30), ("Charlie", 35)]

names, ages = zip(*pairs) # 언패킹
print(names) # ('Alice', 'Bob', 'Charlie')
print(ages) # (25, 30, 35)
```

→ **zip(\*리스트)**를 사용하면 튜플을 다시 개별 리스트로 분리 가능

### • zip()을 사용한 리스트 정렬

```
students = ["Alice", "Bob", "Charlie"]
scores = [90, 80, 95]

sorted_students = [name for name, _ in
sorted(zip(scores, students), reverse=True)]
print(sorted_students) # ['Charlie', 'Alice', 'Bob']
```

→ 점수를 기준으로 학생 이름을 정렬하고  
**zip(scores, students) → (점수, 이름) 튜플로 묶음**  
**sorted()**를 사용해 점수를 기준으로 정렬



# [참고] zip() 사용 시 주의 사항

## 길이가 다른 리스트를 zip()하면

### • zip()을 이용한 딕셔너리 생성

```
a = [1, 2, 3]
b = ["one", "two"]

print(list(zip(a, b))) # [(1, 'one'), (2, 'two')]
```

→ 짧은 리스트에 맞춰서 짝이 맞는 요소까지만 묶임  
3은 짝이 없어서 결과에서 제외됨

```
from itertools import zip_longest

a = [1, 2, 3]
b = ["one", "two"]

print(list(zip_longest(a, b, fillvalue="없음")))
[(1, 'one'), (2, 'two'), (3, '없음')]
```

→ 만약 부족한 값은 None으로 채우고 싶다면  
zip\_longest() 사용

# [참고] zip() 핵심 정리

| 기능               | 사용법                                           | 결과                                  |
|------------------|-----------------------------------------------|-------------------------------------|
| 리스트 묶기           | • <code>zip([1, 2], ['a', 'b'])</code>        | • <code>[(1, 'a'), (2, 'b')]</code> |
| <b>for</b> 문과 사용 | • <code>for x, y in zip(a, b):</code>         | • 여러 리스트 동시 순회                      |
| 딕셔너리 변환          | • <code>dict(zip(keys, values))</code>        | • <code>{key: value, ...}</code>    |
| 언패킹              | • <code>zip(*zipped_list)</code>              | • 리스트 다시 분리                         |
| 정렬               | • <code>sorted(zip(scores, names))</code>     | • 특정 기준으로 정렬                        |
| 길이 맞추기           | • <code>zip_longest(a, b, fillvalue=0)</code> | • 부족한 값 <code>fillvalue</code> 로 채움 |

# [참고] zip\_longest()

## ✓ 1. zip\_longest()?

- 리스트(이터러블) 길이가 다를 때, 짧은 리스트를 자동으로 채워주는 zip()의 확장 버전
- 부족한 값을 fillvalue로 채움



## 사용 방법

- zip\_longest()는 itertools 모듈에서 제공  
→ import itertools 필요

```
from itertools import zip_longest
```

```
zip_longest(리스트1, 리스트2, ..., fillvalue=채울값)
```

# [참고] zip() vs zip\_longest() 차이점

## 예제

### • zip() 사용 시 짧은 리스트에 맞춤

```
a = [1, 2, 3]
b = ["one", "two"]

print(list(zip(a, b)))
결과: [(1, 'one'), (2, 'two')] ← (3, ?) 없음
```

→ zip()은 짝이 맞는 요소까지만 반환하고 나머지는 버려짐

### • zip\_longest() 사용 시 부족한 부분을 채움

```
from itertools import zip_longest

a = [1, 2, 3]
b = ["one", "two"]

print(list(zip_longest(a, b, fillvalue="없음")))
결과: [(1, 'one'), (2, 'two'), (3, '없음')]
```

→ 길이가 짧은 리스트를 fillvalue="없음"으로 채움

# [참고] zip\_longest() 다양한 활용

## 예제

### • 여러 리스트를 같은 길이로 맞추기

```
names = ["Alice", "Bob"]
scores = [90, 85, 80]

result = list(zip_longest(names, scores, fillvalue="정보 없음"))
print(result)
결과: [('Alice', 90), ('Bob', 85), ('정보 없음', 80)]
```

→ **names**가 부족하므로 "정보 없음"이 자동 추가됨

### • 딕셔너리 변환 시 활용

```
keys = ["name", "age"]
values = ["Alice"]

person = dict(zip_longest(keys, values, fillvalue="Unknown"))
print(person)
결과: {'name': 'Alice', 'age': 'Unknown'}
```

→ **values**의 요소가 부족해서 "Unknown"으로 채워짐

# [참고] zip\_longest() 다양한 활용

## 예제

### • CSV 데이터 정렬 시 활용

```
columns = ["이름", "나이", "도시"]
rows = [
 ["Alice", 25],
 ["Bob", 30, "Seoul"],
 ["Charlie"]
]

formatted_rows = [list(zip_longest(columns, row, fillvalue="정보 없음")) for row in rows]

for row in formatted_rows:
 print(dict(row))
```

```
{'이름': 'Alice', '나이': 25, '도시': '정보 없음'}
{'이름': 'Bob', '나이': 30, '도시': 'Seoul'}
{'이름': 'Charlie', '나이': '정보 없음', '도시': '정보 없음'}
```

→ 데이터가 일부 비어 있는 경우, `zip_longest()`를 사용하면 자동으로 `None` 또는 기본값을 채울 수 있음

# [참고] zip\_longest() 핵심 정리

| 기능          | zip()                                                                                            | zip_longest()                                                                                                                     |
|-------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 리스트 길이 다를 때 | <ul style="list-style-type: none"> <li>짧은 리스트 기준으로 맞춤</li> </ul>                                 | <ul style="list-style-type: none"> <li>긴 리스트 기준으로 맞추고 부족한 부분 채움</li> </ul>                                                        |
| 기본 동작       | <ul style="list-style-type: none"> <li>zip([1,2,3], ['a','b']) → [(1, 'a'), (2, 'b')]</li> </ul> | <ul style="list-style-type: none"> <li>zip_longest([1,2,3], ['a','b'], fillvalue='X') → [(1, 'a'), (2, 'b'), (3, 'X')]</li> </ul> |
| fillvalue   | <ul style="list-style-type: none"> <li>없음</li> </ul>                                             | <ul style="list-style-type: none"> <li>기본값 설정 가능 (fillvalue="없음")</li> </ul>                                                      |
| 딕셔너리 변환     | <ul style="list-style-type: none"> <li>dict(zip(keys, values))</li> </ul>                        | <ul style="list-style-type: none"> <li>dict(zip_longest(keys, values, fillvalue='Unknown'))</li> </ul>                            |

# 문자열 슬라이싱 (Slicing)

## ✓ 문자열 Slicing

- 인덱스를 사용하여 문자열의 특정 부분을 추출하는 방법
- **str[start:end:step]** 형태로 사용



## 사용 방법

### • 리스트 형식

```
text = "Hello, Python!"
print(text[0:5]) # "Hello"
print(text[:5]) # "Hello" (처음부터 5까지)
print(text[7:]) # "Python!" (7부터 끝까지)
print(text[::-1]) # "!nohtyP ,olleH" (문자열 뒤집기)
```



# 문자열 포매팅 (Formatting)

## ✔ 문자열 Formatting

- % 연산자 활용
- .format() 메서드 활용
- f-string (Python 3.6 이상)



## 사용 방법

### • 다양한 방식으로 사용

```
name = "Alice"
age = 25

% 연산자
print("이름: %s, 나이: %d" % (name, age))

format() 메서드
print("이름: {}, 나이: {}".format(name, age))
print("이름: {0}, 나이: {1}".format(name, age)) # 인덱스 사용
print("이름: {name}, 나이: {age}".format(name=name,
age=age)) # 키워드 사용

f-string (가장 현대적인 방법)
print(f"이름: {name}, 나이: {age}")
```

# 슬라이싱 기본 문법 및 Formatting

문자열[start:end:step] 형태로 사용하고 Formatting은 변수나 값을 문자열에 삽입하는 방법

## 기본 문법

### · 사용 방법

```
text = "Hello, Python!"

0부터 4번째 문자까지 (끝 인덱스는 포함되지 않음)
print(text[0:5]) # 'Hello'

처음부터 5번째 문자까지
print(text[:5]) # 'Hello'

7번째 문자부터 끝까지
print(text[7:]) # 'Python!'

문자열 전체 복사
print(text[:]) # 'Hello, Python!'

step 활용 (2칸씩 건너뛰기)
print(text[::2]) # 'Hlo yhn'

문자열 뒤집기
print(text[::-1]) # '!nohtyP ,olleH'
```

## Formatting

### · % 연산자 사용 (옛날 방식)

- %s → 문자열, %d → 정수, %f → 실수

```
name = "Alice"
age = 25
print("이름: %s, 나이: %d" % (name, age))
이름: Alice, 나이: 25
```

# 슬라이싱 기본 문법 및 Formatting

## Formatting

### • format() 함수 사용 (Python 3부터)

```
name = "Alice"
age = 25
print("이름: {}, 나이: {}".format(name, age))
결과: 이름: Alice, 나이: 25

인덱스 사용
print("이름: {0}, 나이: {1}".format(name, age))
결과: 이름: Alice, 나이: 25

변수명 직접 사용
print("이름: {name}, 나이: {age}".format(name="Alice",
age=25))
결과: 이름: Alice, 나이: 25
```

### • f-string 활용 (최신 방식, Python 3.6 이상)

- 간단한 연산, 소수점 자리수 지정, 정렬/공백 추가 가능

```
name = "Alice"
age = 25
print(f"이름: {name}, 나이: {age}")
결과: 이름: Alice, 나이: 25

간단한 연산 가능
a = 5
b = 3
print(f"5 + 3 = {a + b}")
결과: 5 + 3 = 8
소수점 자리 지정 가능

pi = 3.14159
print(f"파이 값: {pi:.2f}") # 소수점 둘째 자리까지 출력
결과: 파이 값: 3.14

정렬/공백 지정 가능
text = "Python"
print(f"[{text:>10}]") # 오른쪽 정렬 (10칸)
print(f"[{text:<10}]") # 왼쪽 정렬 (10칸)
print(f"[{text:^10}]") # 가운데 정렬 (10칸)
```

# 제너레이터 — 대용량 메모리 효율 처리

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

## 【 코드 예시 】

### 핵심 내용

- ✓ **yield:** 값을 하나씩 생성, 일시 중단
- ✓ 전체 데이터를 메모리에 올리지 않음
- ✓ **CSV** 수백만 행 스트리밍 처리에 핵심
- ✓ **generator expression:** () 괄호 사용
- ✓ **itertools.islice / chain** 등과 조합

```
제너레이터 실전
제너레이터 함수
def read_rows(path):
 with open(path) as f:
 next(f) # 헤더 스킵
 for line in f:
 yield line.strip().split(',')

메모리 절약: 한 번에 한 행씩
for row in read_rows('large.csv'):
 process(row)

generator expression
total = sum(float(r[2]) for r in read_rows('f.csv'))

리스트 vs 제너레이터 메모리 비교
import sys
lst = [x**2 for x in range(10_000_000)] # 400MB+
gen = (x**2 for x in range(10_000_000)) # ~120 byte
```

# dataclass · TypedDict — 데이터 레코드 모델링

## 핵심 내용

- ✓ **dataclass**  
: 클래스 보일러플레이트 자동 생성
- ✓ **\_\_init\_\_** · **\_\_repr\_\_** · **\_\_eq\_\_** 자동 구현
- ✓ **field()**로 기본값·팩토리 설정
- ✓ **TypedDict** : dict에 타입 힌트 추가
- ✓ **Pandas DataFrame** 행을 타입  
안전하게 표현

## 【 코드 예시 】

```
from dataclasses import dataclass, field
from typing import TypedDict, Optional
```

```
@dataclass
class SalesRecord:
 date: str
 region: str
 amount: float
 tags: list = field(default_factory=list)
```

```
r = SalesRecord('2024-01', '서울', 1500.0)
print(r) # SalesRecord(date='2024-01', ...)
```

```
TypedDict: dict에 타입 힌트
class Row(TypedDict):
 region: str
 sales: float
 category: Optional[str]
```

# 자료구조가 데이터 분석 코드의 기반인 이유

## DataFrame은 dict의 확장

- Pandas DataFrame 내부는 dict[컬럼명 → numpy 배열]
- dict를 이해하면 Pandas 동작 원리가 보임.

## 컴프리헨션 = 빠른 전처리

- 필터링·변환을 한 줄로 표현
- Pandas apply 대비 간단한 변환은 컴프리헨션이 더 빠를 때도 있음

## Counter로 value\_counts 대체

- Pandas 없이도 Counter 하나로 빈도 집계 가능
- 스크립트 레벨 분석에서 의존성 최소화.

## 제너레이터 = 스트리밍 파이프라인

- 수백만 행 CSV를 한꺼번에 올리지 않고 행 단위로 처리
- 메모리 한계를 극복하는 핵심 패턴.

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

# [Practice 1]

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 리스트 + 딕셔너리 기반 데이터 필터링기

## 실습 목표

### ▪ 목표

- 리스트 안에 있는 딕셔너리 구조에서 조건에 맞는 데이터를 필터링
- for, if, list comprehension, filter() 등 Python의 필터링 기법 체득

## 【 실습 과제 】

- 다음은 직원(Employee)들의 정보를 담은 리스트입니다.
- 이 데이터를 활용하여 아래 조건을 만족하는 필터링 로직을 작성하세요.

```
employees = [
 {"name": "Alice", "department": "Engineering", "age": 30, "salary": 85000},
 {"name": "Bob", "department": "Marketing", "age": 25, "salary": 60000},
 {"name": "Charlie", "department": "Engineering", "age": 35, "salary": 95000},
 {"name": "David", "department": "HR", "age": 45, "salary": 70000},
 {"name": "Eve", "department": "Engineering", "age": 28, "salary": 78000},
]
```



# 리스트 + 딕셔너리 기반 데이터 필터링기

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

## 【 실습 시나리오 】

| 문제 | 실습 내용                                                     |
|----|-----------------------------------------------------------|
| 1  | 부서가 "Engineering"이고 salary >= 80000인 직원들의 이름만 리스트로 출력하세요. |
| 2  | 30세 이상인 직원의 이름과 부서를 튜플 (name, department) 형태로 리스트로 출력하세요. |
| 3  | 급여 기준으로 직원 리스트를 salary 내림차순으로 정렬하고, 상위 3명의 이름과 급여를 출력하세요. |
| 4  | 모든 부서별 평균 급여를 출력하는 코드를 작성해보세요.                            |

# 평가 기준

## 캠퍼스명\_반\_이름. py로 제출 / Practice Rule (총점 100점)

### Code의 Comm. (20점)

- Python Code를 통한 이해관계자와의 Comm.
  - 프로그램 전체 설명 및 변경내역(머리말)
  - 함수/기능 설명(중간)

### 코드 간결성 (35점)

- 불필요한 반복 지양

### 오류/예외 처리 (35점)

- 예외, 오류에 대한 코드 반영 여부  
(간단한 코드라도 오류/예외 처리 연습 목적)

### 납기(프로젝트 연습, 10점)

- 기한 내 납기
  - 1시간 내 제출 완료

# 리스트 + 딕셔너리 기반 데이터 필터링기

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

## 【 실습 시나리오 – 예시 】

| 문제 | 실습 내용                                                                                                                                                                                                                                                                      |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <p>부서가 "Engineering"이고 salary &gt;= 80000인 직원들의 이름만 리스트로 출력하세요.</p> <pre>eng_high_salary = [emp["name"] for emp in employees                     if emp["department"] == "Engineering" and emp["salary"] &gt;= 80000]  print(eng_high_salary) # ['Alice', 'Charlie']</pre> |

리스트 컴프리헨션으로 조건에 맞는 딕셔너리만 추출 - **emp["key"]** 방식으로 딕셔너리 값 접근

# 리스트 + 딕셔너리 기반 데이터 필터링기

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

## 【 실습 시나리오 – 예시 】

| 문제 | 실습 내용                                                                                                                                                                                                                                                           |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2  | <p>30세 이상인 직원의 이름과 부서를 튜플 (name, department) 형태로 리스트로 출력하세요.</p> <pre>over_30 = [(emp["name"], emp["department"]) for emp in employees if emp["age"] &gt;= 30]  print(over_30)  # [('Alice', 'Engineering'), ('Charlie', 'Engineering'), ('David', 'HR')]</pre> |

튜플 형태 (name, department)로 반환 - 이 구조는 종종 테이블 구조와 매핑될 수 있음

# 리스트 + 딕셔너리 기반 데이터 필터링기

2. 자료구조와 제어문, 연산자, 반복문 및 컴프리헨션

## 【 실습 시나리오 – 예시 】

| 문제 | 실습 내용                                                                                                                                                                                                                                                                                                 |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3  | <p>급여 기준으로 직원 리스트를 salary 내림차순으로 정렬하고, 상위 3명의 이름과 급여를 출력하세요.</p> <pre>sorted_by_salary = sorted(employees, key=lambda x: x["salary"], reverse=True) top3 = [(emp["name"], emp["salary"]) for emp in sorted_by_salary[:3]]  print(top3) # [('Charlie', 95000), ('Alice', 85000), ('Eve', 78000)]</pre> |

**sorted()**는 **key**와 **reverse**를 조합해서 다중 조건 정렬도 가능  
**S/W 필터링**과 정렬 알고리즘 구현에서 유용

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## [심화 실습]

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# [심화 실습] 자료구조 집계 · 컴프리헨션 · 제너레이터

## Python\_Practice1\_Data.json 활용

### 1) 리스트/딕셔너리 컴프리헨션

- Python\_Practice1\_Data.json(Sales) 리스트에서
  - ① amount  $\geq$  1000인 거래만 필터링하고,
  - ② 지역별 총매출 dict를 컴프리헨션으로 계산

### 2) Counter + defaultdict

- Counter로 지역별 거래 건수를,  
defaultdict로 카테고리별 amount 리스트

### 3) 제너레이터 — 메모리 비교

- amount > 1000 인 행만 yield 하는 제너레이터를 작성하고,  
리스트 버전과 메모리 크기를 비교

### 4) 종합 - 월별 카테고리 매출 집계

- sales 데이터를 month·category 기준으로 그룹핑해 총매출 dict를 완성  
(컴프리헨션 + defaultdict)

# [심화 실습] 자료구조 집계 · 컴프리헨션 · 제너레이터

## Checkpoint

- region\_total 값 정확 (assert 통과)
- Counter.most\_common() 순서 정확
- generator sys.getsizeof < list 확인
- top3 금액 내림차순 정렬 정확

## 감점 대상 (감점점수)

- 컴프리헨션 대신 for 루프만 사용 (-2)
- defaultdict 대신 if key not in dict 패턴 (-1)
- 제너레이터를 list로 변환해 비교 (메모리 차이 미확인, -2)
- Counter 대신 직접 루프로 카운팅 (-1)



# [실습 1] 자료구조 집계 · 컴프리헨션 · 제너레이터

## Python\_Practice1\_Data.json 활용

### 1) 리스트/딕셔너리 컴프리헨션

- Python\_Practice1\_Data.json(Sales) 리스트에서
  - ① amount  $\geq$  1000인 거래만 필터링하고,
  - ② 지역별 총매출 dict를 컴프리헨션으로 계산

### 2) Counter + defaultdict

- Counter로 지역별 거래 건수를,  
defaultdict로 카테고리별 amount 리스트

### 3) 제너레이터 — 메모리 비교

- amount > 1000 인 행만 yield 하는 제너레이터를 작성하고,  
리스트 버전과 메모리 크기를 비교

### 4) 종합 - 월별 카테고리 매출 집계

- sales 데이터를 month·category 기준으로 그룹핑해 총매출 dict를 완성  
(컴프리헨션 + defaultdict)

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 3. 함수 · 파일 · 예외 처리

- **functools · 데코레이터 · pathlib · CSV/JSON/Parquet · logging**

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 함수 정의와 호출

## ✓ 1. 함수?

- 특정 작업을 수행하는 코드 블록
- 재사용이 가능

## ✓ 2. 기본 사용법 (def로 함수 정의)

예시

```
def 함수이름(매개변수):
 # 함수 내용
 return 결과
```



## 사용 예시

```
def greet(name):
 return f"안녕하세요, {name}님!"

함수 호출
message = greet("철수")
print(message) # 출력: 안녕하세요, 철수님!
```

# 매개변수와 반환값

## 함수의 매개변수 (Parameters)

- 매개변수를 통해 값을 입력받아 동작

```
def greet(name):
 return f"안녕하세요, {name}님!"

print(greet("철수")) # 출력: 안녕하세요, 철수님!
```

→ name이라는 매개변수를 받아서 문자열을 반환하는 함수

## 함수의 반환값 (Return Value)

- return 키워드를 사용해 결과를 반환
- 반환값이 없을 경우 None이 기본값

```
def add(a, b):
 return a + b # 결과 반환

result = add(3, 5)
print(result) # 출력: 8
```

→ return을 사용하면 함수의 결과값을 외부에서 활용

# 매개변수의 기본값 설정

## 기본값 설정 시 함수 호출 시 해당 인자를 생략

### • 기본값 설정

```
def introduce(name, age=20):
 print(f"이름: {name}, 나이: {age}")
```

```
introduce("영희") # 출력: 이름: 영희, 나이: 20
introduce("철수", 25) # 출력: 이름: 철수, 나이: 25
```

→ age=20이 기본값이므로 introduce("영희")처럼 호출해도 정상 작동

## 기본값이 있는 매개변수는 항상 뒤쪽에

### • 기본값 인자는 항상 뒤쪽에 위치

```
오류 발생! (기본값 있는 인자는 뒤에 와야 함)
def wrong_example(age=20, name):
 print(f"이름: {name}, 나이: {age}")
```

# 기본값이 있는 인자는 항상 마지막에 배치

```
def correct_example(name, age=20):
 print(f"이름: {name}, 나이: {age}")
```

# 함수 고급 패턴 — \*args, \*\*kwargs, 클로저

## 【 코드 예시 】

### 핵심 내용

- ✓ **\*args**  
: 가변 위치 인자 (tuple로 저장)
- ✓ **\*\*kwargs**  
: 가변 키워드 인자 (dict로 저장)
- ✓ 클로저: 내부 함수가 외부 함수 변수 기억
- ✓ **lambda**: 간단한 한 줄 변환 함수
- ✓ **functools.partial**: 인자 고정해 새 함수 생성

```
함수 고급 패턴
*args, **kwargs
def agg(*args, **kwargs):
 print(f'값: {args}') # tuple
 print(f'옵션: {kwargs}') # dict
agg(1,2,3, method='mean')

클로저: 외부 변수 기억
def make_multiplier(n):
 def multiply(x): return x * n # n 기억
 return multiply
double = make_multiplier(2)
double(5) # 10

partial: 인자 고정
from functools import partial
def scale(x, factor): return x * factor
double2 = partial(scale, factor=2)
list(map(double2, [1,2,3,4])) # [2,4,6,8]
```

# 가변 인자 (\*args, \*\*kwargs)

함수를 호출할 때, 몇 개의 인자를 받을지 미리 정할 수 없을 때 가변 인자를 사용

## \*args (위치 기반 가변 인자)

- 여러 개의 값을 입력받을 때, \*args로 하면 튜플로 저장

```
def sum_numbers(*args): # 여러 개의 숫자를 받을 수 있음
```

```
 return sum(args)
```

```
print(sum_numbers(1, 2, 3, 4, 5)) # 출력: 15
```

```
print(sum_numbers(10, 20)) # 출력: 30
```

→ sum\_numbers(1, 2, 3, 4, 5)처럼 여러 개의 숫자를  
넣으면 args=(1, 2, 3, 4, 5) 튜플로 저장

- 반복문을 사용해서 \*args 처리 가능

```
def print_names(*args):
```

```
 for name in args:
```

```
 print(f"이름: {name}")
```

```
print_names("철수", "영희", "민수")
```

이름: 철수

이름: 영희

이름: 민수

## \*\*kwargs (키워드 기반 가변 인자)

- \*\*kwargs는 키워드 인자(딕셔너리 형태) 로 여러 개의  
값을 입력 받을 수 있음

```
def print_info(**kwargs):
```

```
 for key, value in kwargs.items():
```

```
 print(f"{key}: {value}")
```

```
print_info(name="철수", age=25, hobby="축구")
```

name: 철수  
age: 25  
hobby: 축구

→ print\_info(name="철수", age=25, hobby="축구")처럼  
키=값 형태로 전달하면,

```
kwargs={'name': '철수', 'age': 25, 'hobby': '축구'}
```

딕셔너리 로 저장

- 반복문을 사용해서 \*\*kwargs 처리 가능

```
def introduce(**info):
```

```
 for key, value in info.items():
```

```
 print(f"{key}: {value}")
```

```
introduce(name="영희", age=22, city="서울")
```

name: 영희  
age: 22  
city: 서울

# 가변 인자 (\*args, \*\*kwargs)

args가 먼저, \*\*kwargs가 나중에 오는 순서는 기본으로 지켜야 함

## \*args와 \*\*kwargs 함께 사용하기

### • 순서 예시

```
def mix_example(a, b, *args, **kwargs):
 print(f"a: {a}, b: {b}")
 print(f"args: {args}") # 튜플
 print(f"kwargs: {kwargs}") # 딕셔너리

mix_example(1, 2, 3, 4, 5, name="철수", age=30)
```

→ 1, 2 → a, b에 저장  
 3, 4, 5 → args 튜플에 저장  
 name="철수", age=30 → kwargs 딕셔너리에 저장

```
a: 1, b: 2
args: (3, 4, 5)
kwargs: {'name': '철수', 'age': 30}
```

### • 잘못된 순서 (오류 발생!)

```
def wrong_example(a, b, **kwargs, *args): # ❌ 오류 발생
 pass
```

→ \*args가 먼저 와야 하고, \*\*kwargs는 마지막에

- 매개변수와 반환값: return을 사용해 값을 반환
- 기본값 설정: def func(a=10)처럼 기본값 설정 가능
- \*args: 여러 개의 위치 인자를 받을 때 (튜플로 저장)
- \*\*kwargs: 여러 개의 키워드 인자를 받을 때 (딕셔너리로 저장)
- 함께 사용할 때 순서: 일반 인자 → \*args → \*\*kwargs



# [정리] \*args, \*kwargs

Python에서 여러 개의 인자를 한꺼번에 받을 때 사용하는 문법

| 구분      | 설명 및 예시                                                                                                               |
|---------|-----------------------------------------------------------------------------------------------------------------------|
| *args   | <ul style="list-style-type: none"> <li>여러 개의 인자를 튜플(tuple)로 묶어서 함수에 전달</li> <li>개수 제한 없이 여러 개의 인자를 넘길 수 있음</li> </ul> |
| *kwargs | <ul style="list-style-type: none"> <li>여러 개의 키워드 인자를 딕셔너리로 전달</li> </ul>                                              |

```
#args, kwargs 예제
print("*"*10)
print("축을 기준으로 계산")
print("*"*10)

def print_numbers(*args):
 print(args)

print_numbers(1, 2, 3, 4, 5)

(1, 2, 3, 4, 5)
100
```

\*args는 전달된 인자들을 튜플 형태로  
저장  
→ args는 (1, 2, 3, 4, 5)라는 튜플

```
print("*"*10)
print("축을 기준으로 계산")
print("*"*10)

def add_numbers(*args):
 return sum(args)

print(add_numbers(10, 20, 30, 40))

축을 기준으로 계산

100
```

args는  
(10, 20, 30, 40)로  
저장  
→ sum(args)는 100

# [정리] \*args, \*kwargs

Python에서 여러 개의 인자를 한꺼번에 받을 때 사용하는 문법

```
def greet(message, *names):
 for name in names:
 print(f"{message}, {name}!")

greet("Hello", "Alice", "Bob", "Charlie")
```

```
Hello, Alice!
Hello, Bob!
Hello, Charlie!
```

첫 번째 인자인 "Hello"는 message에 저장  
나머지 "Alice", "Bob", "Charlie"는 names에 튜플로 저장

```
def multiply(a, b, c):
 return a * b * c

numbers = [2, 3, 4]
result = multiply(*numbers)
print(result)
```

```
24
```

\*numbers는 [2, 3, 4] 리스트의 요소를 각각 a, b, c에 전달

# [정리] \*args, \*kwargs

Python에서 여러 개의 인자를 한꺼번에 받을 때 사용하는 문법

```
def example_function(*args, **kwargs):
 print("args:", args)
 print("kwargs:", kwargs)

example_function(1, 2, 3, name="Alice", age=25)

args: (1, 2, 3)
kwargs: {'name': 'Alice', 'age': 25}
```

args는 (1, 2, 3) 튜플  
kwargs는 {'name': 'Alice', 'age': 25} 딕셔너리

## "\*args와 \*kwargs"

- ✔ args는 여러 개의 위치 인자를 받을 때 사용
- ✔ args는 튜플 형태로 저장됨
- ✔ 리스트 언패킹(리스트)에도 활용 가능
- ✔ kwargs와 함께 사용 가능 (kwargs는 딕셔너리)

# def와 일급 객체

## def 개념

- Python의 함수는 일급 객체(First-class Object)  
→ 변수에 할당, 인자로 전달, 리턴값으로 사용 가능
- def 키워드로 함수 정의

## 예시코드

```
def add(a, b):
 return a + b

변수에 할당
f = add
print(f(10, 5)) # 15

함수 인자로 전달
def operate(func, x, y):
 return func(x, y)

print(operate(add, 3, 4)) # 7
```

# 람다함수

## ✓ 1. 람다함수란?

- 한 줄로 간단하게 쓸 수 있는 "작은" 함수
- 보통 짧고 간단한 기능을 만들 때 사용

## ✓ 2. 일반 함수 vs. 람다 함수

|       | 일반함수                                                                 | 람다함수                                                          |
|-------|----------------------------------------------------------------------|---------------------------------------------------------------|
| 함수 정의 | <pre>def add(x, y):     return x + y  print(add(2, 3)) # 출력: 5</pre> | <pre>add = lambda x, y: x + y  print(add(2, 3)) # 출력: 5</pre> |

## ✓ 3. 언제 쓸까? (복잡한 로직을 쓸 때는 제외 – def )

| 상황                                        | 람다함수 사용 이유      |
|-------------------------------------------|-----------------|
| · 짧은 함수를 한 줄로 표현                          | · 코드의 간결성       |
| · <b>map(), filter(), sort()</b> 등과 함께 사용 | · 간단한 변환을 쉽게 적용 |

## 사용 예시

### ✓ 리스트 정렬할 때

```
students = [("철수", 90), ("영희", 80), ("민수", 95)]
students.sort(key=lambda x: x[1]) # 점수 기준 정렬
print(students)
출력: [('영희', 80), ('철수', 90), ('민수', 95)]
```

### ✓ map()과 함께 사용 (리스트의 모든 값 변환)

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, numbers))
print(squared)
출력: [1, 4, 9, 16, 25]
```

### ✓ filter()와 함께 사용 (특정 조건만 남기기)

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)
출력: [2, 4, 6]
```

# 고차함수

## ✓ 1. 고차함수란?

- 다른 함수를 인자로 받거나, 결과로 반환하는 함수
- Python의 `map()`, `filter()`, `reduce()` 등이 대표적

## 사용 예시

### ✓ 함수 인자로 받기

- `double` 함수(람다)를 `apply_function`에 전달하여 실행

```
def apply_function(func, value):
 return func(value)

double = lambda x: x * 2

print(apply_function(double, 10)) # 20
```

### ✓ 함수 반환

- `make_multiplier(3)`가 새로운 함수(`lambda x: x * 3`)를 반환

```
def make_multiplier(n):
 return lambda x: x * n # 배수를 반환하는 람다 함수

times3 = make_multiplier(3) # 3배수 함수 생성

print(times3(5)) # 15
```

# 고차함수

## ✓ 1. 고차함수란?

- 다른 함수를 인자로 받거나, 결과로 반환하는 함수
- Python의 `map()`, `filter()`, `reduce()` 등이 대표적

## 사용 예시

### ✓ `map()`: 리스트의 각 요소를 변환

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, numbers))

print(squared) # [1, 4, 9, 16, 25]
```

### ✓ `filter()`: 조건에 맞는 요소만 선택

```
numbers = [1, 2, 3, 4, 5, 6]
even = list(filter(lambda x: x % 2 == 0, numbers))

print(even) # [2, 4, 6]
```

### ✓ `reduce()`: 리스트의 값을 하나로 합침 (두 개씩 연산하며 최종 결과를 반환)

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
sum_all = reduce(lambda x, y: x + y, numbers)
print(sum_all) # 15
```

# 클로저(Closure)

## Closure

- 클로저는 함수가 자신이 선언된 환경을 기억하는 구조 (뒤에 자세히)
- 내부 함수가 외부 함수의 변수를 참조

## 예시코드

```
def multiplier(factor):
 def multiply(x):
 return x * factor
 return multiply

double = multiplier(2)
print(double(10)) # 20
```



# 제너레이터(Generator)와 yield

## Generator & yield

- 제너레이터는 이터레이터를 생성하는 함수
- **yield**는 **return**과 달리 중단 상태를 저장하고 다음 호출 시 이어서 실행

## 예시코드

```
def countdown(n):
 while n > 0:
 yield n
 n -= 1

for i in countdown(5):
 print(i)
```

5  
4  
3  
2  
1

# 제너레이터(Generator)와 yield

【 Generator vs List 】

| 비교 항목  | List Comprehension       | Generator                |
|--------|--------------------------|--------------------------|
| 메모리 사용 | 전체 메모리 사용                | 필요 시 생성 (lazy)           |
| 반환 형태  | 리스트 객체                   | 제너레이터 객체                 |
| 구문 예시  | [x*x for x in range(10)] | (x*x for x in range(10)) |

# 내장 함수 활용

내장 함수(Built-in Functions)란 별도의 설치 없이 바로 사용할 수 있는 함수

| 구분             | 함수                                | 예시                                                                                                                                                                   |
|----------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 기본<br>내장<br>함수 | • <b>print()</b> → 출력 함수          | • <code>print("안녕하세요!")</code> # 출력: 안녕하세요!                                                                                                                          |
|                | • <b>type()</b> → 데이터 타입 확인       | • <code>print(type(10))</code> # 출력: <class 'int'><br>• <code>print(type(3.14))</code> # 출력: <class 'float'><br>• <code>print(type("문자"))</code> # 출력: <class 'str'> |
|                | • <b>id()</b> → 변수의 메모리 주소 확인     | • <code>a = 10</code><br>• <code>print(id(a))</code> # (출력 값은 실행할 때마다 다를 수 있음)                                                                                       |
| 숫자 관련<br>내장 함수 | • <b>abs()</b> → 절댓값              | • <code>print(abs(-5))</code> # 출력: 5<br>• <code>print(abs(10))</code> # 출력: 10                                                                                      |
|                | • <b>pow()</b> → 거듭제곱 ( $a^b$ 계산) | • <code>print(pow(2, 3))</code> # $2^3 = 8$<br>• <code>print(pow(5, 2))</code> # $5^2 = 25$                                                                          |
|                | • <b>round()</b> → 반올림            | • <code>print(round(3.14159, 2))</code> # 출력: 3.14 (소수점 2번째 자리까지)<br>• <code>print(round(7.6))</code> # 출력: 8                                                        |

# 내장 함수 활용

내장 함수(Built-in Functions)란 별도의 설치 없이 바로 사용할 수 있는 함수

| 구분                   | 함수                               | 예시                                                                                                                                                                                                                                                          |
|----------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 리스트 & 튜플<br>관련 내장 함수 | • <b>len()</b> → 길이(개수) 확인       | <ul style="list-style-type: none"> <li>• <code>print(len([1, 2, 3, 4]))</code> # 출력: 4</li> <li>• <code>print(len("Hello"))</code> # 출력: 5</li> </ul>                                                                                                       |
|                      | • <b>max()</b> → 최대값             | <ul style="list-style-type: none"> <li>• <code>print(max(3, 5, 7, 2))</code> # 출력: 7</li> <li>• <code>print(max([10, 20, 30]))</code> # 출력: 30</li> </ul>                                                                                                   |
|                      | • <b>min()</b> → 최소값             | <ul style="list-style-type: none"> <li>• <code>print(min(3, 5, 7, 2))</code> # 출력: 2</li> <li>• <code>print(min([10, 20, 30]))</code> # 출력: 10</li> </ul>                                                                                                   |
|                      | • <b>sum()</b> → 합계 구하기          | <ul style="list-style-type: none"> <li>• <code>print(sum([1, 2, 3, 4, 5]))</code> # 출력: 15</li> <li>• <code>print(sum([10, 20, 30]))</code> # 출력: 60</li> </ul>                                                                                             |
|                      | • <b>sorted()</b> → 정렬 (오름차순 기본) | <ul style="list-style-type: none"> <li>• <code>numbers = [5, 2, 9, 1]</code></li> <li>• <code>print(sorted(numbers))</code> # 출력: [1, 2, 5, 9] (오름차순)</li> <li>• <code>print(sorted(numbers, reverse=True))</code><br/># 출력: [9, 5, 2, 1] (내림차순)</li> </ul> |

# 내장 함수 활용

내장 함수(Built-in Functions)란 별도의 설치 없이 바로 사용할 수 있는 함수

| 구분           | 함수                            | 예시                                                                                                              |
|--------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 문자열 관련 내장 함수 | • <b>ord()</b> → 문자 → 유니코드 변환 | • <code>print(ord('A'))</code> # 출력: 65<br>• <code>print(ord('가'))</code> # 출력: 44032                           |
|              | • <b>chr()</b> → 유니코드 → 문자 변환 | • <code>print(chr(65))</code> # 출력: A<br>• <code>print(chr(44032))</code> # 출력: 가                               |
| 형변환 함수       | • <b>int()</b> → 정수로 변환       | • <code>print(int("10"))</code> # 문자열 "10" → 정수 10<br>• <code>print(int(3.7))</code> # 실수 3.7 → 정수 3            |
|              | • <b>float()</b> → 실수로 변환     | • <code>print(float("3.14"))</code> # 문자열 "3.14" → 실수 3.14<br>• <code>print(float(10))</code> # 정수 10 → 실수 10.0 |
|              | • <b>str()</b> → 문자열로 변환      | • <code>print(str(123))</code> # 정수 123 → 문자열 "123"<br>• <code>print(str(3.14))</code> # 실수 3.14 → 문자열 "3.14"   |

# 내장 함수 활용

내장 함수(Built-in Functions)란 별도의 설치 없이 바로 사용할 수 있는 함수

| 구분              | 함수                                             | 예시                                                                                                                                                              |
|-----------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 기타 유용한<br>내장 함수 | • <b>enumerate()</b> → 리스트의 인덱스와 값을<br>같이 가져오기 | <pre>fruits = ["사과", "바나나", "체리"] for index, value in enumerate(fruits):     print(index, value)</pre> <div>0 사과<br/>1 바나나<br/>2 체리</div>                       |
|                 | • <b>zip()</b> → 여러 리스트 묶기                     | <pre>names = ["철수", "영희", "민수"] scores = [90, 85, 88]  for name, score in zip(names, scores):     print(name, score)</pre> <div>철수 90<br/>영희 85<br/>민수 88</div> |
|                 | • <b>map()</b> → 리스트의 모든 요소에 함수 적용             | <pre>numbers = [1, 2, 3, 4, 5] squared = list(map(lambda x: x ** 2, numbers)) print(squared) # 출력: [1, 4, 9, 16, 25]</pre>                                      |
|                 | • <b>filter()</b> → 조건을 만족하는 요소만 필터링           | <pre>numbers = [1, 2, 3, 4, 5, 6] even_numbers = list(filter(lambda x: x % 2 == 0, numbers)) print(even_numbers) # 출력: [2, 4, 6]</pre>                          |

# 모듈 사용, 주요 표준 라이브러리

Python은 강력한 모듈(**Module**)과 라이브러리(**Library**)를 제공

## ✓ 1. Python에는?

- 모듈(**Module**): 함수나 클래스, 변수 등을 포함한 파일 (.py)
- 라이브러리(**Library**): 여러 모듈을 모아둔 것
- 패키지(**Package**): 여러 모듈을 하나로 묶은 폴더

## ✓ 2. 기본적으로 제공되는 표준 라이브러리(**Standard Library**)가 있으며 , import를 통해 쉽게 사용

# 모듈 사용

## 모듈 가져오기 (import)

### · 기본적인 import 사용법

```
import math # math 모듈 가져오기

print(math.sqrt(16)) # 출력: 4.0 (제곱근)
print(math.pi) # 출력: 3.141592653589793
```

→ **math.sqrt(16), math.pi** 처럼 모듈명을 붙여서 사용

### · from 모듈 import 함수 사용법

- 특정 함수만 가져와서 사용하면 모듈명을 생략

```
from math import sqrt, pi

print(sqrt(25)) # 출력: 5.0 (제곱근)
print(pi) # 출력: 3.141592653589793
```

→ **from math import sqrt, pi** 덕분에 **math.sqrt()** 대신 **sqrt()** 만 사용 가능

### · import 모듈 as 별칭 사용법

```
import numpy as np # numpy를 np로 사용
import pandas as pd # pandas를 pd로 사용
```

→ **as** 키워드를 사용하면 모듈명을 간단하게 사용

### · from 모듈 import \* (비추천)

```
from math import * # math 모듈의 모든 함수 가져오기

print(sin(1)) # math.sin()을 사용하지 않아도 됨
```

→ 이름 충돌 위험 ↑  
어떤 함수가 어디서 왔는지 모호해질 수 있어 권장 X



# 주요 표준 라이브러리 소개

| 라이브러리명        | 기능                                | 예시                                                                                                                                                                                                                                                |
|---------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>math</b>   | · 수학 계산을 위한 라이브러리                 | <pre>import math  print(math.sqrt(25)) # 출력: 5.0 (제곱근) print(math.factorial(5)) # 출력: 120 (5!) print(math.gcd(12, 18)) # 출력: 6 (최대공약수) print(math.pi) # 출력: 3.141592653589793</pre>                                                               |
| <b>random</b> | · 랜덤 숫자 또는 리스트에서 무작위 요소를 선택할 때 사용 | <pre>import random  print(random.randint(1, 10)) # 1~10 사이의 랜덤 정수 print(random.random()) # 0~1 사이의 랜덤 실수 print(random.choice(["사과", "바나나", "체리"])) # 리스트에서 랜덤 선택  numbers = [1, 2, 3, 4, 5] random.shuffle(numbers) # 리스트 섞기 print(numbers)</pre> |

# 주요 표준 라이브러리 소개

| 라이브러리명          | 기능                | 예시                                                                                                                                                                                                           |
|-----------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>datetime</b> | · 날짜와 시간을 다룰 때 사용 | <pre>import datetime  now = datetime.datetime.now() print(now) # 현재 날짜와 시간 출력  today = datetime.date.today() print(today) # 현재 날짜 출력  # 날짜 생성 d = datetime.date(2025, 2, 13) print(d) # 출력: 2025-02-13</pre> |
| <b>os</b>       | · 파일 및 디렉토리 관리    | <pre>import os  print(os.getcwd()) # 현재 작업 디렉토리 확인 os.mkdir("새폴더") # 새 폴더 생성 os.rmdir("새폴더") # 폴더 삭제</pre>                                                                                                   |

# 주요 표준 라이브러리 소개

| 라이브러리명      | 기능                                                                                 | 예시                                                                                                                      |
|-------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>sys</b>  | <ul style="list-style-type: none"> <li>• <b>Python</b> 인터프리터와 관련된 기능 제공</li> </ul> | <pre>import sys  print(sys.version) # Python 버전 확인 print(sys.argv)   # 실행 시 입력된 인자 확인 sys.exit()        # 프로그램 종료</pre> |
| <b>time</b> | <ul style="list-style-type: none"> <li>• 시간 지연 및 현재 시간 확인</li> </ul>               | <pre>import time  print(time.time()) # 현재 시간(초) time.sleep(2)      # 2초 동안 프로그램 멈춤 print("2초 후 실행됨")</pre>              |

# 주요 표준 라이브러리 소개

| 라이브러리명      | 기능                                                                             | 예시                                                                                                                                                                                                        |
|-------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>json</b> | <ul style="list-style-type: none"> <li>• <b>JSON</b> 데이터를 읽고 쓸 때 사용</li> </ul> | <pre>import json  data = {"name": "철수", "age": 25} json_string = json.dumps(data) # JSON 문자열 변환 print(json_string)  parsed_data = json.loads(json_string) # JSON 문자열 → # 딕셔너리 변환 print(parsed_data)</pre> |
| <b>re</b>   | <ul style="list-style-type: none"> <li>• 문자열에서 특정 패턴 찾기</li> </ul>             | <pre>import re  text = "My phone number is 010-1234-5678." pattern = r"\d{3}-\d{4}-\d{4}" # 전화번호 패턴 match = re.search(pattern, text) if match:     print(match.group()) # 출력: 010-1234-5678</pre>         |

# [참고] 인터프리터 비교 및 종류

## 인터프리터와 컴파일러 비교

| 구분    | 인터프리터(Python)        | 컴파일러(C, JAVA)    |
|-------|----------------------|------------------|
| 실행 방식 | · 한 줄 씩 실행           | · 전체 코드를 번역 후 실행 |
| 속도    | · 느림 (실시간 해석)        | · 빠름 (미리 번역)     |
| 오류 발견 | · 코드 실행 중 오류 발생      | · 컴파일 단계에서 오류 발견 |
| 예시언어  | · Python, javascript | · C, C++, JAVA   |

## Python 인터프리터 종류

| 인터프리터             | 설명                         |
|-------------------|----------------------------|
| <b>CPython</b>    | · 기본 Python 인터프리터(공식 버전)   |
| <b>PyPy</b>       | · 빠른 실행속도를 위한 JIT 컴파일러 포함  |
| <b>Jython</b>     | · Java와 함께 사용할 수 있는 Python |
| <b>IronPython</b> | · .NET 환경에서 실행 가능한 Python  |

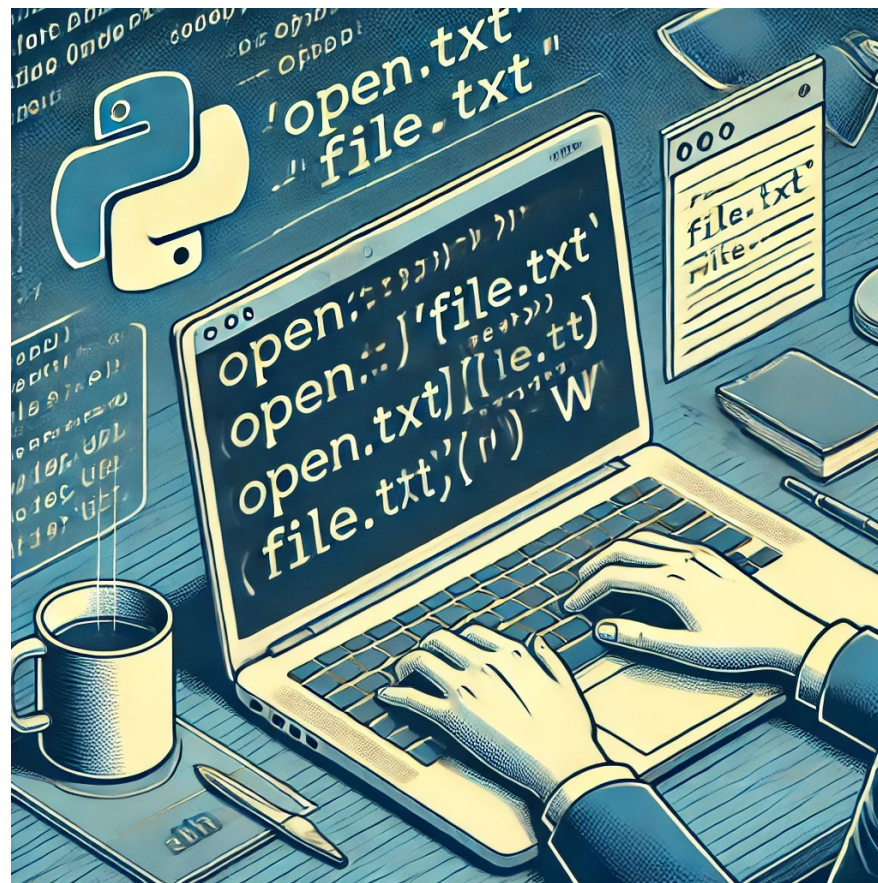
# 파일 읽기/쓰기

## ✓ 1. File I/O

- 파일 열기: `open()` 함수 사용
- 파일 읽기/쓰기: 파일 모드 (`r`, `w`, `a`) 설정
- CSV 파일 다루기: `csv` 모듈 활용
- JSON 파일 다루기: `json` 모듈 활용

## ✓ 2. 파일 열기, 읽기, 쓰기 (`open()`, 모드 설정)

| 모드        | 실행                        |
|-----------|---------------------------|
| <b>r</b>  | · 읽기모드(파일 있어야 함)          |
| <b>w</b>  | · 쓰기모드(없으면 생성, 기존 내용 삭제)  |
| <b>a</b>  | · 추가모드 (없으면 생성, 기존 내용 유지) |
| <b>r+</b> | · 읽기 + 쓰기 (기존 내용 유지)      |
| <b>w+</b> | · 읽기 + 쓰기 (기존 내용 삭제)      |



# 파일 읽기/쓰기

## 파일 읽기/쓰기

- 파일 쓰기 ('w') : 기존 파일 내용을 삭제하고 새로 작성

```
파일 열기 (쓰기 모드)
file = open("example.txt", "w")

파일에 쓰기
file.write("Hello, Python!\n")
file.write("파일 입출력을 배우고 있어요!\n")

파일 닫기 (반드시 닫아야 함)
file.close()
```

- 파일 읽기 ('r') : 파일이 있어야 하고 없으면 오류

```
파일 열기 (읽기 모드)
file = open("example.txt", "r")

파일 내용 읽기
content = file.read()
print(content)

파일 닫기
file.close()
```

- 파일 한 줄씩 읽기 (readline())

```
file = open("example.txt", "r")

print(file.readline()) # 첫 번째 줄 읽기
print(file.readline()) # 두 번째 줄 읽기

file.close()
```

- 여러 줄 읽기 (readlines()) : 각 줄을 리스트 형태로 받

```
file = open("example.txt", "r")

lines = file.readlines() # 모든 줄을 리스트로 반환
print(lines)

file.close()
```

- 내용 추가 ('a') / with 문 (파일 자동으로 닫음)

```
file = open("example.txt", "a")
file.write("새로운 내용을 추가합니다!\n")
file.close()
```

```
with open("example.txt", "r") as file:
 content = file.read()
 print(content) # 파일 자동 닫힘
```

# CSV 파일 다루기 (csv 모듈)

CSV(Comma-Separated Values)는 쉼표(,)로 구분된 데이터 파일. 엑셀과 비슷, 텍스트 파일 형태로 저장

## CSV 파일

- CSV 파일 읽기 (csv.reader())

```
import csv

CSV 파일 열기 (읽기 모드)
with open("data.csv", "r", encoding="utf-8") as file:
 reader = csv.reader(file) # CSV 읽기 객체 생성
 for row in reader: # 한 줄씩 읽기
 print(row)
```

### data.csv.

```
이름,나이,도시
철수,25,서울
영희,30,부산
민수,22,대구
```

### 출력

```
['이름', '나이', '도시']
['철수', '25', '서울']
['영희', '30', '부산']
['민수', '22', '대구']
```

- CSV 파일 쓰기 (csv.writer())

```
import csv

CSV 파일 열기 (쓰기 모드)
with open("new_data.csv", "w", encoding="utf-8",
 newline="") as file:
 writer = csv.writer(file)

 # 헤더 추가
 writer.writerow(["이름", "나이", "도시"])

 # 데이터 추가
 writer.writerow(["철수", 25, "서울"])
 writer.writerow(["영희", 30, "부산"])
 writer.writerow(["민수", 22, "대구"])
```

→ newline=""을 설정하면 빈 줄이 추가



# JSON 파일 다루기 (json 모듈)

JSON(JavaScript Object Notation) 데이터를 저장하고 전달하는 경량 형식으로 Python의 딕셔너리와 유사한 구조

## CSV 파일

- JSON 파일 저장 (json.dump())

```
import json

data = {"이름": "철수", "나이": 25, "도시": "서울"}

JSON 파일로 저장
with open("data.json", "w", encoding="utf-8") as file:
 json.dump(data, file, ensure_ascii=False, indent=4)
```

→ ensure\_ascii=False → 한글이 깨지지 않도록 설정  
indent=4 → 보기 좋게 들여쓰기 설정

### data.json

```
{
 "이름": "철수",
 "나이": 25,
 "도시": "서울"
}
```

- JSON 파일 읽기 (json.load())

```
import json

JSON 파일 열기
with open("data.json", "r", encoding="utf-8") as file:
 data = json.load(file)
 print(data) # 출력: {'이름': '철수', '나이': 25, '도시': '서울'}
```

→ JSON 파일을 딕셔너리로 변환해서 사용 가능

# 파일 읽기·쓰기 — CSV · JSON · Parquet · pathlib

## 【 코드 예시 】

### 핵심 내용

- ✓ **pathlib.Path**  
: macOS 경로 처리 표준
- ✓ **with**  
: 파일 자동 닫기 (컨텍스트 매니저)
- ✓ **csv.DictReader**  
: 헤더를 키로 행 읽기
- ✓ **json.load / json.dump**  
: API 응답 처리
- ✓ **Parquet: 컬럼형 포맷, CSV보다 10× 빠른 읽기**

```
파일 I/O 실전
from pathlib import Path
import json, csv

data_dir = Path('data')

JSON 읽기 (API 응답)
with open(data_dir/'resp.json') as f:
 data = json.load(f)

CSV 읽기
with open(data_dir/'sales.csv') as f:
 rows = list(csv.DictReader(f))

Parquet (pandas 연동)
import pandas as pd
df = pd.read_parquet(data_dir/'sales.parquet')
df.to_parquet(data_dir/'out.parquet') # 저장
```

# 데코레이터와 클로저 - 데코레이터

데코레이터와 클로저는 Python에서 함수형 프로그래밍을 할 때 매우 중요한 개념이며 둘 다 함수와 관련이 있지만, 다른 특성을 가짐

## 데코레이터 주요 특징

- 기능 확장
  - 기존 함수의 코드를 직접 수정하지 않고 새로운 기능을 추가
- 구문
  - 데코레이터는 '@' 기호를 사용하여 함수 정의 위에 적용
- 작동 원리
  - 데코레이터는 함수를 인수로 받아 새로운 함수를 반환하는 callable 객체
- 고차 함수
  - 데코레이터는 함수를 인자로 받거나 함수를 반환하는 고차 함수의 특성
- 유연성
  - \*args와 \*\*kwargs를 사용하여 다양한 인자와 반환값을 처리

## "Python 코드의 재사용성과 가독성 향상에 도움"

```
def my_decorator(func):
 def wrapper():
 print("Before the function is called")
 func()
 print("After the function is called")
 return wrapper

@my_decorator
def greet():
 print("Hello!")

greet()
```

### ✓ my\_decorator 역할

- greet 함수를 감싸서 추가 기능(함수 호출 전후 메시지 출력)을 제공

# 기본적인 Decorator 예제

## 간단한 Decorator 예제

- 변수 없는 Decorator

```
데코레이터 함수 정의
def my_decorator(func):
 def wrapper():
 print("함수 실행 전")
 func()
 print("함수 실행 후")
 return wrapper

데코레이터 적용
@my_decorator
def hello():
 print("Hello, Python!")

hello()
```

```
함수 실행 전
Hello, Python!
함수 실행 후
```

## 인자를 받는 함수에 대한 Decorator

- 인자를 받아서 처리할 때의 예시

```
def my_decorator(func):
 def wrapper(*args, **kwargs):
 print("함수 실행 전")
 result = func(*args, **kwargs)
 print("함수 실행 후")
 return result
 return wrapper

@my_decorator
def add(a, b):
 return a + b

print(add(3, 5))
```

```
함수 실행 전
함수 실행 후
8
```

# 기본적인 Decorator 예제

## 인자를 받는 함수에 대한 Decorator

- 인자를 받아서 처리할 때의 예시

```
def my_decorator(func):
 def wrapper(*args, **kwargs):
 print("함수 실행 전")
 result = func(*args, **kwargs)
 print("함수 실행 후")
 return result
 return wrapper
```

```
@my_decorator
def add(a, b):
 return a + b
```

```
print(add(3, 5))
```

```
함수 실행 전
함수 실행 후
8
```

Decorator가 실행과정을 감싸고 있음

- 함수 실행전 / 함수실행 후가 먼저 출력
- 그후에 add(3,5)가 실행

add → 안에 정의한 wrapper 함수로 대체

→ print(add(3,5))를 실행하면 wrapper(3,5) 실행

→ 실행전 출력

→ 원래 함수 실행 fun(3,5) : add(3,5) 실행

→ add(3,5) 값 반환

→ 실행 후 출력

→ wrapper 는 result 값 반환 (8)

# 실용적인 Decorator 예제

## 실행 시간 측정 Decorator

- 여러 개의 인자를 한꺼번에 받는 Decorator

```
import time

def timer(func):
 def wrapper(*args, **kwargs):
 start = time.time()
 result = func(*args, **kwargs)
 end = time.time()
 print(f"실행 시간: {end - start:.4f}초")
 return result
 return wrapper

@timer
def slow_function():
 time.sleep(2)
 print("작업 완료!")

slow_function()
```

작업 완료!  
실행 시간: 2.0025초

## 로그인 검증 Decorator

- Login 방법

```
def require_login(func):
 def wrapper(user):
 if user.get("is_logged_in"):
 return func(user)
 else:
 print("로그인이 필요합니다.")
 return wrapper

@require_login
def dashboard(user):
 print(f"{user['name']}님의 대시보드입니다.")

user1 = {"name": "Alice", "is_logged_in": True}
user2 = {"name": "Bob", "is_logged_in": False}

dashboard(user1) # 정상 실행
```

Alice님의 대시보드입니다.  
로그인이 필요합니다.

# 실용적인 Decorator 예제

## 실행 시간 측정 Decorator

- 여러 개의 인자를 한꺼번에 받는 Decorator

```
import time

def timer(func):
 def wrapper(*args, **kwargs):
 start = time.time()
 result = func(*args, **kwargs)
 end = time.time()
 print(f"실행 시간: {end - start:.4f}초")
 return result
 return wrapper

@timer
def slow_function():
 time.sleep(2)
 print("작업 완료!")

slow_function()
```

작업 완료!  
실행 시간: 2.0025초

slow\_function 호출 (실제는 wrapper가 실행)

→ start 에 현재 시간 저장

→ slow\_function 실행

2초 대기 후 작업완료 출력

현재 시간을 end 변수에 저장

→ 실행시간 계산하고 출력

→ wrapper 반환 값 = slow\_function 반환값과 동일하게 반환

즉, Decorator는 호출하게 되면 먼저 wrapper가 실행되면서 그 안에 있는 코드들이 실행된 후 그 값을 전달

# 데코레이터와 클로저 - 클로저

클로저는 내부 함수가 외부 함수의 변수를 참조할 수 있는 특성이며 즉, 내부 함수가 외부 함수의 지역 변수를 기억하고 사용

## 클로저 동작 특징

- 외부 함수 안에 내부 함수가 정의
- 내부 함수는 외부 함수의 변수에 접근 가능
- 외부 함수 종료 후에도  
내부 함수는 외부 함수의 변수를 기억하고 사용

## 클로저와 데코레이터의 관계

- 데코레이터는 내부적으로 클로저를 사용하여  
기존 함수를 감싸고, 그 함수의 동작을 확장  
→ 즉, 데코레이터는 클로저를 활용한 함수

## 클로저 예시

```
def outer():
 x = 10 # 외부 함수의 변수

 def inner():
 print(x) # 내부 함수에서 외부 함수의 변수에 접근
 return inner # 내부 함수 반환

 # 클로저가 만들어짐
 closure = outer()

 # 클로저 실행
 closure() # x = 10을 출력
```

10

### ✓ 클로저 사용 이유

- 클로저를 사용하면 외부 함수의 변수를 계속해서 사용할 수 있는 상태를 유지
- 이를 통해 상태 저장 혹은 함수를 커스터마이징 할 수 있음



# [참고] 데코레이터와 클로저 함께 사용하기

## 사용 예제

- outer\_decorator는 데코레이터이고, 클로저인 wrapper를 내부에 정의하여, 함수의 실행 전후로 처리할 로직

### 데코레이터와 클로저 함께 사용

```
def outer_decorator(func):
 def wrapper(*args, **kwargs):
 print("함수 실행 전 처리")
 result = func(*args, **kwargs)
 print("함수 실행 후 처리")
 return result
 return wrapper
@outer_decorator

def add(a, b):
 return a + b

print(add(3, 4))
```

```
함수 실행 전 처리
함수 실행 후 처리
7
```

#### ※ 데코레이터

함수를 인자로 받아 새로운 함수로 반환하고,  
기존 함수를 수정하거나 확장하는 방법

#### ※ 클로저

내부 함수가 외부 함수의 지역 변수를 참조할 수 있는 특성

# 데코레이터 체이닝

데코레이터 체이닝이란 더 강력하고 깔끔한 코드 작성 목적으로 여러 개의 데코레이터를 연속으로 적용하는 것

## 데코레이터 2개 적용

```
def decorator1(func):
 def wrapper():
 print("[decorator1] 실행 전")
 func()
 print("[decorator1] 실행 후")
 return wrapper
```

```
def decorator2(func):
 def wrapper():
 print("[decorator2] 실행 전")
 func()
 print("[decorator2] 실행 후")
 return wrapper
```

```
@decorator1
@decorator2
def say_hello():
 print("안녕하세요!")
```

```
say_hello()
```

```
[decorator1] 실행 전
[decorator2] 실행 전
안녕하세요!
[decorator2] 실행 후
[decorator1] 실행 후
```

※ say\_hello() → decorator2 적용 → decorator1 적용

- ✓ say\_hello는 decorator1에 의해 감싸져 있음
- ✓ decorator1.wrapper() 실행 → print("[decorator1] 실행 전")
- ✓ decorator2.wrapper() 실행 → print("[decorator2] 실행 전")
- ✓ say\_hello() 실행 → "안녕하세요!" 출력
- ✓ decorator2.wrapper() 실행 종료 → print("[decorator2] 실행 후")
- ✓ decorator1.wrapper() 실행 종료 → print("[decorator1] 실행 후")

택배박스(decorator1) 열고 물건포장(decorator2) 열어서  
물건 확인(say\_hello) 하고 물건 포장 (decorator2 종료) 닫고  
택배박스(decorator1 종료) 닫기

# functools.wraps의 필요성

데코레이터 사용 시, 원래 함수의 이름(name)과 설명(docstring) 이 사라지는 문제가 발생해서 이름, 설명, 속성을 유지하기 위한 목적

## functools.wraps 미 적용

```
def decorator(func):
 def wrapper():
 """이것은 래퍼 함수입니다."""
 return func()
 return wrapper

@decorator
def say_hello():
 """이 함수는 인사를 합니다."""
 print("안녕하세요!")

print(say_hello.__name__) # wrapper (원래 함수명 사라짐)
print(say_hello.__doc__) # "이것은 래퍼 함수입니다." (원래 설명 사라짐)
```

※ say\_hello() 호출하기 전에 wrapper로 되어 있음 →  
say\_hello.\_\_name\_\_ (함수/클래스 이름) 출력 시 wrapper가 나옴  
→ say\_hello.\_\_doc\_\_ (함수, 클래스 설명)을 출력

- ✓ say\_hello가 decorator(say\_hello)에 의해 wrapper로 감싸짐
- ✓ say\_hello.\_\_name\_\_을 출력하면 "wrapper"이 나옴
- ✓ say\_hello.\_\_doc\_\_을 출력하면 "이것은 래퍼 함수입니다."가 나옴

# functools.wraps의 필요성

데코레이터 사용 시, 원래 함수의 이름(name)과 설명(docstring) 이 사라지는 문제가 발생해서 이름, 설명, 속성을 유지하기 위한 목적

## functools.wraps 적용

```
import functools

def decorator(func):
 @functools.wraps(func) # 원래 함수 정보를 유지
 def wrapper():
 """이것은 래퍼 함수입니다."""
 return func()
 return wrapper

@decorator
def say_hello():
 """이 함수는 인사를 합니다."""
 print("안녕하세요!")

print(say_hello.__name__) # say_hello (원래 이름 유지)
print(say_hello.__doc__) # "이 함수는 인사를 합니다."
(원래 설명 유지)
```

→functools.wraps(func)를 사용하면 say\_hello.\_\_name\_\_과 say\_hello.\_\_doc\_\_이 원래대로 유지

# functools.wraps + 데코레이터 체이닝 적용

**functools.wraps** 를 사용하면 데코레이터를 여러 개 체이닝할 때도 원래 함수 정보를 유지

## functools.wraps 적용

```
import functools

def decorator1(func):
 @functools.wraps(func)
 def wrapper():
 print("[decorator1] 실행 전")
 func()
 print("[decorator1] 실행 후")
 return wrapper

def decorator2(func):
 @functools.wraps(func)
 def wrapper():
 print("[decorator2] 실행 전")
 func()
 print("[decorator2] 실행 후")
 return wrapper
```

```
@decorator1
@decorator2

def say_hello():
 """이 함수는 인사를 합니다."""
 print("안녕하세요!")

print(say_hello.__name__) # say_hello (이름 유지)
print(say_hello.__doc__) # "이 함수는 인사를 합니다." (설명 유지)
say_hello()
```

```
say_hello
이 함수는 인사를 합니다.
[decorator1] 실행 전
[decorator2] 실행 전
안녕하세요!
[decorator2] 실행 후
[decorator1] 실행 후
```

# 왜 데코레이터와 클로저를?

## Why?

- 실제 FastAPI, LLM, RAG 구조에서도 매우 실용적이고 핵심적인 역할

## FastAPI 라우팅 구조 자체가 데코레이터 기반

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
 return {"item_id": item_id}
```

- **@app.get(...)**는 내부적으로 **APIRoute()**라는 데코레이터로 구현
- 따라서 사용자 정의 데코레이터도 로깅, 인증, 캐싱, 권한 제어 등에 쉽게 활용 가능

# 왜 데코레이터와 클로저를?

## Why?

- 실제 FastAPI, LLM, RAG 구조에서도 매우 실용적이고 핵심적인 역할

## LLM 응답 생성 흐름을 데코레이터로 래핑

```
def clean_input(func):
 def wrapper(prompt: str):
 prompt = prompt.strip().lower()
 return func(prompt)
 return wrapper

@clean_input
def generate_llm_response(prompt: str):
 return call_openai(prompt)
```

- 입력 클렌징, 토큰화, 벡터화 전처리를 데코레이터로 모듈화
- 응답 후처리도 동일하게 **@postprocess\_output** 등으로 감쌀 수 있음

# 왜 데코레이터와 클로저를?

## Why?

- 실제 FastAPI, LLM, RAG 구조에서도 매우 실용적이고 핵심적인 역할

## RAG 구조에서 클로저 활용

```
def retriever(collection):
 def search(query):
 return collection.similarity_search(query)
 return search

semantic_search = retriever(vector_store)
semantic_search("건축 설계 기준")
vector_store는 유지됨
```

- **retriever()**는 클로저 구조  
→ 상태 저장된 **vector store**에 계속 접근  
(FAISS, Chroma, pgvector)
- 클로저로 구성하면 **API**나 **Pipeline**에 주입하기 쉬움

RAG : Retrieval-Augmented Generation



# 왜 데코레이터와 클로저를?

## Why?

- 실제 FastAPI, LLM, RAG 구조에서도 매우 실용적이고 핵심적인 역할

## LLM 의사결정 Agent 로직에서 데코레이터 활용

```
def log_agent_action(func):
 def wrapper(*args, **kwargs):
 print(f"Agent 실행: {func.__name__}")
 return func(*args, **kwargs)
 return wrapper

@log_agent_action
def search_documents(query):
 # RAG + DB 검색
 ...
```

- 각 Step (검색, 추론, 요약) 마다 데코레이터로 추적/로깅 가능
- langchain이나 llamaindex에서도 이와 유사한 구조 존재

# 데코레이터 — 분석 파이프라인 공통 기능 분리

## 【 코드 예시 】

### 핵심 내용

- ✓ 데코레이터  
: 함수를 인자로 받아 기능 추가
- ✓ @timer : 실행 시간 측정
- ✓ @retry  
: 실패 시 재시도 (API 수집에서 필수)
- ✓ @lru\_cache : 동일 입력 결과 캐싱
- ✓ functools.wraps  
: 원본 함수 메타데이터 보존

```
데코레이터 실전
import time
from functools import wraps, lru_cache

def timer(func):
 @wraps(func)
 def wrapper(*args, **kwargs):
 t = time.perf_counter()
 result = func(*args, **kwargs)
 print(f'{func.__name__}: {time.perf_counter()-t:.3f}s')
 return result
 return wrapper

@timer
def load_data(path):
 return pd.read_parquet(path)

캐싱: 동일 인자면 재계산 생략
@lru_cache(maxsize=128)
def expensive_stats(key): ...
```

# 오류와 예외의 차이

## 오류

- Python이 코드를 실행할 수 없는 상태
- 보통 문법 오류(SyntaxError) 때문에 발생
- 개발자가 수정해야만 해결됨

### 오류

```
print("Hello, Python!
```

## 예외

- 코드는 실행되지만, 실행 중 문제가 발생하는 경우
- 예외가 발생해도 try-except로 처리하면 프로그램이 계속 실행
- 개발자가 적절히 처리하여 프로그램이 멈추지 않도록 할 수 있음

**try-except를 사용하면 해결 가능**

```
print(10 / 0) # 0으로 나눌 수 없음
```

# 예외 처리 구문

**try-except** 구문을 사용하여 예외가 발생해도 프로그램이 계속 실행되게 할 수 있음

## 기본 예외 처리 (try-except)

- try 블록에서 예외가 발생하면 → except 블록이 실행

```
try:
 print(10 / 0) # 0으로 나누기 (예외 발생!)
except ZeroDivisionError:
 print("0으로 나눌 수 없습니다!")
```

## 여러 개의 예외 처리

- 입력값에 따른 다양한 예외 처리

```
try:
 num = int(input("숫자를 입력하세요: ")) # 숫자가
 # 아닌 값 입력하면 예외 발생
 result = 10 / num # 0 입력하면 예외 발생
 print("결과:", result)
except ValueError:
 print("숫자만 입력해주세요!")
except ZeroDivisionError:
 print("0으로 나눌 수 없습니다!")
```

# 예외 처리 구문

**try-except** 구문을 사용하여 예외가 발생해도 프로그램이 계속 실행되게 할 수 있음

## 예외 메시지 출력 (as e)

- as e를 사용하면 예외 메시지 출력 가능

```
try:
 num = int(input("숫자를 입력하세요: "))
 result = 10 / num
except Exception as e: # 모든 예외 처리 가능
 print("예외 발생:", e)
```

예외 발생: division by zero

## finally: 예외 여부와 상관없이 실행

- 입력값에 따른 다양한 예외 처리

```
try:
 file = open("test.txt", "r") # 파일이 없으면 예외 발생
except FileNotFoundError:
 print("파일이 없습니다.")
finally:
 print("프로그램 종료") # 항상 실행됨
```

파일이 없습니다.  
프로그램 종료

# 사용자 정의 예외 작성

개발자가 직접 예외를 만들 수 있음

## 기본 사용자 정의 예외

- raise 키워드를 사용하여 강제로 예외 발생시킬 수 있음

```
class MyError(Exception): # Exception 클래스를 상속받
아 예외 생성
 pass
try:
 raise MyError # 예외 발생
except MyError:
 print("사용자 정의 예외 발생!")
```

## 예외 메시지 추가

- 사용자 예외를 만들 때 \_\_init\_\_()을 활용하여 메시지 추가 가능

```
class CustomError(Exception):
 def __init__(self, message):
 self.message = message

 def __str__(self):
 return f"CustomError: {self.message}"

try:
 raise CustomError("잘못된 입력입니다.") # 사용자
예외 발생
except CustomError as e:
 print(e)
```

CustomError: 잘못된 입력입니다.

# 사용자 정의 예외 작성

개발자가 직접 예외를 만들 수 있음

## 특정 조건에서 예외 발생 (raise)

- raise 키워드를 사용하여 강제로 예외 발생시킬 수 있음

```
def check_age(age):
 if age < 0:
 raise ValueError("나이는 0 이상이어야 합니다.")
 print("나이:", age)
try:
 check_age(-5) # 음수 입력 → 예외 발생
except ValueError as e:
 print("오류:", e)
```

오류: 나이는 0 이상이어야 합니다.

| 구분        | 설명                                                   |
|-----------|------------------------------------------------------|
| 오류 vs 예외  | • 문법 오류(SyntaxError)는 수정해야 하지만, 예외(Exception)는 처리 가능 |
| 예외 처리     | • try-except-finally를 사용하여 프로그램이 멈추지 않도록!            |
| 사용자 정의 예외 | • 개발자가 직접 예외를 만들고 raise로 발생 가능                       |

# 예외 처리 — 파이프라인 안정성 패턴

## 핵심 내용

- ✓ **try-except-else-finally** 완전 구조
- ✓ 특정 예외 계층으로 정밀하게 잡기
- ✓ 사용자 정의 예외: 도메인 오류 표현
- ✓ **finally**: 파일/연결 반드시 닫기
- ✓ **raise**: 상위로 예외 전파 패턴

## 【 코드 예시 】

```
예외 처리 실전
사용자 정의 예외
class DataValidationError(ValueError):
 def __init__(self, col, val):
 super().__init__(f'{col}={val} 검증 실패')

def safe_load(path):
 try:
 df = pd.read_parquet(path)
 if df.empty: raise DataValidationError('rows', '0')
 return df
 except FileNotFoundError:
 logger.error(f'파일 없음: {path}')
 return None
 except DataValidationError as e:
 logger.warning(str(e))
 return None
 finally:
 logger.info(f'로딩 시도: {path}')
```



# 왜 로깅이 필요한가?

## 【 print()의 한계 】

- 디버깅용으로는 좋지만 실 서비스에서는 부적절
- 출력 레벨 구분이 불가능하고, 로그 관리가 안 됨

## 【 로깅의 필요성 】

- 운영 중 오류 추적 가능  
(트래픽 많을 때 발생한 문제 재현)
- 로그 레벨을 통해  
개발/운영/디버깅 로그를 구분
- 파일로 저장, 회전(logging rotation),  
알람 연계 가능

# logging 기본 사용법

```
import logging
```

```
logging.basicConfig(level=logging.INFO)
```

```
logging.info("서비스 시작")
```

```
logging.warning("주의: 설정 값이 누락되었습니다.")
```

```
logging.error("에러 발생: 파일을 찾을 수 없습니다.")
```

- **logging.basicConfig()**는 단순한 설정만 가능
- 실무에서는 이 방식은 한 번만 호출되어야 함

## 실무 요구 사항 예시

- 콘솔에는 **INFO** 이상만 출력
- 파일에는 **DEBUG**까지 모두 저장
- 매일 로그를 분리 저장 (파일 이름에 날짜 포함)
- 에러는 별도 파일에 저장

## 【 로그 레벨 이해 】

| 로그 레벨           | 설명                   |
|-----------------|----------------------|
| <b>DEBUG</b>    | 상세 정보 (디버깅용)         |
| <b>INFO</b>     | 정상 동작 정보             |
| <b>WARNING</b>  | 경고, 문제가 될 수 있는 상황    |
| <b>ERROR</b>    | 오류 발생 (실행은 계속됨)      |
| <b>CRITICAL</b> | 치명적인 오류 (서비스 중단 가능성) |

# 로깅 구조 예시

```
import logging
from logging.handlers import TimedRotatingFileHandler
import os

로그 디렉토리 생성
log_dir = "logs"
os.makedirs(log_dir, exist_ok=True)

로거 생성
logger = logging.getLogger("MyApp")
logger.setLevel(logging.DEBUG)

1. 콘솔 핸들러
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.INFO)
console_format = logging.Formatter("[%%(levelname)s] %(message)s")
console_handler.setFormatter(console_format)
```

# 로깅 구조 예시

# 2. 파일 핸들러 (DEBUG 이상 모든 로그)

```
file_handler = TimedRotatingFileHandler(
 filename=f"{log_dir}/app.log",
 when="midnight", # 자정마다 회전
 interval=1,
 backupCount=7, # 최근 7일 보관
 encoding="utf-8"
)
file_handler.setLevel(logging.DEBUG)
file_format = logging.Formatter(
 "%(asctime)s | %(levelname)s | %(filename)s:%(lineno)d | %(message)s"
)
file_handler.setFormatter(file_format)
```

# 3. 에러 핸들러 (ERROR 이상만 별도 저장)

```
error_handler = logging.FileHandler(f"{log_dir}/error.log")
error_handler.setLevel(logging.ERROR)
error_handler.setFormatter(file_format)
```

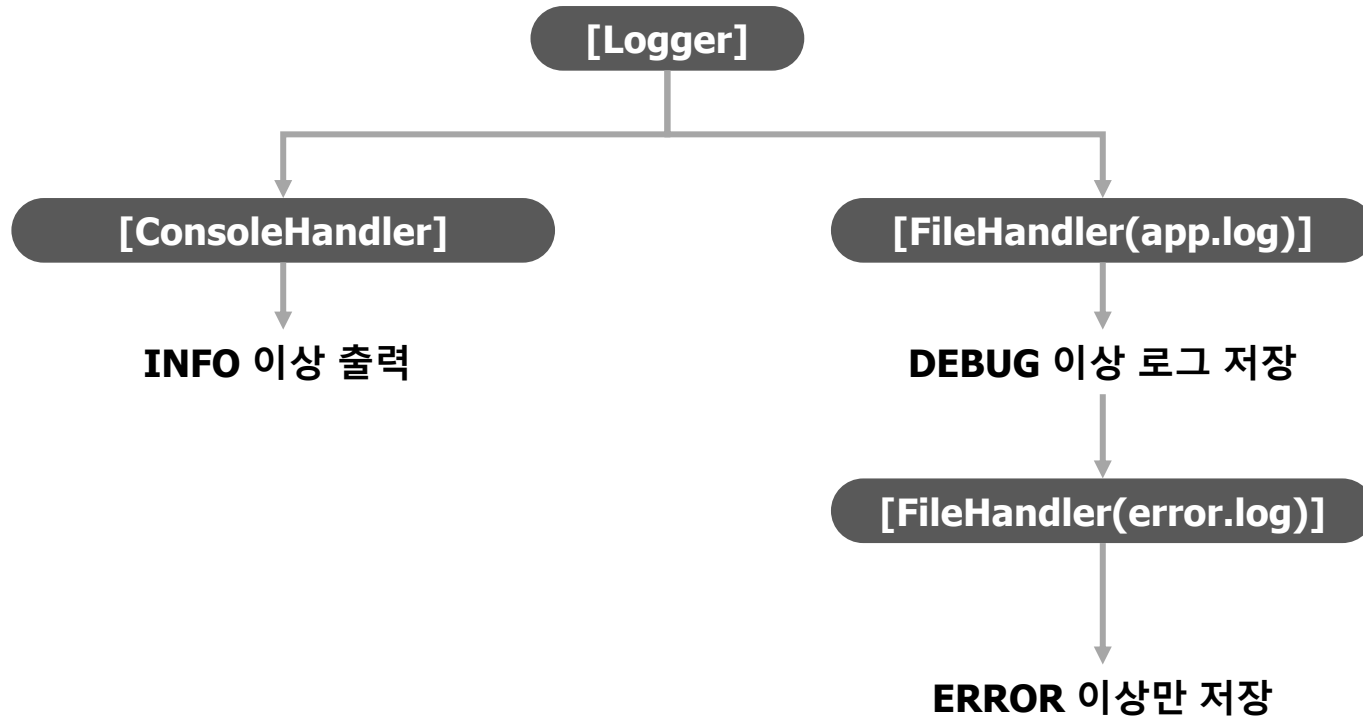
# 핸들러 등록

```
logger.addHandler(console_handler)
logger.addHandler(file_handler)
logger.addHandler(error_handler)
```

# 테스트 로그

```
logger.debug("디버그 정보")
logger.info("정상 흐름")
logger.warning("경고 메시지")
logger.error("에러 발생")
logger.critical("치명적 오류")
```

# 로깅 구조 구성도 (시각화)



# logging — print() 대신 프로덕션 로깅

## 【 코드 예시 】

### 핵심 내용

- ✓ 5단계 레벨:  
DEBUG·INFO·WARNING  
·ERROR·CRITICAL
- ✓ basicConfig : 빠른 기본 설정
- ✓ FileHandler : 파일에 저장
- ✓ TimedRotatingFileHandler  
: 날짜별 회전
- ✓ 분석 파이프라인에서 실행  
흐름 추적

```
logging 파이프라인 실전
import logging
from logging.handlers import TimedRotatingFileHandler

logger = logging.getLogger('pipeline')
logger.setLevel(logging.DEBUG)

콘솔: INFO 이상
ch = logging.StreamHandler()
ch.setLevel(logging.INFO)

파일: DEBUG 이상, 날짜별 회전
fh = TimedRotatingFileHandler(
 'logs/pipeline.log', when='midnight',
 backupCount=7, encoding='utf-8')
fh.setLevel(logging.DEBUG)

fmt = logging.Formatter('%(asctime)s|%(levelname)s|%(message)s')
[h.setFormatter(fmt) for h in [ch,fh]]
[logger.addHandler(h) for h in [ch,fh]]
```

# 왜 .env 파일이 필요한가?

## 보안

- 코드에 민감정보(DB 비밀번호, API 키 등)를 직접 하드코딩하면 안 됨
- **GitHub** 등에 실수로 올릴 경우 보안 사고 발생 가능

## 이식성

- 운영환경/개발환경/로컬환경 별 설정을 **.env** 파일로 손쉽게 전환 가능

## 재현성

- 다른 개발자에게 **.env.example** 파일만 공유하면 동일한 환경 구성 가능



## 【 환경변수 】

- 운영체제나 프로그램 실행 환경에서 설정된 전역 변수

## 관련 뷰

```
--Linux/macOS
export DB_USER=admin
export DB_PASS=secret123
```



```
import os
print(os.getenv("DB_USER")) # admin
```

**Python 호출**

# env 파일 형식

## 【 .env 예시 】

```
환경설정 파일
DEBUG=True
DB_HOST=localhost
DB_PORT=5432
DB_USER=myuser
DB_PASS=mypassword
```

- 주석은 # 사용
- 문자열에 따옴표는 선택

# Python에서 .env 읽는 방법

## 【 python-dotenv 설치 】

- `pip install python-dotenv`



## 사용 예제

```
from dotenv import load_dotenv
import os

load_dotenv() # .env 파일 자동 로드

db_user = os.getenv("DB_USER")
db_pass = os.getenv("DB_PASS")

print(f"DB 사용자: {db_user}, 비밀번호: {db_pass}")
```

- `.env`는 현재 경로 기준으로 조회
- 루트 디렉터리 위치 확인 필요

# .env + Git 연동시 주의사항

【 반드시 .gitignore에 추가 】

- .env



## .env.example 생성

```
.env.example
DB_USER=
DB_PASS=
DEBUG=
```

- 팀원들은 이 파일 복사해서 .env 파일로 사용하도록 함

## 【 보안 강화 팁 】

| 항목              | 설명                             |
|-----------------|--------------------------------|
| <b>.env 암호화</b> | dotenv-vault 같은 툴로 관리          |
| <b>CI/CD 연동</b> | GitHub Actions의 Secrets로 대체 가능 |
| <b>다중환경 관리</b>  | .env.dev, .env.prod 등 분리 가능    |

# .env 기반 환경변수 — 보안과 이식성

## 핵심 내용

- ✓ .env 파일  
: API 키·DB 비밀번호 분리
- ✓ 코드에 하드코딩 금지  
(GitHub 노출 위험)
- ✓ python-dotenv  
: load\_dotenv()로 로드
- ✓ os.getenv() : 환경변수 읽기
- ✓ 운영/개발/로컬 환경별 .env 분리  
전략

## 【 코드 예시 】

```
.env 보안 패턴
.env 파일 내용
API_KEY=sk-xxxx
DB_URL=postgresql://user:pass@host/db
ENV=development

from dotenv import load_dotenv
import os

load_dotenv() # .env 파일 로드

api_key = os.getenv('API_KEY')
db_url = os.getenv('DB_URL')
env = os.getenv('ENV', 'development')

.gitignore 에 반드시 추가
.env
.env.local
.env.production
```

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 4. 타입 힌트와 데이터 모델링

- 타입 힌트 · Union · Optional · Literal · Pydantic v2 · mypy

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 기본 타입 힌트

## 【 개념 】

- 런타임에는 영향을 미치지 않지만, 에디터에서의 자동완성과 정적 검사 도구의 오류 탐지에 큰 도움



## 예제 및 실무 예시

```
def add(x: int, y: int) -> int:
 return x + y
```



# typing 기본: List, Dict, Optional

## 【 List, Dict 】

### ▪ 예제

```
from typing import List, Dict

names: List[str] = ["Alice", "Bob"]
ages: Dict[str, int] = {"Alice": 25, "Bob": 30}
```

## 【 Optional 】

### ▪ 예제

```
from typing import Optional

def greet(name: Optional[str] = None) -> str:
 if name:
 return f"Hello, {name}"
 return "Hello, anonymous"
```

## 【 Optional 】

### ▪ 예제

```
from typing import Optional

def greet(name: Optional[str] = None) -> str:
 if name:
 return f"Hello, {name}"
 return "Hello, anonymous"
```

## Key Features

### 【 Optional[X] == Union[X, None] 】

- **Optional[str]** → "문자열(str)" 또는 "None"이 올 수 있다"는 의미
- 내부적으로는 **Union[str, None]** 와 동일

### 【 사용 이유 】

| 상황                         | 설명                                            |
|----------------------------|-----------------------------------------------|
| 초기 값이 없을 수 있음              | <b>def func(id: Optional[int])</b>            |
| 파라미터 생략 가능                 | <b>def send(msg: Optional[str] = None)</b>    |
| 반환값이 있을 수도 없을 수도 (DB 조회 시) | <b>def find(id: int) -&gt; Optional[User]</b> |

**주의: Optional**은 타입만을 위한 표현

- **Optional[str]**은 실제로는 아무 동작을 하지 않음
- 정적 분석 도구(**mypy**, **pyright** 등) 이 있을 때만 오류 탐지

# Union과 Any, Literal

## 【 Union과 Any, Literal 】

### ▪ 예제

```
from typing import Union, Any, Literal

Union: 여러 타입 중 하나
def parse(value: Union[str, int]) -> str:
 return str(value)

Any: 모든 타입 허용 (비추천)
def log(data: Any):
 print(f"Logging: {data}")

Literal: 특정 값만 허용
def turn(direction: Literal["left", "right"]):
 print(f"Turning {direction}")
```

## 【 주요 내용 】

### Union[str, int] | 여러 타입 중 하나 허용

```
def parse(value: Union[str, int]) -> str:
 return str(value)
```

- **value**로 문자열(**str**) 또는 정수(**int**) 를 받음
- 내부에서 **str(value)**로 문자열로 변환해서 반환
- 즉, 아래처럼 모두 허용

```
parse("hello") # OK
parse(123) # OK
parse(3.14) # X float은 타입 오류 (mypy 기준)
```

→ **Union[X, Y]**: 매개변수가 **X** 또는 **Y** 중 하나 단, 런타임에는 영향 없음  
**mypy** 같은 도구가 검사할 때만 유효

# Union과 Any, Literal

## 【 Union과 Any, Literal 】

### ▪ 예제

```
from typing import Union, Any, Literal

Union: 여러 타입 중 하나
def parse(value: Union[str, int]) -> str:
 return str(value)

Any: 모든 타입 허용 (비추천)
def log(data: Any):
 print(f"Logging: {data}")

Literal: 특정 값만 허용
def turn(direction: Literal["left", "right"]):
 print(f"Turning {direction}")
```

## 【 주요 내용 】

**Any** | 아무 타입이나 허용 (하지만 비추천)

```
def log(data: Any):
 print(f"Logging: {data}")
```

- **Any**는 모든 타입을 허용
- 타입 제한 없음
  - 타입 체크를 포기한 것과 같기 때문에  
유지보수성, 자동완성 기능 저하  
오타나 잘못된 인자도 통과됨

```
log("hello") #문자열
log(123) # 숫자
log([1, 2, 3]) # 리스트
log(None) #None
```

→ **Any**는 필요할 때만 사용  
웬만하면 쓰지 말 것

# Union과 Any, Literal

## 【 Union과 Any, Literal 】

### ▪ 예제

```
from typing import Union, Any, Literal

Union: 여러 타입 중 하나
def parse(value: Union[str, int]) -> str:
 return str(value)

Any: 모든 타입 허용 (비추천)
def log(data: Any):
 print(f"Logging: {data}")

Literal: 특정 값만 허용
def turn(direction: Literal["left", "right"]):
 print(f"Turning {direction}")
```

## 【 주요 내용 】

**Literal["left", "right"]** | 딱 정해진 값만 허용

```
def turn(direction: Literal["left", "right"]):
 print(f"Turning {direction}")
```

- 이 함수는 **direction**이 반드시 **"left"** 또는 **"right"** 중 하나
- 그 외 값은 정적 검사 도구에서 오류로 판단

```
turn("left") # OK
turn("right") # OK
turn("up") # X mypy 등에서 오류
```

→ 매개변수로 고정된 값을 제한할 때 사용  
특히 **enum** 비슷한 조건 제어, 파라미터 값 제한에 유용

## 【 요약 】

| 문법                       | 의미         | 사용 목적         |
|--------------------------|------------|---------------|
| <b>Union[str, int]</b>   | 둘 중 하나 허용  | 입력 타입 유연성 확보  |
| <b>Any</b>               | 아무 타입이나 허용 | 타입 강제 해제(비추천) |
| <b>Literal["A", "B"]</b> | 특정 값만 허용   | 고정된 선택지 강제    |

# Generic\_타입 안정성을 높이면서 재사용 가능한 구조를 만들 때 필수

## 【 Generic — 타입을 파라미터처럼 】

- Java/C++의 템플릿과 같은 개념

```
from typing import TypeVar, Generic, List
```

```
T = TypeVar("T") # T는 어떤 타입이든 될 수 있음
```

```
class Stack(Generic[T]): # Stack 클래스는 T 타입을 사용하는 제네릭 클래스
```

```
 def __init__(self):
 self._items: List[T] = [] # T 타입으로 이루어진 리스트
```

```
 def push(self, item: T) -> None:
 self._items.append(item) # 같은 타입만 추가 가능
```

```
 def pop(self) -> T:
 return self._items.pop() # 같은 타입을 꺼냄
```

```
s_int = Stack[int]()
s_int.push(10)
print(s_int.pop()) # 정수 전용 Stack
```

```
s_str = Stack[str]()
s_str.push("hello")
```

## Key Features

| 키워드                 | 의미                                   |
|---------------------|--------------------------------------|
| <b>TypeVar("T")</b> | 타입 파라미터 T를 선언                        |
| <b>Generic[T]</b>   | 이 클래스는 "타입 T"를 파라미터로 받는 제네릭 클래스임을 선언 |
| <b>Stack[int]()</b> | T에 int를 넣은 인스턴스 (정수 전용 스택)           |
| <b>Stack[str]()</b> | T에 str을 넣은 인스턴스 (문자열 전용 스택)          |

## 【 사용 목적 】

- 재사용성
  - int, str, float 등 어떤 타입에도 사용할 수 있는 공통 구조
- 정적 타입 안전성 확보
  - mypy, Pyright 등이 push() 시점에서 타입 오류 탐지 가능
- IDE 자동완성 도움
  - Stack[str]이면 .pop()의 리턴값이 str로 자동완성됨

\* mypy : Python에서 타입 힌트(type hint)를 정적 검사 도구  
즉, 코드 실행 없이도 타입 오류를 찾아주는 툴 (pip install mypy)

# Protocol — Duck Typing을 위한 정적 검사

## 【 Protocol 】

- 실제 상속 없이도 인터페이스 기반 타입 체크가 가능하도록 함

\* Duck Typing : 객체의 형(type) 보다는  
행동(method)에 따라 처리하는 방식

```
from typing import Protocol
```

```
class SupportsWrite(Protocol):
```

```
① "write(str) 메서드를 가진 객체"를 정의
def write(self, s: str) -> None: ...
```

```
def write_hello(writer: SupportsWrite) -> None:
```

```
② writer는 write를 지원해야 함
writer.write("Hello\n")
```

```
class FileLike:
```

```
③ 실제 구현 클래스 (Protocol 상속 안 해도 OK)
```

```
def write(self, s: str) -> None:
 print(f"Writing: {s}")
```

```
write_hello(FileLike()) # ④ FileLike는 write() 있으므로 OK
```

## Key Features

| 특징          | 설명                            |
|-------------|-------------------------------|
| 타입 안정성      | 런타임 전에 .write() 없으면 오류로 감지 가능 |
| 유연성 유지      | FileLike는 Protocol 상속 없이도 동작  |
| 강제 인터페이스 선언 | JAVA/C++의 Interface 같은 역할     |
| IDE 자동 완성   | 타입 추론이 정확해짐                   |

## 【 사용 목적 】

- 다양한 객체들이 같은 인터페이스(write, send 등)를 지원할 때
- 라이브러리 사용자에게 규약을 명확히 전달하고 싶을 때
- mypy, pyright 등을 활용한 정적 타입 검사 강화



# Callable — 함수 타입 정의

## 【 Callable 】

- Callable[[ArgType1, ArgType2], ReturnType]
  - 함수를 인자로 받는 함수의 타입을 명확하게 정의하는 예제

```
from typing import Callable
```

```
def compute(x: int, y: int, op: Callable[[int, int], int]) -> int:
 return op(x, y)
```

```
result = compute(3, 4, lambda a, b: a + b)
```

## Key Features

| 구성               | 의미                   |
|------------------|----------------------|
| <b>Callable</b>  | 함수 타입 정의용            |
| <b>[...]</b>     | 인자의 타입들 (리스트 형태로 나열) |
| <b>-&gt; ...</b> | 반환 타입                |

**Callable[[str, float], bool]** : 문자열과 실수를 받아 bool 반환

```
from typing import Callable, List
```

```
def sort_by(data: List[int], key_func: Callable[[int], int]) -> List[int]:
 return sorted(data, key=key_func)
```

```
data = [3, 5, 1, 7]
```

```
print(sort_by(data, lambda x: -x)) # 내림차순 정렬
```

# 정적 타입 검사 도구 mypy 활용

## 【 mypy 】

### ■ 예제

```
test.py
def add(x: int, y: int) -> int:
 return x + y

add(1, 2) # OK
add("a", "b") # X 타입 오류, 런타임에서는 실행
```

### ■ 실행 및 출력

```
mypy test.py

test.py:5: error: Argument 1 to "add" has
incompatible type "str"; expected "int"
test.py:5: error: Argument 2 to "add" has
incompatible type "str"; expected "int"
Found 2 errors in 1 file (checked 1 source file)
```

## Key Features

### 【 사용 이유 】

| 이유         | 설명                       |
|------------|--------------------------|
| 버그 예방      | 코드 실행 전 타입 오류 조기 발견 가능   |
| 코드 자동완성 향상 | IDE가 더 정확한 타입 추론         |
| 유지보수 편리    | 협업 시 명확한 Interface 규약 제공 |
| 테스트 비용 감소  | 일부 오류는 테스트 없이도 사전 차단 가능  |

### 【 프로젝트에서 자주 쓰는 명령어 】

```
특정 파일 검사
mypy main.py
```

```
특정 폴더 검사
mypy src/
설정파일 지정
mypy --config-file mypy.ini
```

```
[mypy]
python_version = 3.11
strict = True
disallow_untyped_defs = True
ignore_missing_imports = True
```

- `strict = True`: 가장 강력한 타입 검사 옵션
- `disallow_untyped_defs = True`: 모든 함수에 타입 힌트를 강제

## 【 정리 】

| 문법                               | 용도             |
|----------------------------------|----------------|
| <b>List[str], Dict[str, int]</b> | 컨테이너 타입 지정     |
| <b>Optional[int]</b>             | int or None 허용 |
| <b>Union[str, int]</b>           | 여러 타입 중 하나 허용  |
| <b>Generic[T]</b>                | 타입 파라미터 일반화    |
| <b>Protocol</b>                  | 인터페이스 기반 구조 정의 |
| <b>Callable</b>                  | 함수의 시그니처 지정    |
| <b>mypy, pyright</b>             | 정적 타입 체크 도구    |

# 타입 힌트 기초 — 분석 코드 가독성 향상

## 핵심 내용

- ✓ 기본: `str·int·float·bool·None`
- ✓ 컬렉션:  
`list[str]·dict[str,Any]·tuple[int,...]`
- ✓ 함수 시그니처  
: 매개변수 + 반환 타입
- ✓ 타입 힌트는 런타임에 영향 없음  
(주석 역할)
- ✓ VS Code Pylance가 실시간으로 타입  
검사

## 【 코드 예시 】

```
타입 힌트 기초
from typing import Any

기본 타입 힌트
def clean_name(s: str) -> str:
 return s.strip().lower()

def compute_avg(values: list[float]) -> float:
 return sum(values) / len(values)

def lookup(d: dict[str, Any], key: str,
 default: float = 0.0) -> float:
 return float(d.get(key, default))

3.10+ 문법: | 사용 가능
def parse(val: str | int | None) -> str:
 return str(val) if val is not None else ""
```

# Optional · Union · Literal · Any

## 핵심 내용

- ✓ **Optional[X] = X | None**  
— 결측치 표현
- ✓ **Union[X,Y]**  
: 여러 타입 허용 (3.10+: X|Y)
- ✓ **Literal**  
: 특정 값만 허용 (enum 대안)
- ✓ **Any**: 모든 타입 허용 — 사용 최소화
- ✓ **Callable** : 함수를 인자로 받을 때

## 【 코드 예시 】

```
Optional·Union·Literal
from typing import Optional, Union, Literal, Callable

Optional: 결측치 허용
def get_val(d: dict, key: str) -> Optional[float]:
 return d.get(key) # None 반환 가능

Literal: 특정 값만 허용
def fill_strategy(
 method: Literal['mean', 'median', 'drop']
) -> None: ...

Callable: 함수를 인자로
def apply_fn(
 data: list[float],
 fn: Callable[[float], float]
) -> list[float]:
 return [fn(x) for x in data]
```

# Pydantic v2 — 데이터 검증과 직렬화

## 핵심 내용

- ✓ **BaseModel**  
: 스키마 정의 + 자동 검증
- ✓ **Field:**  
**gt/lt/ge/le/regex/description**
- ✓ **model\_validate()**  
: dict → 객체 변환
- ✓ **model\_dump()**  
: 객체 → dict/JSON
- ✓ **ValidationError**  
: 어느 필드가 왜 틀렸는지

## 【 코드 예시 】

```
Pydantic v2 실전
from pydantic import BaseModel, Field
from typing import Optional

class SalesRecord(BaseModel):
 date: str
 region: str
 amount: float = Field(gt=0, description='양수')
 category: Optional[str] = None

검증 성공
r = SalesRecord(**{'date': '2024', 'region': '서울', 'amount': 1500})
r.model_dump() # dict 변환

검증 실패
try:
 SalesRecord(date="", region='서울', amount=-100)
except Exception as e:
 print(e) # 어느 필드, 왜 실패했는지 상세
```

런타임 전 타입 오류를 IDE와 CLI에서 동시에 잡아낸다  
CI에 통합하면 코드 품질 자동 게이트

### 【 코드 예시 】

```
타입 오류가 있는 코드
def compute_avg(values: list[float]) -> float:
 return sum(values) / len(values)

result = compute_avg(['a', 'b', 'c']) # list[str] ← 오류!

mypy 실행
$ mypy analysis.py

analysis.py:4: error: Argument 1 has incompatible type
'list[str]'; expected 'list[float]'
```

→ 데이터 분석 코드의 컬럼 타입 오류,  
결측치 처리 누락을 사전에 차단

# Pydantic을 분석 파이프라인에 활용하기

## 핵심 내용

- ✓ CSV 행 → BaseModel로 일괄 검증
- ✓ API JSON 응답  
→ 스키마 검증 후 저장
- ✓ 잘못된 행 건너뛰기 vs 오류 집계
- ✓ model\_json\_schema()  
: OpenAPI 스키마 자동 생성
- ✓ 후속 과목(FastAPI) API 스키마와  
동일한 구조

```
파이프라인 검증 패턴
from pydantic import BaseModel, ValidationError
import csv

class Record(BaseModel):
 date: str
 amount: float
 region: str

valid, errors = [], []

with open('sales.csv') as f:
 for i, row in enumerate(csv.DictReader(f)):
 try:
 valid.append(Record(**row))
 except ValidationError as e:
 errors.append({'row': i, 'error': str(e)})

print(f'유효: {len(valid)}건, 오류: {len(errors)}건')

오류 리포트 저장
import json
Path('errors.json').write_text(json.dumps(errors,
ensure_ascii=False))
```



# 타입 시스템이 데이터 분석 코드를 바꾸는 이유

## 컬럼명 오타 사전 차단

- amount vs Amount  
— Pydantic ValidationError가 런타임 전에 잡아냄
- 수백만 행 처리 후 발견보다 낫다

## 팀 협업 계약서

- 함수 시그니처에 타입이 있으면 teammate가 잘못된 인자를 전달하기 어려움
- 문서 없이도 인터페이스 명확

## IDE 자동완성 품질 향상

- 타입 정보 → Pylance가 정확한 자동완성과 오류 하이라이팅 제공
- 개발 속도와 정확성 동시 향상

## 후속 과목 직접 연결

- FastAPI·SpringBoot 연동 시 JSON 스키마 정의가 필수
- Pydantic BaseModel이 그 역할을 담당

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## [Practice 2]

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# [실습 2] 파일 I/O, 예외 처리, Pydantic 검증 파이프라인

## Python\_Practice1\_Data.json 활용

### 1) 예외 처리 + 파일 읽기

- 함수명 : `safe_load_csv()`
- 완성파일 없으면 `None` 반환·`logger.error`, 성공 시 `dict` 리스트 반환·`logger.info`, `finally`에서 '로딩 종료' 출력

### 2) Pydantic v2 스키마 정의

- `SalesRecord` 모델을 완성
- `month·region`: 비어있으면 안 됨  
`amount`: 0 초과  
`category`: 없어도 됨

### 3) 검증 파이프라인 (`valid` / `errors` 분리)

- `raw_data`를 순회하며 `SalesRecord`로 변환
- 성공→`valid`, 실패→`errors({row, error})` 리스트

### 4) 결과 파일 저장 + 재로딩 확인

- `valid` 레코드를 CSV로, `errors`를 JSON으로 저장하고 다시 읽어 건수를 검증

# [실습 2] 파일 I/O, 예외 처리, Pydantic 검증 파이프라인

## Checkpoint

- `safe_load_csv` 동작 + `assert None` 통과
- `ValidationError` 발생 시 오류 내용 출력
- `valid` 4건 / `errors` 3건 `assert` 통과
- 재로딩 후 `len(reloaded)==4` 통과

## 감점 대상 (감점점수)

- `try-except` 없이 파일 읽기 (-3)
- `finally` 블록 누락 (-1)
- `ValidationError` 대신 `Exception`으로 (-1)
- `model_dump()` 대신 `dict` 직접 구성 (-1)
- `json.dump ensure_ascii=False` 미설정 (한글 깨짐, -1)

# 평가 기준

## 캠퍼스명\_반\_이름. py로 제출 / Practice Rule (총점 100점)

### Code의 Comm. (20점)

- Python Code를 통한 이해관계자와의 Comm.
  - 프로그램 전체 설명 및 변경내역(머리말)
  - 함수/기능 설명(중간)

### 코드 간결성 (35점)

- 불필요한 반복 지양

### 오류/예외 처리 (35점)

- 예외, 오류에 대한 코드 반영 여부  
(간단한 코드라도 오류/예외 처리 연습 목적)

### 납기(프로젝트 연습, 10점)

- 기한 내 납기
  - 1시간 내 제출 완료

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 5. 코드 품질과 기초 테스트

- Ruff · pytest · coverage · pre-commit · VS Code 디버거

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 디버깅

## ✓ 버그란?

- 프로그램이 예상대로 동작하지 않는 원인
- 입력 오류, 논리 오류, 실행 오류 등 다양한 유형 존재

## ✓ 디버깅(Debugging) 과정

- 문제 재현 → 원인 분석 → 해결 방법 탐색 및 적용 → 테스트 후 정상 작동 확인

| 예외 타입                    | 설명                    | 예제                                       |
|--------------------------|-----------------------|------------------------------------------|
| <b>SyntaxError</b>       | • 문법 오류               | • <code>print("Hello"</code> (괄호 닫힘 없음)  |
| <b>NameError</b>         | • 정의되지 않은 변수 사용       | • <code>print(x)</code> (x가 정의되지 않음)     |
| <b>TypeError</b>         | • 잘못된 타입 사용           | • <code>"2" + 2</code> (문자열과 숫자 연산)      |
| <b>IndexError</b>        | • 리스트 인덱스 범위 초과       | • <code>my_list[5]</code> (길이가 5 미만인 경우) |
| <b>KeyError</b>          | • 딕셔너리에서 존재하지 않는 키 접근 | • <code>my_dict["없는키"]</code>            |
| <b>ValueError</b>        | • 변환할 수 없는 값을 사용      | • <code>int("hello")</code>              |
| <b>AttributeError</b>    | • 존재하지 않는 속성에 접근      | • <code>"hello".push()</code>            |
| <b>ZeroDivisionError</b> | • 0으로 나누기             | • <code>10 / 0</code>                    |

# 디버깅

## print()를 활용한 디버깅

- 변수 값을 중간중간 출력하여 확인
- 실행 흐름 추적

```
def add_numbers(a, b):
 print(f"DEBUG: a={a}, b={b}") # 디버깅 출력
 return a + b

print(add_numbers(3, 5))
```

→ 단점: 코드가 지저분  
삭제해야 할 print 문이 많아질 수 있음

## logging 모듈을 활용한 디버깅

- print() 대신 logging을 사용 시, 실행 환경에 따라 출력 방식 조정 가능

```
import logging

logging.basicConfig(level=logging.DEBUG) # 로그 레벨 설정
logging.debug("디버깅 메시지")
logging.info("정보 메시지")
logging.warning("경고 메시지")
logging.error("에러 메시지")
logging.critical("심각한 오류 메시지")
```

- 파일로 로그 저장하기

```
logging.basicConfig(filename="app.log",
 level=logging.DEBUG, format="%(asctime)s -
 %(levelname)s - %(message)s")
logging.info("파일에 기록되는 로그 메시지")
```



# 디버깅

## traceback을 활용한 에러 로그 출력

- 변수 값을 중간중간 출력하여 확인
- 실행 흐름 추적

```
import traceback

try:
 result = 10 / 0
except Exception as e:
 print("에러 발생:", e)
 traceback.print_exc() # 전체 오류 정보 출력
```

## pdb(Python Debugger) 모듈을 활용한 디버깅

- 아래 명령어 실행 후 pdb prompt에서 점검

```
import pdb
```

```
def divide(a, b):
 pdb.set_trace() # 여기서 실행이 멈춤
 return a / b
print(divide(10, 2))
```

```
> test.py(6)divide()
-> return a / b
(Pdb)
```

| 명령어   | 설명         |
|-------|------------|
| n     | • 다음 줄 실행  |
| s     | • 함수 내부 이동 |
| c     | • 계속 실행    |
| p 변수명 | • 변수값 출력   |
| q     | • 디버깅 종료   |

# 디버깅

## assert 문을 활용한 디버깅

- assert는 코드 실행 중 특정 조건을 검사하는 데 유용
- 실무에서는 if문과 raise Exception을 함께 사용

```
def divide(a, b):
 assert b != 0, "b는 0이 될 수 없습니다!"
 return a / b

print(divide(10, 2))
print(divide(10, 0)) # AssertionError 발생
```

## 코드 스타일 검사 (flake8, pylint)

- flake8 설치 및 실행

```
pip install flake8
flake8 my_script.py
```

→ 불필요한 공백, 변수 명명 규칙 위반, 들여쓰기 오류 등을 자동 검사

- pylint : 코드 스타일뿐만 아니라 코드 품질까지 분석

```
pip install pylint
pylint my_script.py
```

```
● (RDE) > pip install pylint

Defaulting to user installation because normal site-packages is not writeable
Collecting pylint
 Downloading pylint-3.3.4-py3-none-any.whl (522 kB)
 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 522.3/522.3 KB 10.4 MB/s eta 0:00:00
Collecting dill>=0.2
 Downloading dill-0.3.9-py3-none-any.whl (119 kB)
 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 119.4/119.4 KB 20.2 MB/s eta 0:00:00
Collecting tomlkit>=0.10.1
 Downloading tomlkit-0.13.2-py3-none-any.whl (37 kB)
Requirement already satisfied: platformdirs>=2.2.0 in /config/.local/lib/python3.10/site-packages (from pylint) (4.3.6)
Collecting isort!=5.13.0,<7,>=4.2.5
 Downloading isort-6.0.0-py3-none-any.whl (94 kB)
 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 94.1/94.1 KB 15.3 MB/s eta 0:00:00
Collecting tomli>=1.1.0
 Downloading tomli-2.2.1-py3-none-any.whl (14 kB)
Collecting mccabe<0.8,>=0.6
 Downloading mccabe-0.7.0-py2.py3-none-any.whl (7.3 kB)
Collecting astroid<=3.4.0-dev0,>=3.3.8
 Downloading astroid-3.3.8-py3-none-any.whl (275 kB)
 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.2/275.2 KB 25.8 MB/s eta 0:00:00
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.10/dist-packages (from astroid<=3.4.0-dev0,>=3.3.8->pylint) (4.12.2)
Installing collected packages: tomlkit, tomli, mccabe, isort, dill, astroid, pylint
 WARNING: The scripts isort and isort-identify-imports are installed in '/config/.local/bin' which is not on PATH.
 Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
 WARNING: The scripts pylint, pylint-config, pyreverse and symilar are installed in '/config/.local/bin' which is not on PATH.
 Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
```

# 디버깅

```
Successfully installed astroid-3.3.8 dill-0.3.9 isort-6.0.0 mccabe-0.7.0 pylint-3.3.4 tomli-2.2.1 tomlkit-0.13.2
(*|default-system:professor) ~/workspace/collaboration/python_code
⊗ (RDE) > pylint Decorator1.py
zsh: command not found: pylint
(*|default-system:professor) ~/workspace/collaboration/python_code
⊗ (RDE) > /config/.local/bin/pylint Decorator1.py

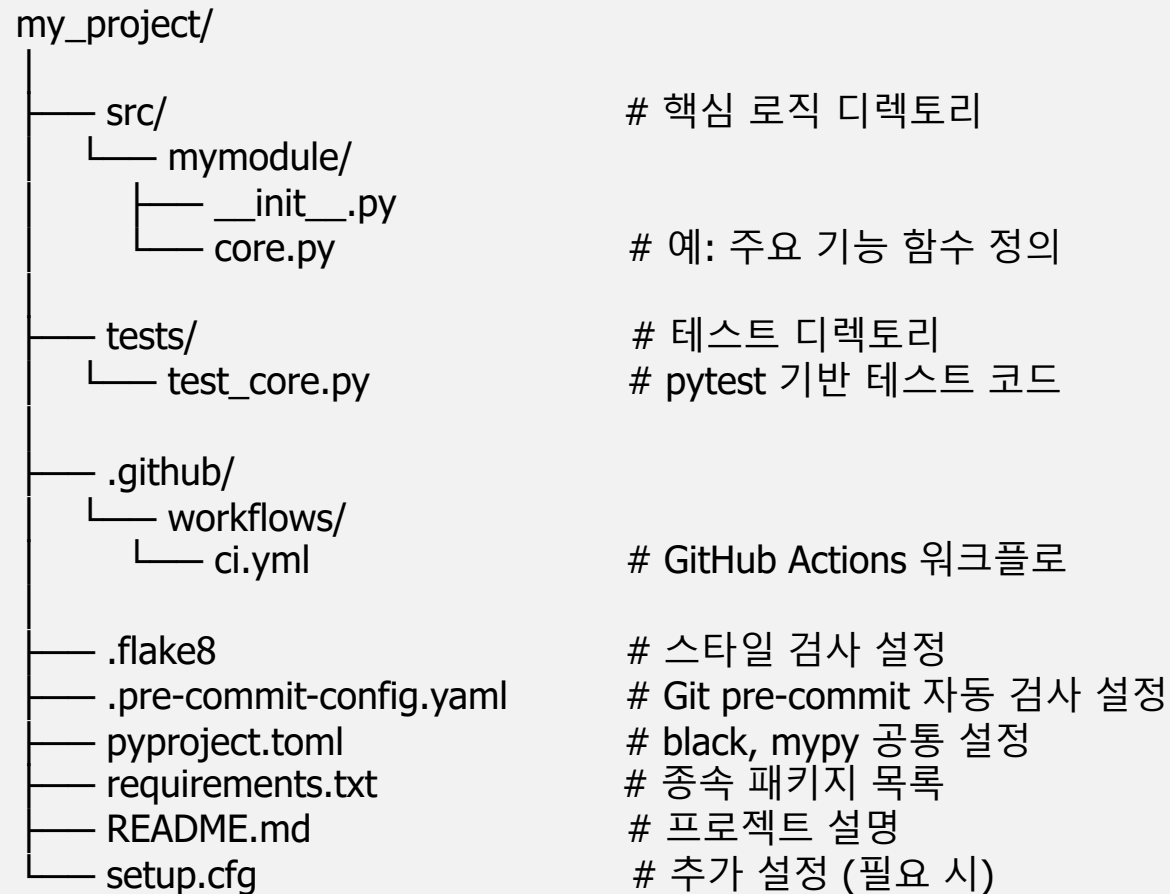
Decorator1.py:17:0: C0304: Final newline missing (missing-final-newline)
Decorator1.py:1:0: C0114: Missing module docstring (missing-module-docstring)
Decorator1.py:1:0: C0103: Module name "Decorator1" doesn't conform to snake_case naming style (invalid-name)
Decorator1.py:3:0: C0116: Missing function or method docstring (missing-function-docstring)
Decorator1.py:13:0: C0116: Missing function or method docstring (missing-function-docstring)

Your code has been rated at 6.15/10

(*|default-system:professor) ~/workspace/collaboration/python_code
⊗ (RDE) >
```

**/config/.local/bin/pylint** myscript.py

# 프로젝트 디렉토리 구조(예시)



# pytest - 테스트 자동화

## pytest

### ▪ 목적

- 함수 단위 테스트 및 통합 테스트
- 자동화CI/CD에서 신뢰성 검증 도구로 사용

### ▪ 실행 방식

- 현재 디렉토리 이하의 모든 test 파일을 자동 탐색하여 실행

### ▪ 기본 동작 규칙

- 파일 이름이 아래와 같아야 함  
test\_\*.py 혹은 \*\_test.py
- 함수 이름이 다음과 같아야 함  
test\_\*
- 현재 디렉토리 이하에서 재귀적으로 탐색  
→ pytest를 실행한 위치 기준으로  
서브폴더까지 모두 검사

## 【 예시 】

### 폴더 구조

```
my_project/
├── src/
│ └── mymodule/
│ └── core.py
├── tests/
│ └── test_core.py
```

```
src/mymodule/core.py
def add(a: int, b: int) -> int:
 return a + b
```

```
tests/test_core.py
from src.mymodule.core import add

def test_add():
 assert add(1, 2) == 3
```

### my\_project에서 실행

```
cd my_project
pytest
```

테스트 파일 또는 디렉토리 이름이 **test\_** 접두어 또는 **\_test** 접미어 형식을 따르지 않으면 무시

테스트 함수 이름이 **test\_**로 시작하지 않으면 실행되지 않음

**conftest.py** 같은 파일은 특별히 **hook**용으로 예외적으로 자동 탐지

## 【 특별한 실행(?) 】

| 구분                     | 예제 코드                                            |
|------------------------|--------------------------------------------------|
| 특정 디렉토리나 파일만 지정하고 싶다면? |                                                  |
| tests 디렉토리만 검사         | <code>pytest tests/</code>                       |
| src 내부도 검사             | <code>pytest src/</code>                         |
| 특정 파일만                 | <code>pytest tests/test_core.py</code>           |
| 강제로 특정 함수 실행           | <code>pytest tests/test_core.py::test_add</code> |



### 【 요약 】

| 항목       | 설명                   |
|----------|----------------------|
| 기본 탐색    | 현재 디렉토리 하위 전체        |
| 기본 파일 이름 | test_*.py, *_test.py |
| 함수 이름    | test_로 시작            |
| 특정 위치 지정 | pytest [디렉토리/파일 경로]  |
| 특정 함수 실행 | pytest 파일.py::함수명    |

# pytest – pytest.ini 설정

## pytest.ini

- 파일 위치
  - 보통 프로젝트 루트 디렉토리에 위치
  - pytest 실행 시 자동으로 탐지
- 위치 예시

```
my_project/
├── pytest.ini ← 여기에 위치
├── tests/
└── src/
```

# pytest – pytest.ini 설정

## 【 기본 예제 】

### pytest.ini

```
pytest.ini
[pytest]
minversion = 6.0
addopts = -ra -q --tb=short
testpaths =
 tests
python_files = test_*.py *_test.py
python_classes = Test*
python_functions = test_*
```

## 항목별 상세 설명

| 항목                      | 설명                           |
|-------------------------|------------------------------|
| <b>minversion</b>       | 최소 pytest 버전 요구 사항           |
| <b>addopts</b>          | 기본 실행 옵션 지정 (명령어 입력 없이 적용)   |
| <b>testpaths</b>        | 기본 테스트 폴더 지정 (eg, tests만 검사) |
| <b>python_files</b>     | 테스트로 인식할 파일 이름 패턴 지정         |
| <b>python_classes</b>   | 테스트 클래스로 인식할 클래스 이름 패턴       |
| <b>python_functions</b> | 테스트 함수 이름 패턴                 |

# pytest – pytest.ini 설정

## 【 기본 예제 】

### pytest.ini

```
pytest.ini
[pytest]
minversion = 6.0
addopts = -ra -q --tb=short
testpaths =
 tests
python_files = test_*.py *_test.py
python_classes = Test*
python_functions = test_*
```

## 항목별 상세 설명

| 항목                                     | 설명                                                                                                     |
|----------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>addopts = -ra -q --tb=short</b>     | • -rA : 모든 요약 출력 표시 (실패, 통과, 건너뛴 등)                                                                    |
|                                        | • -q : quiet 모드 (간결하게 출력)                                                                              |
|                                        | • --tb=short : traceback을 짧게 출력                                                                        |
| <b>testpaths = tests src/unittests</b> | <ul style="list-style-type: none"> <li>• 지정된 경로에서만 테스트를 검색</li> <li>• 루트 폴더에 .py 파일이 있어도 무시</li> </ul> |
| <b>python_files = check_*.py</b>       | • check_something.py 파일만 테스트로 인식                                                                       |

✓ **pytest 실행 시 pytest.ini에 설정된 옵션이 자동으로 적용**

# pytest 기초 — 분석 함수 단위 테스트

## 핵심 내용

- ✓ **test\_\*.py** 파일, **test\_** 함수 자동 인식
- ✓ **assert** 문으로 기댓값 검증
- ✓ **fixture**: 공통 데이터·설정 공유
- ✓ **parametrize**: 여러 입력 케이스 한 번에
- ✓ **pytest -v**: 상세 결과 / **-x**: 첫 실패 즉시 중단

## 【 코드 예시 】

```
pytest 분석 테스트
import pytest
import pandas as pd
from src.clean import clean_nulls

@pytest.fixture
def sample_df():
 return pd.DataFrame({'a':[1,None,3],'b':['x','y',None]})

def test_clean_removes_nulls(sample_df):
 result = clean_nulls(sample_df, cols=['a'])
 assert result['a'].isna().sum() == 0

def test_shape_preserved(sample_df):
 result = clean_nulls(sample_df)
 assert result.shape[0] == 3 # 행 수 유지

실행
$ pytest tests/ -v
$ pytest tests/ -x # 첫 실패시 중단
```

# pytest-cov — 테스트 커버리지 측정

어떤 코드 경로가 테스트되지 않았는지 파악. **MLOps** 과목의 **CI** 파이프라인에서 필수 지표.

## 【 코드 예시 】

```
$ pip install pytest-cov

커버리지 포함 실행
$ pytest tests/ --cov=src --cov-report=term-missing

출력 예시:
Name Stmts Miss Cover Missing
src/clean.py 25 3 88% 42-44, 67
src/features.py 38 0 100%
TOTAL 63 3 95%
```

→ 커버리지 **80%** 이상 목표  
**GitHub Actions CI**와 동일한 설정

# black – 코드 포맷터

## black

### ▪ 목적

- Python 코드의 스타일을 자동으로 정리
- PEP8을 기반으로 하지만, 자체 규칙도 적용
- 팀 간 코드 스타일 논쟁 제거
- git diff를 보기 좋게 유지

**Black의 철학**  
 “당신의 스타일은 중요하지 않다.  
 일관성이 더 중요하다.”

### 【 설치 】

#### 설치

```
pip install black
```

- 프로젝트 전체에 적용 시, **requirements.txt** 또는 **pyproject.toml**에 추가

### 【 사용방법 】

#### 기본 사용

```
black .
```

- 현재 디렉토리 하위의 모든 **.py** 파일을 자동 정리

#### 특정 파일만 포맷팅

```
black src/mymodule/core.py
```

#### 변경 사항 미리 보기 (check 모드)

```
black --check src/
```

- 수정 없이, 포맷 변경이 필요한지 검사만 함
- **CI**에서 유용 (**GitHub Actions** 등)

# black – 코드 포맷터

## 【 주요 규칙 】

```
BEFORE
def my_func(a=1,b=2): return a+b

AFTER (by black)
def my_func(a=1, b=2):
 return a + b
```



| 항목     | 설명                         |
|--------|----------------------------|
| 들여쓰기   | 공백 4칸 (PEP8과 동일)           |
| 줄 길이   | 기본 88자 (PEP8의 79자 보다 완화)   |
| 괄호 감싸기 | 함수 인자가 길면 자동으로 줄바꿈 및 괄호 사용 |
| 문자열    | ' 또는 " 중 하나로 통일 (보통 ')     |
| 연산자 공백 | 연산자 앞뒤 공백 유지 (a + b)       |



# black – 코드 포맷터

## 설정 파일: pyproject.toml

### ■ 설정파일(like ini)

- black은 설정 파일로 ***pyproject.toml***을 사용

```
pyproject.toml
[tool.black]
line-length = 88
target-version = ['py310']
include = '\.pyi?$'
exclude = '''
/(
 \.git
| \.venv
| build
| dist
)/
'''
```

### 【 주요 항목 】

| 항목                    | 설명                         |
|-----------------------|----------------------------|
| <b>line-length</b>    | 최대 줄 길이                    |
| <b>target-version</b> | 지원하는 Python 버전 (예 : py310) |
| <b>include</b>        | 대상 파일 패턴                   |
| <b>exclude</b>        | 무시할 폴더/파일 정규식              |

# black – 코드 포맷터

## 【 pre-commit 연동 예시 】

```
.pre-commit-config.yaml
- repo: https://github.com/psf/black
 rev: 24.3.0
 hooks:
 - id: black
```

pre-commit install

- git commit 전에 자동으로 포맷팅 적용

## 【 GitHub Actions 예시 】

```
.github/workflows/ci.yml
- name: Run black
 run: black --check .
```

## 【 장점 및 주의사항 】

| 구분   | 항목                     | 설명                                  |
|------|------------------------|-------------------------------------|
| 장점   | 자동화                    | 사람 개입 없이 스타일 정리                     |
|      | 일관성                    | 누구나 같은 결과                           |
|      | 빠름                     | 대규모 프로젝트도 빠르게 포매팅                   |
|      | 통합                     | <b>pre-commit, CI/CD</b> 에 쉽게 통합 가능 |
| 주의사항 | 커스터마이징 제한              | 스타일 규칙이 강제적이라 세부 조정 불가              |
|      | <b>flake8</b> 과 충돌 가능성 |                                     |
|      | 전체 변경 위험               | 기존 코드 스타일이 많으면 차이가 크게 발생할 수도...     |

# [참고] PEP8 (Python Enhancement Proposal)

## PEP란

### “Python 개선 제안서”

“Python 언어에 새로운 기능, 스타일, 정책 등을 제안할 때 작성하는 공식 문서”

- PEP 8 – 스타일 가이드  
(Style Guide for Python Code)
- PEP 7 – C 코드 스타일 가이드  
(for CPython C 코어 개발자)
- PEP 484 – 타입 힌트 제안
- PEP 572 – 할당 표현식 (:=)

## PEP 8 ?

| 항목       | 설명                                                                                                          |
|----------|-------------------------------------------------------------------------------------------------------------|
| 번호는 순번   | <ul style="list-style-type: none"> <li>• 8번째 제안서</li> <li>• 스타일 가이드 버전 8 아님</li> </ul>                      |
| 2001년 발표 | <ul style="list-style-type: none"> <li>• Python 창시자 Guido van Rossum 외 여러 코어 개발자 공동 작성</li> </ul>           |
| 목적       | <ul style="list-style-type: none"> <li>• “코드는 읽는 사람이 많고, 작성하는 사람보다 더 많다.”로 일관되고 가독성 좋은 코딩 스타일 제안</li> </ul> |
| 기반도구     | <ul style="list-style-type: none"> <li>• black, flake8, isort</li> </ul>                                    |

# pre-commit - 자동 검사 Hook

## pre-commit

### ■ 목적

- Git은 커밋 전/후 등 특정 시점에 자동 실행되는 Hook 기능 제공
- pre-commit은 그 중 커밋 직전(pre-commit)에 특정 검사를 자동으로 실행하는 도구
- 코드 포매팅, 린트, 타입 검사, 보안 검사 등을 커밋 전에 미리 실행하여 문제 있는 코드는 커밋을 차단

코드 품질을 "Git 단계에서" 지키기 위한 자동 방어

## 【 설치 】

### 설치

```
pip install pre-commit
pre-commit install
```

- 설치 후 프로젝트에 초기 설정
- **.git/hooks/pre-commit** 스크립트가 자동 생성되어 **Git 커밋 전에 실행**

## 【 실행방법 】

### 전체 파일 한 번에

```
pre-commit run --all-files
```

### 실제로는 커밋 시 자동 실행

```
git add .
git commit -m "Add new feature"
```

- **black, flake8, mypy** 자동 실행  
→ 실패 시 커밋 중단

# pre-commit - 자동 검사 Hook

```
.pre-commit-config.yaml
repos:
 - repo: https://github.com/psf/black
 rev: 24.3.0
 hooks:
 - id: black

 - repo:
 https://github.com/PyCQA/flake8
 rev: 6.1.0
 hooks:
 - id: flake8

 - repo: https://github.com/pre-
 commit/mirrors-mypy
 rev: v1.8.0
 hooks:
 - id: mypy
```

## 【 주요 구성 요소 】

| 키            | 설명                                |
|--------------|-----------------------------------|
| <b>repo</b>  | 도구의 Git 저장소                       |
| <b>rev</b>   | 사용할 버전 태그                         |
| <b>hooks</b> | 사용할 검사 항목 (black, flake8, mypy 등) |

# pre-commit - 자동 검사 Hook

```
1. 설정 파일 작성 (.pre-commit-config.yaml)
2. 설치
pre-commit install

3. 모든 파일 검사
pre-commit run --all-files

4. 커밋 시 자동 검사 실행됨
git commit -m "Initial commit"
```

**black, flake8, mypy를 pre-commit에 연결**

오류가 나면 커밋이 차단되고, 수정 후 다시 **add/commit** 하면 됨

# pre-commit - 자동 검사 Hook

```
my_project/
├── src/
├── tests/
├── .pre-commit-config.yaml
├── pyproject.toml
├── .flake8
└── requirements.txt
```

## # 코드 내 print 제거 검사

```
- repo: https://github.com/pre-commit/pre-commit-hooks
 rev: v4.5.0
 hooks:
 - id: check-added-large-files
 - id: check-merge-conflict
 - id: check-yaml
 - id: debug-statements
```

**debug-statements**는 **print()** 같은 디버깅 코드가 남아있을 경우 커밋을 차단

## 【 장점 】

| 항목         | 설명                   |
|------------|----------------------|
| 코드 품질 보장   | 린트, 포맷, 타입 검사를 자동 수행 |
| 협업 시 실수 방지 | 커밋 전에 자동 검사로 오류 방지   |
| 자동화 쉬움     | Git hook에 자동으로 붙음    |
| CI 연동 가능   | Github Action 와도 호환  |



# pre-commit · requirements.txt · 재현 환경

## 핵심 내용

- ✓ **pre-commit**  
: 커밋 전 Ruff·mypy 자동 실행
- ✓ 오류 있으면 커밋 차단  
→ 팀 코드 품질 유지
- ✓ **requirements.txt**  
: pip freeze로 버전 고정
- ✓ **pip install -r**  
: 동일 환경 1분 내 재현
- ✓ **pyproject.toml**  
: 현대적 프로젝트 설정 방식

## 【 코드 예시 】

```
pre-commit + 환경 관리
pre-commit 설치 및 설정
$ pip install pre-commit

.pre-commit-config.yaml
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
 hooks:
 - id: ruff
 args: [--fix]
 - id: ruff-format

$ pre-commit install # git hook 등록
$ pre-commit run --all-files # 전체 검사

환경 재현
$ pip freeze > requirements.txt
$ pip install -r requirements.txt

.gitignore 필수 항목
.venv/ .env __pycache__/ *.pyc .coverage
```

# GitHub Actions CI 설정

name: CI - Lint & Test

1

on: [push, pull\_request]

jobs:

quality-check:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- name: Set up Python

uses: actions/setup-python@v4

with:

python-version: '3.10'

- name: Install dependencies

run: |

python -m pip install --upgrade pip

pip install -r requirements.txt

2

- name: Run black  
run: black --check .

- name: Run flake8  
run: flake8 .

- name: Run mypy  
run: mypy src/

- name: Run pytest  
run: pytest

경로: **.github/workflows/ci.yml**

pytest  
black  
flake8  
mypy  
pre-commit

**requirements.txt 내용**

## 실전 적용 흐름 (예시)



## 【 .vscode/settings.json 예시 】

```
{
 "python.linting.enabled": true,
 "python.linting.flake8Enabled": true,
 "python.formatting.provider": "black",
 "python.linting.mypyEnabled": true,
 "editor.formatOnSave": true
}
```

***GitHub 프로젝트 초기 세팅 시 그대로 적용하면 팀 전체의 코드 품질과 협업 효율 향상 가능***

# Ruff — 2026 Python 린팅·포매팅 표준

## 핵심 내용

- ✓ **flake8 + black + isort**를 하나로 대체
- ✓ **Rust** 작성 — 기존 도구 대비 **10–100×** 빠름
- ✓ **E·F·I·UP** 규칙:  
에러·경고·임포트·업그레이드
- ✓ **--fix**: 자동 수정 가능한 오류 일괄 처리
- ✓ **VS Code** 저장 시 자동 포맷 설정 가능

## 【 코드 예시 】

```
Ruff 실전
$ pip install ruff

린팅 검사
$ ruff check .

자동 수정 (가능한 항목)
$ ruff check --fix .

포매팅
$ ruff format .

pyproject.toml 설정
[tool.ruff]
line-length = 88
select = ['E','F','I','UP']
ignore = ['E501'] # 긴 줄 무시

[tool.ruff.format]
quote-style = 'double'
indent-style = 'space'
```

# VS Code 디버거와 코드 품질 도구 통합 활용

## 중단점(Breakpoint) 활용

- F9로 중단점 설정 → F5 실행 → 변수 인스펙터로 DataFrame 내용 실시간 확인
- print 디버깅 탈출

## Ruff 저장 자동 포맷

- settings.json: editor.formatOnSave: true + editor.defaultFormatter: charliermarsh.ruff 설정

## pytest Test Explorer

- VS Code Python Test Explorer 확장 설치 → 사이드바에서 테스트 실행·결과 확인
- TDD 워크플로

## Pylance 타입 검사

- python.analysis.typeCheckingMode : basic 설정
- 저장할 때마다 타입 오류 빨간 밑줄로 즉시 표시

# 코드 품질 도구가 데이터 분석 팀에 필요한 이유

## 분석 코드도 팀이 공유한다

- 혼자 만든 분석 스크립트도 GitHub에 올리는 순간 팀 자산
- Ruff·pytest가 없으면 리뷰가 어렵다

## 재현 가능한 분석 = 신뢰

- 6개월 뒤 같은 결과가 나오는 분석이 좋은 분석
- requirements.txt + 테스트가 재현성을 보장

## MLOps CI/CD 기반

- DevOps·MLOps 과목에서 GitHub Actions로 pytest + Ruff를 자동 실행
- 지금 배운 설정을 그대로 사용

## 디버거 > print() 탈출

- 데이터 분석에서 DataFrame 상태를 print로 찍는 건 비효율
- 중단점으로 컬럼·타입·값 즉시 확인

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 6. 비동기와 병렬 처리 입문

▪ **asyncio · httpx · GIL · multiprocessing · cProfile**

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.



# 동시성 vs 병렬성 — 개념과 선택 기준

도구

- asyncio + httpx

- multiprocessing

실행  
방식

- 한 스레드, 대기 중 다른 작업 실행

- 여러 프로세스, 진짜 동시 실행

적합한  
작업

- I/O bound: API 호출, 파일 읽기

- CPU bound: 수치 계산, 이미지 처리

GIL 영향

- 영향 없음 (대기 중 전환)

- GIL 우회 (별도 프로세스)

데이터  
분석 적용

- 공공 API 대량 수집, 비동기 DB 쿼리

- 대용량 파일 병렬 전처리, 모델 훈련

# 병렬처리 vs. 병행처리 (Parallelism vs. Concurrency)

## 【 병렬처리 vs 병행처리 】

| 구분 | 병렬처리(Parallelism) | 병행처리(Concurrency)  |
|----|-------------------|--------------------|
| 의미 | 동시에 여러 작업 실행      | 여러 작업을 빠르게 번갈아 실행  |
| 전제 | 멀티코어 필요           | 싱글코어도 가능           |
| 예시 | A와 B가 동시에 계산됨     | A를 하다가 B로 전환, 다시 A |

- 멀티코어 CPU가 있다면 **multiprocessing**으로 병렬처리 가능
- 싱글코어 환경에서도 I/O 위주의 작업은 **threading**으로 효율적 실행 가능

# Python의 GIL(Global Interpreter Lock)

## 【 GIL(Global Interpreter Lock) 】

- Python(CPython)의 인터프리터에서 하나의 스레드만 실행되도록 강제하는 락
- CPU-bound 작업에서 병렬성 저하 발생
- I/O-bound 작업에서는 큰 문제 없음



## 계층형 권한 설계의 핵심 기능

```
import threading

def cpu_task():
 for _ in range(10**7):
 pass

threads = []
for _ in range(4):
 t = threading.Thread(target=cpu_task)
 threads.append(t)
 t.start()

for t in threads:
 t.join()
```

위 코드는 4개 스레드를 만들지만 GIL 때문에 실질적으로는 하나씩 처리됨

# Global Interpreter Lock (GIL)

Python의 인터프리터에서 동시 실행을 제한하는 상태 관리 메커니즘으로 **multithreading**과 관련된 중요한 개념

| 구분    | 설명                                                                                                                                                                                                                                                                                                                                        |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 개념    | <ul style="list-style-type: none"> <li>Python의 단일 프로세스에서 한 번에 하나의 스레드만 실행할 수 있게 하는 잠금</li> </ul>                                                                                                                                                                                                                                          |
| 목적    | <ul style="list-style-type: none"> <li>메모리 관리               <ul style="list-style-type: none"> <li>Python 메모리 관리 시스템에서 동기화를 도움</li> <li>여러 스레드가 동시에 같은 객체에 접근하거나 수정 시 발생하는 데이터 충돌 방지</li> </ul> </li> <li>간단한 구현               <ul style="list-style-type: none"> <li>GIL을 통해 multithreading을 쉽게 구현, 복잡한 동기화 문제 처리</li> </ul> </li> </ul> |
| 동작원리  | <ul style="list-style-type: none"> <li>Python의 GIL은 인터프리터 수준에서 동작               <ul style="list-style-type: none"> <li>Python 프로그램에서 여러 스레드를 사용하더라도, 실제로 한 스레드만 CPU에서 실행</li> </ul> </li> <li>주기적으로 한 스레드가 실행되는 시간을 할당하고, 일정 시간이 지나면 다음 스레드가 실행될 기회를 받음</li> </ul>                                                                        |
| 작동 예시 | <ul style="list-style-type: none"> <li>스레드가 동시에 여러 개 존재하지만, 실제로는 한 번에 하나의 스레드만 실행               <ul style="list-style-type: none"> <li>4개의 스레드를 가진 프로그램이 있다면, GIL은 그 중 한 스레드만 실행하고, 그 스레드가 멈추면 다른 스레드를 실행하는 방식으로 순차적으로 실행</li> </ul> </li> </ul>                                                                                          |

# Global Interpreter Lock (GIL)

Python의 인터프리터에서 동시 실행을 제한하는 상태 관리 메커니즘으로 **multithreading**과 관련된 중요한 개념

| 구분    | 설명                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 문제점   | <ul style="list-style-type: none"> <li>• 병렬성 제한               <ul style="list-style-type: none"> <li>- GIL 때문에 Python은 multithreading에서 진정한 병렬 실행 불가</li> <li>- 특히 CPU 집약적인 작업에서는 GIL이 성능을 제한<br/>eg) 여러 스레드를 사용하더라도 CPU 연산이 많은 작업을 여러 스레드에서 동시에 실행 불가<br/>수학적 계산, 데이터 처리, 암호화</li> </ul> </li> <li>• 멀티코어 활용 불가               <ul style="list-style-type: none"> <li>- GIL 때문에 여러 코어가 있는 컴퓨터에서도, multithreading을 사용한 작업은 하나의 코어에서만 실행<br/>즉, 여러 코어를 활용할 수 없습니다.</li> </ul> </li> </ul>                    |
| 회피 방법 | <ul style="list-style-type: none"> <li>• multiprocessing (Multiprocessing) 사용               <ul style="list-style-type: none"> <li>- GIL의 한계를 피하려면 multithreading 대신 multiprocessing을 사용</li> <li>- multiprocessing은 각 프로세스가 독립적인 메모리 공간을 갖기 때문에 GIL의 영향이 없고 병렬 처리 가능</li> </ul> </li> <li>• C 확장 모듈 사용               <ul style="list-style-type: none"> <li>- 일부 C 확장 모듈에서는 GIL을 해제하고 병렬 처리가 가능하도록 구현<br/>eg) NumPy와 같은 C 언어로 작성된 고성능 수치 계산 라이브러리는 GIL을 해제하여<br/>다중 코어에서 병렬 연산을 사용</li> </ul> </li> </ul> |

# [참고] NumPy와 같은 라이브러리에서 GIL 해제

## NumPy의 GIL 해제와 병렬 처리

- 배열 기반 연산을 C 레벨에서 처리하기 때문에, GIL을 해제하고 병렬 연산을 수행

```
import numpy as np
import time

두 개의 큰 배열을 생성
arr1 = np.random.rand(10000000)
arr2 = np.random.rand(10000000)

두 배열을 더하는 연산
start_time = time.time()
result = arr1 + arr2
NumPy가 내부적으로 최적화된 C 코드로 처리
end_time = time.time()

print(f"NumPy 배열 연산 시간: {end_time - start_time} 초")
```

→ `arr1 + arr2` 연산은 NumPy 내부의 C 구현을 사용하여 매우 빠르게 처리되며 이때 GIL이 해제되어 CPU에서 다중 코어를 사용

## NumPy의 병렬화와 BLAS 라이브러리

- 배열 연산을 처리할 때 BLAS (Basic Linear Algebra Subprograms) 라이브러리를 사용
  - 행렬 연산, 벡터 연산 등을 최적화, 병렬 처리가 가능한 방식으로 연산을 수행
- OpenMP와 같은 라이브러리로 다중 스레드를 활용
  - 행렬 곱셈이나 벡터-행렬 연산에서는 다중 코어를 활용하여 연산을 빠르게 처리

```
두 개의 큰 행렬을 곱하는 연산
A = np.random.rand(1000, 1000)
B = np.random.rand(1000, 1000)

start_time = time.time()
C = np.dot(A, B)
NumPy 내부적으로 BLAS 라이브러리 사용
end_time = time.time()

print(f"행렬 곱셈 시간: {end_time - start_time} 초")
```

→ 행렬 곱셈은 NumPy의 `np.dot` 함수 (내부적으로 BLAS 라이브러리를 사용하여 병렬 연산 수행) ∴ 이 덕분에 다중 코어 CPU를 활용해 연산이 훨씬 빠름

# threading 모듈 (경량 병렬 처리, I/O-bound에 적합)

## 【 threading 모듈 】

- 장점: 메모리 공유, 생성 빠름
- 단점: GIL로 인해 CPU-bound 작업에 비효율

## 예제

```
import threading

def worker(num):
 print(f"Thread {num} working")

threads = []
for i in range(5):
 t = threading.Thread(target=worker, args=(i,))
 threads.append(t)
 t.start()
```

```
lock = threading.Lock()
shared = 0
```

```
def task():
 global shared
 for _ in range(100000):
 with lock:
 shared += 1
```

동기화 예시: Lock

# multithreading (Multithreading)

## multithreading

### • 개념

- 하나의 프로세스 내에서 여러 스레드를 사용하여 병렬적으로 작업을 처리하는 기법
- 스레드는 프로세스 내에서 실행되는 흐름으로 하나의 프로세스에서 여러 스레드가 동시에 실행 가능

### • 특징

- 스레드는 같은 프로세스를 공유하므로 메모리와 자원을 공유
- multithreading은 주로 I/O 작업에 유리 (파일 입출력, 네트워크 통신 등)
- Global Interpreter Lock (GIL) 때문에 CPU 집약적인 작업에서는 성능 향상이 크지 않음

```
import threading
import time

스레드에서 실행할 함수
def print_numbers():
 for i in range(1, 6):
 print(i)
 time.sleep(1)

def print_letters():
 for letter in ['A', 'B', 'C', 'D', 'E']:
 print(letter)
 time.sleep(1)

스레드 생성
thread1 = threading.Thread(target=print_numbers)
thread2 = threading.Thread(target=print_letters)

스레드 시작
thread1.start()
thread2.start()

스레드가 끝날 때까지 기다리기
thread1.join()
thread2.join()
```

1

A

2

B

3

C

4

D

5

E

- 이 코드에서는 두 스레드가 병렬로 실행되어, 숫자와 알파벳을 동시에 출력
- time.sleep(1)로 각 스레드가 1초마다 일시 정지하도록 설정, 출력 순서를 확인



# multiprocessing 모듈 (프로세스 기반 병렬처리)

## 【 multiprocessing 모듈 】

- 각 프로세스는 별도 메모리 공간 → **GIL 회피 가능**
- **CPU-bound** 작업에 유리

## 예제

```
from multiprocessing import Process

def task(name):
 print(f"{name} is working")

if __name__ == "__main__":
 p1 = Process(target=task, args=("Process-1",))
 p2 = Process(target=task, args=("Process-2",))
 p1.start()
 p2.start()
 p1.join()
 p2.join()
```

```
from multiprocessing import Pool

def square(n):
 return n * n

with Pool(processes=4) as pool:
 result = pool.map(square, [1, 2, 3, 4])
 print(result) # [1, 4, 9, 16]
```

## Pool 예제

# Multiprocessing

## Multiprocessing

### • 개념

- 여러 개의 프로세스를 사용하여 작업을 병렬 처리하는 기법
- 각각의 프로세스는 독립된 메모리 공간과 자원을 갖고 실행
- CPU 집약적인 작업에 적합

### • 특징

- 각 프로세스는 별도의 메모리 공간을 갖기 때문에, 서로 영향을 미치지 않음
- 병렬 처리가 가능하여 CPU 성능을 더 잘 활용
- I/O 작업보다는 CPU 집약적인 작업에서 유리

- multiprocessing을 사용하면 두 프로세스가 완전히 독립적으로 실행되어, # 프로세스가 끝날 때까지 기다리기 두 작업이 병렬로 실행

```
import multiprocessing
import time

프로세스에서 실행할 함수
def print_numbers():
 for i in range(1, 6):
 print(i)
 time.sleep(1)

def print_letters():
 for letter in ['A', 'B', 'C', 'D', 'E']:
 print(letter)
 time.sleep(1)

프로세스 생성
process1 = multiprocessing.Process(target=print_numbers)
process2 = multiprocessing.Process(target=print_letters)

프로세스 시작
process1.start()
process2.start()

프로세스가 끝날 때까지 기다리기
process1.join()
process2.join()
```

1  
2  
3  
4  
5  
A  
B  
C  
D  
E

# queue vs multiprocessing.Queue

## 【 비교 】

| 구분       | queue.Queue | multiprocessing.Queue |
|----------|-------------|-----------------------|
| 대상       | 스레드 간       | 프로세스 간                |
| 공유 가능 여부 | 메모리 공유 가능   | 메모리 공유 불가 (직렬화 필요)    |

## 프로세스 간 통신

```
from multiprocessing import Process, Queue

def worker(q):
 q.put("Hello from child")

if __name__ == "__main__":
 q = Queue()
 p = Process(target=worker, args=(q,))
 p.start()
 print(q.get()) # Hello from child
 p.join()
```

# multithreading vs multiprocessing

| 특징                 | multithreading                    | multiprocessing                    |
|--------------------|-----------------------------------|------------------------------------|
| 작업 단위              | • 스레드                             | • 프로세스                             |
| 메모리 공유             | • 같은 프로세스 내에서 메모리 공유              | • 별도의 메모리 공간을 가짐                   |
| <b>GIL(Python)</b> | • GIL로 인해 CPU 집약적인 작업에서 성능 향상에 한계 | • GIL 영향 받지 않음, CPU 집약적 작업에서 성능 향상 |
| 적합한 작업             | • I/O 작업 (파일 입출력, 네트워크 등)         | • CPU 집약적인 작업 (계산, 데이터 처리 등)       |
| 자원 소비              | • 프로세스 내에서 자원 공유, 상대적으로 적음        | • 각 프로세스마다 독립적인 자원을 사용, 더 많은 자원 소비 |

# Threading vs Multiprocessing 선택 기준

## 【 기준 예시 】

| 상황               | 추천 방식                                                       |
|------------------|-------------------------------------------------------------|
| <b>I/O-bound</b> | threading                                                   |
| <b>CPU-bound</b> | multiprocessing                                             |
| 많은 수의 작업         | concurrent.futures.ThreadPoolExecutor / ProcessPoolExecutor |

# multithreading vs multiprocessing 선택 기준

| multithreading                                                                               | multiprocessing                                                                                |
|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• I/O 작업이 많은 경우<br/>(예: 파일 읽기/쓰기, 네트워크 요청)</li> </ul> | <ul style="list-style-type: none"> <li>• CPU 집약적인 작업이 많은 경우<br/>(예: 복잡한 계산, 데이터 처리)</li> </ul> |
| <ul style="list-style-type: none"> <li>• 스레드 간의 메모리 공유가 중요한 경우</li> </ul>                    | <ul style="list-style-type: none"> <li>• 작업이 독립적이고 메모리 공유가 필요 없는 경우</li> </ul>                 |
| <ul style="list-style-type: none"> <li>• GIL이 성능에 큰 영향을 미치지 않는 경우 (I/O 작업)</li> </ul>        | <ul style="list-style-type: none"> <li>• GIL을 회피하여 성능을 극대화하고 싶은 경우</li> </ul>                  |

# FastAPI, AI 모델 처리에서의 병렬처리 응용(예시)

## 【 FastAPI + BackgroundTasks 】

### ▪ 예제

```
from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

def process_file(file_id: int):
 # 병렬 처리할 작업
 ...

@app.post("/upload/")
def upload(file_id: int, background_tasks: BackgroundTasks):
 background_tasks.add_task(process_file, file_id)
 return {"status": "processing"}
```

## 【 AI 모델 병렬 추론 (GPU or Multi-core CPU 기반) 】

- PyTorch에서 DataLoader(num\_workers=4)  
→ 다중 프로세스로 데이터 전처리

```
from torch.utils.data import DataLoader
```

```
loader = DataLoader(dataset, batch_size=32,
 shuffle=True, num_workers=4)
```

- num\_workers=4
  - 데이터를 미리 로딩하고 전처리하기 위한 서브 프로세스 수를 의미
- 병렬로 4개의 프로세스가 실행되어 데이터 전처리 속도를 향상
- 각 worker는 \_\_getitem\_\_()을 호출하여 데이터를 읽고 collate\_fn을 통해 배치

- 딥러닝 모델은 빠른 연산이 가능하나, 데이터를 CPU에서 로딩하고 전처리하는 속도가 느리면 GPU가 Idle(대기) 상태
- num\_workers>0으로 설정
  - 데이터 로딩을 다중 프로세스로 병렬화 GPU가 다음 데이터를 기다리지 않도록 미리 준비(pre-fetching)

# multiprocessing — CPU bound 병렬 처리

## 【 코드 예시 】

### 핵심 내용

- ✓ **ProcessPoolExecutor**  
: 간편한 병렬 처리 API
- ✓ **Pool.map**  
: 리스트를 여러 프로세스로 병렬 처리
- ✓ **max\_workers**  
: CPU 코어 수 기준 설정
- ✓ **chunk 분할**  
: 대용량 파일을 N등분해 병렬 처리
- ✓ **주의**  
: 프로세스 간 데이터 전달 비용 고려

```
multiprocessing 실전
from concurrent.futures import ProcessPoolExecutor
import multiprocessing as mp

def process_chunk(chunk):
 # CPU 집중 전처리
 return [transform(row) for row in chunk]

파일을 청크로 분할
def split_chunks(data, n):
 size = len(data) // n
 return [data[i*size:(i+1)*size] for i in range(n)]

n_cores = mp.cpu_count() # macOS M1: 8코어
chunks = split_chunks(all_data, n_cores)

with ProcessPoolExecutor(max_workers=n_cores) as exe:
 results = list(exe.map(process_chunk, chunks))
8코어: 이론상 8x 속도 향상
```



# GIL(Global Interpreter Lock) — Python 동시성의 핵심 제약

## 【 코드 예시 】

### 핵심 내용

- ✓ **CPython** : 한 번에 하나의 스레드만 Python 실행
- ✓ **CPU bound** 작업에서 **threading**은 효과 없음
- ✓ **I/O bound** : **GIL**이 대기 중 해제  
→ **threading** 효과적
- ✓ **NumPy** : C 확장으로 **GIL** 해제 후 병렬 연산
- ✓ **Python 3.13+** **GIL** 옵션 제거 실험 진행 중

```
GIL 이해
import threading

CPU bound — threading 효과 없음 (GIL 때문)
def cpu_task():
 return sum(range(10**7))

I/O bound — threading 효과 있음
import time
def io_task():
 time.sleep(1) # GIL 해제됨

NumPy: GIL 해제로 병렬 연산
import numpy as np
A = np.random.rand(1000, 1000)
B = np.random.rand(1000, 1000)
C = np.dot(A, B) # BLAS: GIL 없이 멀티스레드
```

# asyncio + httpx — 비동기 API 데이터 수집

## 【 코드 예시 】

### 핵심 내용

- ✓ **async def / await**  
: 비동기 함수 정의와 호출
- ✓ **asyncio.gather()**  
: 여러 태스크 동시 실행
- ✓ **httpx.AsyncClient**  
: 비동기 HTTP 클라이언트
- ✓ **return\_exceptions=True**  
: 일부 실패 허용
- ✓ **asyncio.run()** : 이벤트 루프 시작

```
asyncio + httpx 실전
import asyncio, httpx

async def fetch(client, url):
 try:
 r = await client.get(url, timeout=10)
 return r.json()
 except Exception as e:
 return {'error': str(e)}

async def fetch_all(urls):
 async with httpx.AsyncClient() as c:
 tasks = [fetch(c, u) for u in urls]
 return await asyncio.gather(*tasks,
 return_exceptions=True)

실행: 100개 URL 동시 수집
urls = [f'https://api.example.com/data/{i}' for i in range(100)]
results = asyncio.run(fetch_all(urls))
순차: ~100초 → 비동기: ~2초
```

# timeit · cProfile · memory\_profiler — 성능 측정

## 【 코드 예시 】

### 핵심 내용

- ✓ **timeit**: 코드 조각 실행 시간 정밀 측정
- ✓ **cProfile**: 함수별 호출 횟수·시간 분석
- ✓ **sys.getsizeof**: 객체 메모리 크기 (shallow)
- ✓ **memory\_profiler**: 라인별 메모리 추적 (@profile)
- ✓ **VS Code**: cProfile 결과를 시각화 확장 활용

```
성능 측정 도구
import timeit, cProfile, sys

timeit: 반복 측정으로 정확도 향상
t = timeit.timeit(
 """.join(my_list)",
 setup="my_list=['a']*1000",
 number=10000)
print(f'{t:.4f}초') # 평균 실행 시간

cProfile: 전체 함수 호출 분석
cProfile.run('heavy_analysis(df)', sort='cumtime')

sys.getsizeof: 메모리
lst = list(range(10000))
gen = (x for x in range(10000))
print(sys.getsizeof(lst)) # ~87KB
print(sys.getsizeof(gen)) # ~104B
```

# 비동기·병렬 처리가 데이터 분석가에게 필요한 이유

## API 수집 속도 100× 향상

- 공공 API 100개를 순차 호출하면 100초, asyncio 동시 호출하면 ~2초

## 대용량 전처리 병렬화

- 수십 GB CSV를 multiprocessing으로 분할 처리
- macOS M1 8코어 → 이론상 8× 속도 향상

## 병목 찾기 → 올바른 최적화

- 느리다고 느낌으로 최적화하면 안 됨
- cProfile로 어느 함수가 느린지 먼저 측정, 그 다음 최적화.

## 후속 과목 기반 — 모두 **async**

- LangChain·RAG·AI Agent 과목은 전부 async 기반 코드
- 지금 개념 없이는 코드 구조 이해가 어려움

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## [Day 1] 종합 실습

- 실무형 수집·검증·품질 파이프라인

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 데이터 수집 미니 파이프라인

## 사용 API

- ✓ **Open-Meteo** (서울 3일 시간대별 기온·강수확률)  
[https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.9780&hourly=temperature\\_2m,precipitation\\_probability&forecast\\_days=3&timezone=Asia/Seoul](https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.9780&hourly=temperature_2m,precipitation_probability&forecast_days=3&timezone=Asia/Seoul)
- ✓ **Countries.dev** (한국 국가 정보)  
<https://countries.dev/alpha/KOR>
- ✓ **ip-api** (IP 기반 지역 정보)  
<http://ip-api.com/json/8.8.8.8>

# 데이터 수집 미니 파이프라인

## 실습 준비

- ✓ 환경 준비
  - **venv** 생성 및 활성화 후 필요한 패키지를 **requirements.txt**로 관리하여 설치
- ✓ 비동기 수집
  - **asyncio + httpx**를 사용해 위 3개 **API**를 동시에 수집하는 파이프라인 작성 (**asyncio.gather()** 활용)
- ✓ 스키마 검증
  - 수집한 **JSON**에서 필요한 필드를 추출하여 **Pydantic v2** 모델로 타입·범위 검증
- ✓ 저장 및 성능 비교
  - 검증 통과한 데이터를 **CSV**와 **Parquet** 두 형식으로 저장하고 읽기/쓰기 시간 측정·비교
- ✓ 테스트 및 **Git** 커밋
  - **pytest**로 스키마 검증 테스트 작성, **ruff**로 코드 스타일 검사 결과 정리

# 채점 기준 – 100점 (제출 기한 준수!)

## 환경 구성 + 비동기 수집(35점)

- venv 활성화 상태에서 requirements.txt로 설치 완료
- 3개 API 모두 asyncio.gather()로 동시 수집, 응답 정상 확인

## 스키마 검증 + 저장 비교 (45점)

- Pydantic v2 모델 정의, 타입 오류 시 예외 처리 포함
- CSV·Parquet 모두 저장, 성능 측정 결과 출력

## 테스트·커밋 (10점)

- pytest 1건 이상 통과, ruff check 오류 없음, Git 커밋 이력 존재

## 완성도 (10점)

- 주석처리 누락 시 감점처리



# 제출물

## 코드

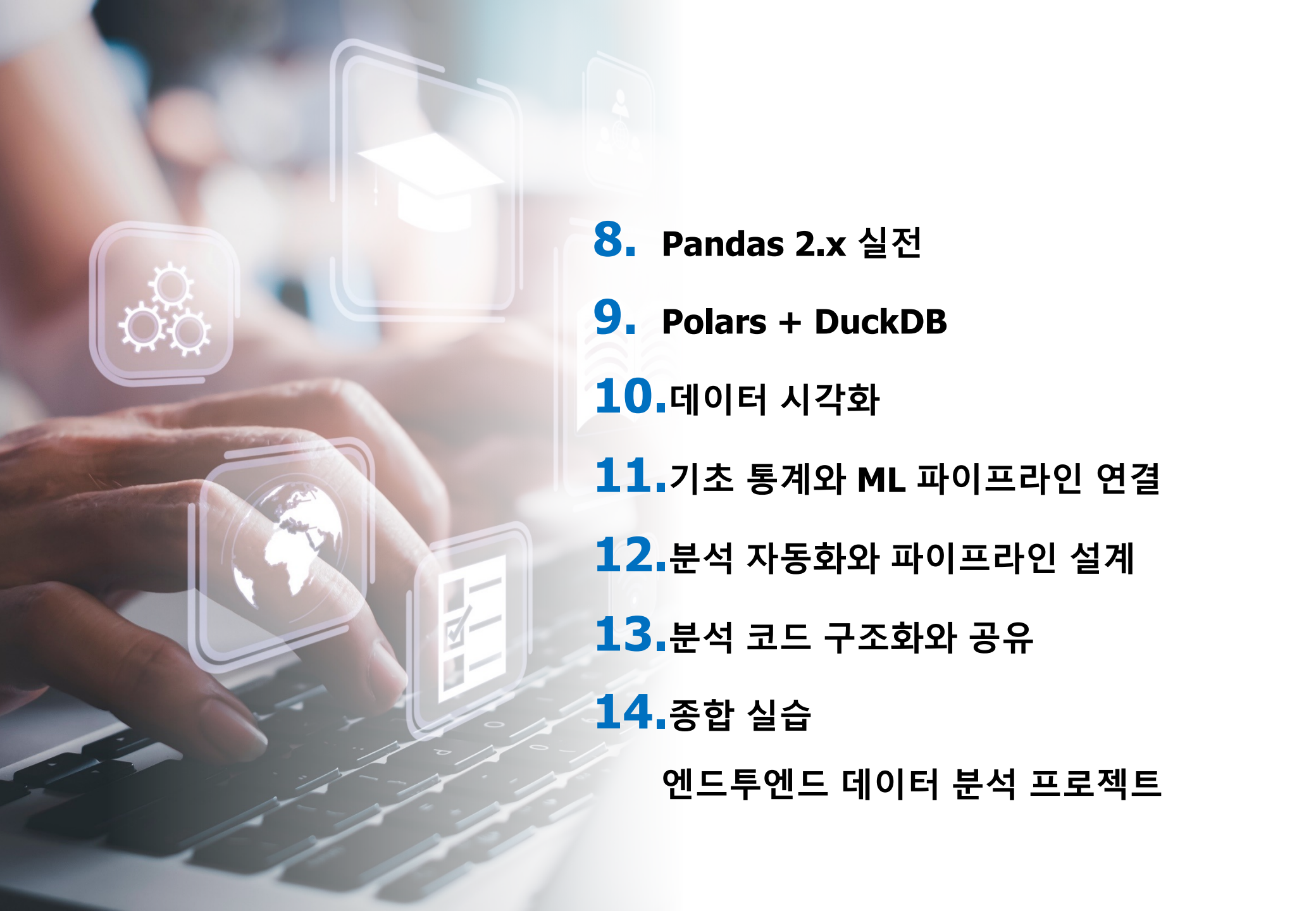
- 폴더 포함 전체 코드
- 캠퍼스명\_반\_이름\_실습명.zip으로 제출
  - 서울\_1반\_홍길동\_day1종합실습.zip
  - Github 연동 후 해당 폴더 구조 그대로 다운후 압축 제출

## 실행결과 정리 (PDF 제출)

- 실행결과 화면 Capture
- Code 분석 결과에 대한 본인 의견
  - 추가 의견  
(개선 사항, 코드 품질 개선 측면 등)

## 제출기한

- 1차 기한 : Day 1 과정 종료 전까지 제출
- 2차 기한 : Day 2 과정 시작 1시간 전
- 제출 기한 초과 시 감점 처리 예정



**8. Pandas 2.x 실전**

**9. Polars + DuckDB**

**10. 데이터 시각화**

**11. 기초 통계와 ML 파이프라인 연결**

**12. 분석 자동화와 파이프라인 설계**

**13. 분석 코드 구조화와 공유**

**14. 종합 실습**

**엔드투엔드 데이터 분석 프로젝트**

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 8. Pandas 2.x 실전

▪ DataFrame · EDA · 결측치·이상치 · groupby · pivot · merge · CoW

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# Pandas — 데이터 분석 표준 라이브러리

## DataFrame: 2D 레이블 표

- 엑셀 시트를 코드로
- 행·열에 레이블이 있고, 컬럼마다 다른 타입 가능
- 내부는 NumPy 배열.

## Series: 1D 레이블 배열

- DataFrame의 한 컬럼
- 인덱스 기반 정렬·조인이 강점
- NumPy 배열 + 인덱스

## Pandas 2.x 핵심 변경

- Copy-on-Write 기본 활성화
- ArrowDtype 지원
- 메모리 효율 향상 / nullable 정수·문자열

## 언제 Pandas?

- 수백만 행 이하 EDA·탐색
- 시각화 라이브러리 연동
- Jupyter 탐색 위주

# DataFrame 기초 EDA — 처음 마주칠 때

## 핵심 내용

- ✓ **read\_csv/parquet**: 데이터 로딩
- ✓ **info()**: 컬럼·타입·결측치·메모리 한눈에
- ✓ **describe()**: 수치형 기술통계 자동
- ✓ **head/tail/sample()**: 행 샘플 확인
- ✓ **dtypes**: 타입 확인, **astype()**: 타입 변환

## 【 코드 예시 】

```
기초 EDA 패턴
import pandas as pd

df = pd.read_csv('sales.csv')

기초 확인
df.shape # (행수, 열수)
df.info() # 타입·결측·메모리
df.describe() # 수치 기술통계
df.describe(include='all') # 범주형 포함

타입 변환
df['date'] = pd.to_datetime(df['date'])
df['region'] = df['region'].astype('category')

컬럼 선택
df['amount'] # Series
df[['region', 'amount']] # DataFrame
df.loc[df['amount'] > 1000] # 조건 필터
```

# 결측치·이상치 탐지 및 처리

## 핵심 내용

- ✓ **isna().sum():** 컬럼별 결측치 수·비율
- ✓ **MCAR/MAR/MNAR:** 결측 메커니즘 이해
- ✓ **fillna:**  
수치(평균/중앙값)/범주형(최빈값)
- ✓ **dropna:** subset 지정으로 특정 컬럼만
- ✓ **IQR:** 사분위 범위로 이상치 탐지·제거

## 【 코드 예시 】

```
결측치·이상치 처리
결측치 파악
df.isna().sum()
df.isna().sum() / len(df) * 100 # 비율

처리 전략
df['amount'].fillna(df['amount'].median())
df['category'].fillna(df['category'].mode()[0])
df.dropna(subset=['date', 'amount'])

IQR 이상치 탐지
Q1 = df['amount'].quantile(0.25)
Q3 = df['amount'].quantile(0.75)
IQR = Q3 - Q1
lo, hi = Q1 - 1.5*IQR, Q3 + 1.5*IQR
df_clean = df[df['amount'].between(lo, hi)]
print(f'이상치 {(~df["amount"].between(lo,hi)).sum()}건 제거')
```

# groupby · pivot\_table · merge — 집계와 결합

## 【 코드 예시 】

### 핵심 내용

- ✓ **groupby + agg():** 다중 집계 한 번에
- ✓ **named aggregation:** 결과 컬럼명 지정
- ✓ **pivot\_table:** 엑셀 피벗 테이블과 동일
- ✓ **merge:** SQL JOIN (inner/left/right/outer)
- ✓ **join:** 인덱스 기준 결합 (merge 간편 버전)

```
집계·결합 실전
다중 집계 + 컬럼명 지정
monthly = df.groupby('month').agg(
 revenue=('amount', 'sum'),
 cnt=('amount', 'count'),
 avg=('amount', 'mean')
).reset_index()

pivot_table
pivot = df.pivot_table(
 values='amount', index='region',
 columns='category', aggfunc='sum',
 fill_value=0)

merge (LEFT JOIN)
result = pd.merge(df_sales, df_cust,
 on='customer_id', how='left')

merge vs join 차이
merge: 컬럼 기준 | join: 인덱스 기준
```

# Pandas 2.x — Copy-on-Write(CoW) 동작 이해

2.0부터 기본 활성화. 뷰(View)를 수정하면 경고/오류 → 명시적 `copy()` 또는 `assign()` 사용 필요

## 【 코드 예시 】

```
Pandas 2.x에서 ChainedAssignmentError
df_seoul = df[df['region'] == '서울']
df_seoul['amount'] = df_seoul['amount'] * 1.1 # 경고!
```

```
방법 1: .copy() 명시
df_seoul = df[df['region'] == '서울'].copy()
df_seoul['amount'] *= 1.1
```

```
방법 2: .loc 직접 수정
df.loc[df['region'] == '서울', 'amount'] *= 1.1
```

→ `assign() + query()` 체이닝 패턴이  
가장 안전하고 가독성도 좋다



# apply vs vectorized 연산 — 성능 차이

## 핵심 내용

- ✓ **apply()**  
: 행/열 단위로 **Python** 함수 적용
- ✓ 벡터화 연산  
: **NumPy** 수준 속도 (**C** 레벨)
- ✓ **apply()**는 편리하지만 느림  
(**Python** 루프)
- ✓ **str** 접근자  
: **.str.lower()/str.contains()**
- ✓ **dt** 접근자  
: **.dt.year/.dt.month** 날짜 처리

## 【 코드 예시 】

```
벡터화 vs apply
느린 방법: apply (Python 루프)
df['upper'] = df['name'].apply(lambda x: x.upper())

빠른 방법: 벡터화
df['upper'] = df['name'].str.upper()

날짜 처리: dt 접근자
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['weekday'] = df['date'].dt.day_name()

속도 비교 (100만 행 기준)
apply: ~3.2s
str.upper(): ~0.08s (40x 빠름)
```

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 9. Polars + DuckDB

- Lazy API · scan\_csv · DuckDB SQL · Arrow · 도구 선택 기준

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 왜 Polars와 DuckDB인가 — 2026 표준

## Pandas의 한계

- 싱글스레드
- 수백만 행 이상에서 속도 저하
- 메모리 사용량이 원본 데이터의 5-10x
- CoW 전까지 복사 많음

## DuckDB — 파일 직접 SQL

- CSV·Parquet 파일을 로딩 없이 SQL 분석
- 서버 없음
- 로컬 DWH(Data Warehouse)급 성능

## Polars — Rust + Arrow

- 멀티스레드 자동 활용
- Lazy API로 쿼리 최적화
- Pandas 대비 5-20x 빠름
- Apache Arrow 기반.

## Arrow 생태계 제로카피

- Polars ↔ DuckDB ↔ PyArrow ↔ Pandas 간 Arrow 포맷으로 제로카피 변환
- 데이터 이동 비용 최소화

# Polars — Eager API vs Lazy API

## 핵심 내용

- ✓ **Eager** : 즉시 실행, 탐색·소규모에 적합
- ✓ **Lazy**  
: `scan_` 함수, 실행 계획 수립 후 `collect()`
- ✓ **Lazy 장점**  
: **predicate pushdown** / **projection pushdown**
- ✓ **collect()**: 실제 실행 트리거
- ✓ **schema\_overrides**  
: 컬럼 타입 명시 (성능↑)

## 【 코드 예시 】

```
Polars Eager·Lazy
import polars as pl

Eager (즉시 실행)
df = pl.read_csv('sales.csv')
result = df.filter(pl.col('amount') > 0)

Lazy (실행 계획 최적화)
result = (
 pl.scan_csv('large.csv',
 schema_overrides={'amount': pl.Float64})
 .filter(pl.col('region') == '서울')
 .filter(pl.col('amount') > 0)
 .group_by('category')
 .agg([pl.col('amount').sum().alias('total'),
 pl.count().alias('cnt')])
 .sort('total', descending=True)
 .collect() # 여기서 실제 실행
)
```

# Polars 핵심 문법 — Pandas와 비교

## 핵심 내용

- ✓ **pl.col():** 컬럼 참조 (Pandas: `df['col']`)
- ✓ **filter():** 행 필터 (Pandas: `df[mask]`)
- ✓ **select():** 컬럼 선택·변환
- ✓ **with\_columns():** 컬럼 추가·수정 (Pandas: `assign`)
- ✓ **group\_by + agg:** `groupby + agg`와 동일

## 【 코드 예시 】

```
Pandas → Polars 변환 예시

Pandas
df[df['amount']>0][['region','amount']]

Polars
df.filter(pl.col('amount') > 0).select(['region','amount'])

컬럼 추가 (assign 대응)
df.with_columns(
 (pl.col('amount') * 1.1).alias('adjusted')
)

문자열 처리
df.with_columns(
 pl.col('region').str.to_uppercase(),
 pl.col('date').str.to_date('%Y-%m-%d')
)

Pandas 변환
df_pd = df.to_pandas()
df_pl = pl.from_pandas(df_pd)
```

# DuckDB — CSV·Parquet에 직접 SQL

## 핵심 내용

- ✓ **duckdb.sql()**  
: 파일에 바로 SQL
- ✓ **\*.csv, \*.parquet** 와일드카드 지원
- ✓ **JOIN·GROUP BY·WINDOW** 함수  
모두 지원
- ✓ **.df()→Pandas / .pl()→Polars** 변환
- ✓ **CREATE TABLE AS SELECT**  
: 영구 저장

## 【 코드 예시 】

```
DuckDB SQL 실전
import duckdb

파일에 직접 SQL (로딩 불필요)
result = duckdb.sql("""
 SELECT region,
 SUM(amount) AS total,
 AVG(amount) AS avg,
 COUNT(*) AS cnt
 FROM 'data/*.csv'
 WHERE year = 2024
 AND amount > 0
 GROUP BY region
 ORDER BY total DESC""").df()

여러 파일 JOIN
duckdb.sql("""
 SELECT s.*, c.tier
 FROM 'sales.parquet' s
 JOIN 'customers.csv' c ON s.cid = c.id""").show()
```

# 도구 선택 가이드 — Pandas vs Polars vs DuckDB

| 항목        | Pandas       | Polars        | DuckDB    |
|-----------|--------------|---------------|-----------|
| API 스타일   | 즉시 실행(Eager) | Lazy (쿼리 최적화) | SQL       |
| 멀티스레딩     | X (GIL)      | 자동            | 자동        |
| 메모리 효율    | 보통           | 높음 (Arrow)    | 높음 (스트리밍) |
| 적합 크기     | ~ 수백만 행      | 수천만 행 +       | 파일 직접 쿼리  |
| 학습 곡선     | 낮음 (표준)      | 중간            | 낮음 (SQL)  |
| Pandas 연동 | 기본           | Arrow 변환      | .df() 변환  |
| 권장        | EDA, 탐색      | 대용량 처리        | 파일 직접 분석  |

# Apache Arrow — 제로카피 데이터 변환 생태계

## Arrow란?

- 컬럼형 인메모리 포맷
- 모든 데이터 도구가 Arrow를 중간 포맷으로 사용하면 직렬화 비용 0

## Parquet의 장점

- 컬럼형 저장 → 필요한 컬럼만 읽음
- CSV 대비 10× 빠른 읽기, 5× 작은 파일 크기

## Polars ↔ DuckDB 제로카피

- `pl.scan_parquet` → `DuckDB.execute()` → `pl.DataFrame`
- 데이터 복사 없이 도구 전환

## 예시

- 원본 CSV 수집 → Parquet 변환 저장 → DuckDB/Polars로 분석
- 탐색은 Pandas로 최종 검토



SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## [Practice 3]

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# [실습 3] Pandas EDA · Polars Lazy · DuckDB SQL 비교

sales\_100k.csv

## 1) Pandas EDA 기초 탐색 + 이상치 처리

- sales\_100k.csv를 로딩하고 기본 EDA를 수행한 뒤 IQR 방법으로 이상치를 제거
- `between(Q1-1.5*IQR, Q3+1.5*IQR)` 은 IQR 기준 정상 범위를 의미합니다

## 2) Pandas groupby named aggregation

- region·category별 총매출·평균·건수를 named aggregation으로 계산하고 총매출 내림차순으로 정렬
- named aggregation: `total=('amount','sum')` 형태로 결과 컬럼명을 직접 지정

## 3) Polars Lazy API로 동일 집계 작성

- 2번 실습과 동일한 집계를 Polars Lazy API로 작성  
`scan_csv → filter → group_by → agg → sort → collect`

## 4) DuckDB SQL + 세 도구 성능 비교

- DuckDB로 동일 집계를 SQL로 작성하고, `timeit`으로 세 도구의 실행 시간을 비교

# [실습 3] Pandas EDA · Polars Lazy · DuckDB SQL 비교

## Checkpoint

- `df.info()`, `isnull().sum()` 출력 후 IQR 방법으로 이상치 제거, 제거 전·후 행 수 출력
- `region·category`별 `total·mean·count` named aggregation으로 계산, 총매출 내림차순 정렬
- `scan_csv` → `filter` → `group_by` → `agg` → `sort` → `collect` 체인 완성 및 결과 출력
- 동일 집계를 SQL GROUP BY로 작성, 결과 DataFrame으로 출력
- `timeit`으로 세 도구 실행 시간 측정, 동일 반복 횟수로 결과 출력

## 감점 대상 (감점점수)

- IQR 공식 오류 —  $Q1 - 1.5 \cdot IQR / Q3 + 1.5 \cdot IQR$  범위 잘못 적용 (-20)
- named aggregation 미사용 — `agg({'amount': 'sum'})` 방식으로 결과 컬럼명이 지정되지 않은 경우 (-20)
- Polars Eager 사용 — `scan_csv` 대신 `read_csv` 사용으로 Lazy API 미적용 (-20)
- `collect()` 누락 — Polars 체인 끝에 `collect()` 없이 LazyFrame 상태로 제출 (-20)
- `timeit` 반복 횟수 미통일 — 세 도구의 number 값이 달라 공정 비교 불가능한 경우 (-20)

# 평가 기준

## 캠퍼스명\_반\_이름. py로 제출 / Practice Rule (총점 100점)

### Code의 Comm. (20점)

- Python Code를 통한 이해관계자와의 Comm.
  - 프로그램 전체 설명 및 변경내역(머리말)
  - 함수/기능 설명(중간)

### 코드 간결성 (35점)

- 불필요한 반복 지양

### 오류/예외 처리 (35점)

- 예외, 오류에 대한 코드 반영 여부  
(간단한 코드라도 오류/예외 처리 연습 목적)

### 납기(프로젝트 연습, 10점)

- 기한 내 납기
  - 1시간 내 제출 완료

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 10. 데이터 시각화

- Matplotlib · Seaborn · Plotly Express · Altair · 시각화 선택

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 데이터 시각화가 분석의 핵심인 이유

## Anscombe의 사중주

- 평균·분산·상관계수가 동일한 4개 데이터셋이 시각화하면 완전히 다른 패턴
- 수치만으로 분석은 위험

## EDA의 첫 번째 도구

- 히스토그램·박스플롯으로 분포 파악
- 산점도로 상관관계
- 히트맵으로 결측치 패턴 확인

## 의사결정자 설득

- 수치 표보다 인터랙티브 차트 한 장이 더 빠르게 설득
- Plotly 차트는 직접 필터·줌 가능

## 모델 평가 시각화

- Confusion Matrix·학습 곡선·Feature Importance — 모델을 숫자가 아닌 그림으로 이해

# 데이터 시각화가 분석의 핵심인 이유

## Anscombe의 사중주

- 평균·분산·상관계수가 동일한 4개 데이터셋이 시각화하면 완전히 다른 패턴
- 수치만으로 분석은 위험



## EDA의 첫 번째 도구

- 히스토그램·박스플롯으로 분포 파악
- 산점도로 상관관계
- 히트맵으로 결측치 패턴 확인



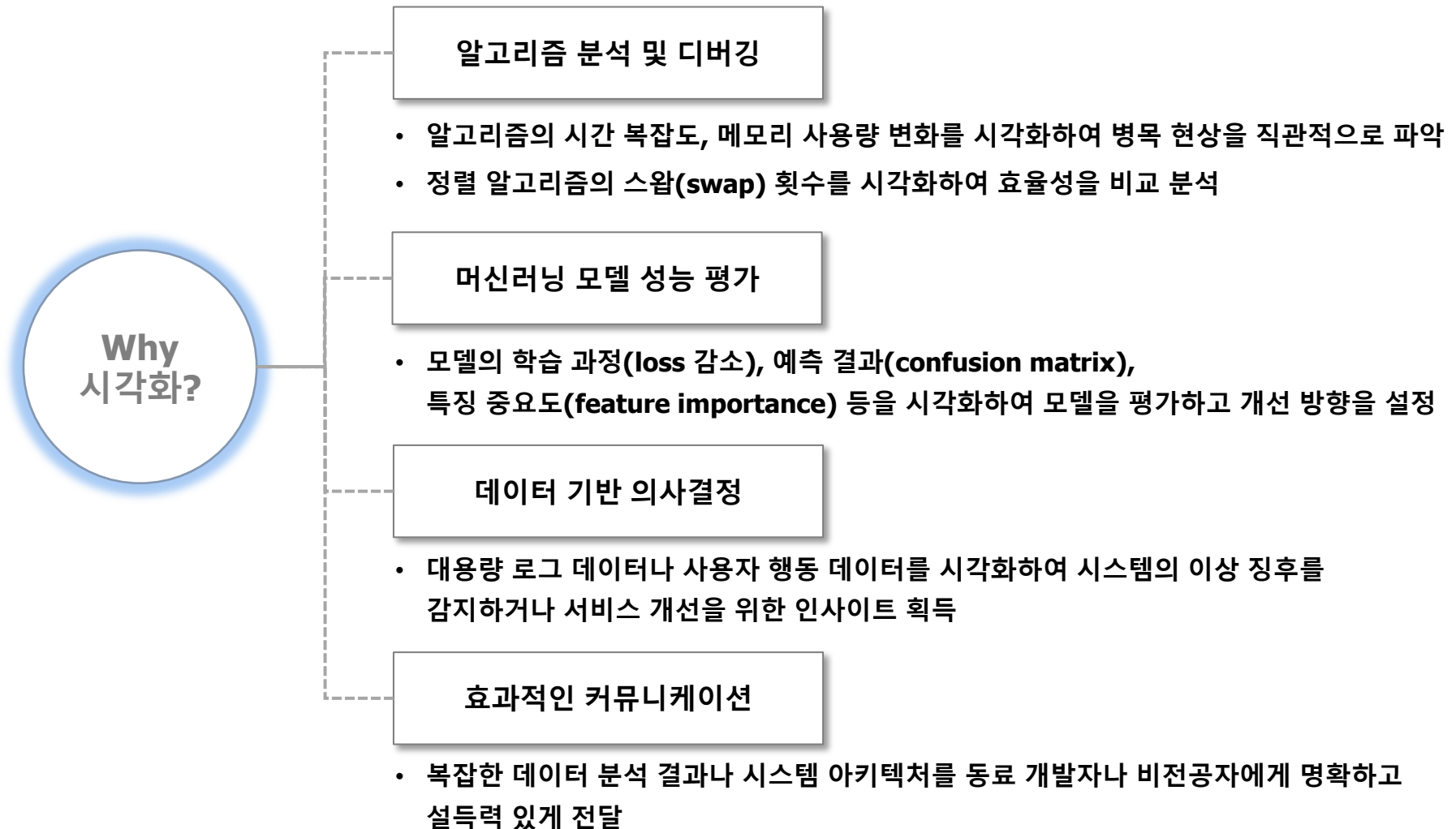
## 의사결정자 설득

- 수치 표보다 인터랙티브 차트 한 장이 더 빠르게 설득
- Plotly 차트는 직접 필터·줌 가능

## 모델 평가 시각화

- Confusion Matrix·학습 곡선·Feature Importance  
— 모델을 숫자가 아닌 그림으로 이해

# 왜 데이터 시각화인가?





## 【 Matplotlib 장점 】

- **Python** 시각화의 어머니와 같은 존재
- 거의 모든 종류의 그래프를 그릴 수 있는 강력하고 유연한 라이브러리
- 저수준(**low-level**) 제어가 가능
- 그래프의 모든 요소를 원하는 대로 커스터마이징 가능



## Matplotlib의 핵심 구조

| 항목                     | 설명                                                                                                                                                                                                              |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Figure</b><br>(피규어) | <ul style="list-style-type: none"><li>▪ 전체 그림판(캔버스) 의미</li><li>▪ 하나의 Figure 안에 여러 개 그래프(Axes)를 저장 가능</li><li>▪ 그림의 크기, 해상도, 배경색 등 속성 전반 관리</li></ul>                                                            |
| <b>Axe</b><br>(축)      | <ul style="list-style-type: none"><li>▪ 실제 데이터가 그려지는 개별 그래프(플롯) 의미</li><li>▪ x축, y축, 눈금(tick), 레이블(label), 제목(title) 등 그래프의 구성 요소를 포함</li><li>▪ 'Axes'는 복수형이 아니라 '축(axis)들이 모여있는 공간'이라는 의미의 단수형 객체 이름</li></ul> |

## 선 그래프 그리기

- plt.plot 사용

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y) # 선 그래프
plt.title("Line Graph") # 제목 설정
plt.xlabel("X-axis") # X축 레이블
plt.ylabel("Y-axis") # Y축 레이블
plt.show() # 그래프 출력
```

## 막대 그래프 그리기

- plt.bar 사용

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C']
values = [5, 7, 3]

plt.bar(categories, values) # 막대 그래프
plt.title("Bar Chart") # 제목 설정
plt.xlabel("Categories") # X축 레이블
plt.ylabel("Values") # Y축 레이블
plt.show() # 그래프 출력
```

# Matplotlib

## 축 레이블 및 제목 설정

- plt.xlabel, plt.ylabel, plt.title
- 축과 제목을 설정하여 그래프를 더 이해하기 쉽게

```
import matplotlib.pyplot as plt
```

```
plt.plot([1, 2, 3], [4, 5, 6])
```

```
plt.title("Example Graph") # 제목 설정
```

```
plt.xlabel("X-axis Label") # X축 이름 설정
```

```
plt.ylabel("Y-axis Label") # Y축 이름 설정
```

```
plt.show()
```



# Matplotlib 예시

```
import matplotlib.pyplot as plt
import numpy as np
```

```
데이터 준비
```

```
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
```

```
1. Figure 객체 생성 (전체 캔버스)
```

```
fig = plt.figure(figsize=(10, 5))
```

```
2. Figure 안에 Axes 객체(서브플롯) 추가
```

```
1행 2열의 첫 번째 위치에 ax1 생성
```

```
ax1 = fig.add_subplot(1, 2, 1)
```

```
1행 2열의 두 번째 위치에 ax2 생성
```

```
ax2 = fig.add_subplot(1, 2, 2)
```

```
3. 각 Axes 객체에 데이터 플로팅
```

```
ax1.plot(x, y1, color='blue', label='sin(x)')
```

```
ax1.set_title('Sine Wave')
```

```
ax1.set_xlabel('x')
```

```
ax1.set_ylabel('y')
```

```
ax1.grid(True)
```

```
ax1.legend()
```

```
ax2.plot(x, y2, color='red', linestyle='--', label='cos(x)')
```

```
ax2.set_title('Cosine Wave')
```

```
ax2.set_xlabel('x')
```

```
ax2.set_ylabel('y')
```

```
ax2.legend()
```

```
Figure 전체 제목 설정
```

```
fig.suptitle('Sine and Cosine Waves', fontsize=16)
```

```
레이아웃 자동 조정
```

```
plt.tight_layout()
```

```
화면에 표시
```

```
plt.show()
```

# Matplotlib 예시

```
import matplotlib.pyplot as plt
import numpy as np
```

```
데이터 준비
```

```
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
```

```
1. Figure 객체 생성 (전체 캔버스)
```

```
fig = plt.figure(figsize=(10, 5))
```

```
2. Figure 안에 Axes 객체(서브플롯) 추가
```

```
1행 2열의 첫 번째 위치에 ax1 생성
```

```
ax1 = fig.add_subplot(1, 2, 1)
```

```
1행 2열의 두 번째 위치에 ax2 생성
```

```
ax2 = fig.add_subplot(1, 2, 2)
```

```
3. 각 Axes 객체에 데이터 플로팅
```

```
ax1.plot(x, y1, color='blue', label='sin(x)')
```

```
ax1.set_title('Sine Wave')
```

```
ax1.set_xlabel('x')
```

```
ax1.set_ylabel('y')
```

```
ax1.grid(True)
```

```
ax1.legend()
```

```
ax2.plot(x, y2, color='red', linestyle='--', label='cos(x)')
```

```
ax2.set_title('Cosine Wave')
```

```
ax2.set_xlabel('x')
```

```
ax2.set_ylabel('y')
```

```
ax2.legend()
```

```
Figure 전체 제목 설정
```

```
fig.suptitle('Sine and Cosine Waves', fontsize=16)
```

```
레이아웃 자동 조정
```

```
plt.tight_layout()
```

```
화면에 표시
```

```
plt.show()
```

【 자주 사용하는 Matplotlib 플롯 】

| 플롯                  | 용도                        |
|---------------------|---------------------------|
| <b>plot()</b>       | 선 그래프 (시계열 데이터, 함수 표현)    |
| <b>scatter()</b>    | 산점도 (두 변수 간의 관계)          |
| <b>bar()/barh()</b> | 막대 그래프 (범주형 데이터 비교)       |
| <b>hist()</b>       | 히스토그램 (수치형 데이터의 분포)       |
| <b>boxplot()</b>    | 박스 플롯 (데이터의 사분위수, 이상치 탐색) |
| <b>imshow()</b>     | 이미지 데이터 시각화 (행렬, 픽셀 데이터)  |

# 다중 그래프 그리기(subplots) – matplotlib

**subplots**를 이용하여 여러 그래프를 하나의 화면에 배치하고, 스타일 설정을 통해 그래프의 시각적 품질 향상

## 기본 사용법

### matplotlib.pyplot.subplots()

#### • 개념

- 여러 개의 그래프를 그리드 형태로 배치
- 여러 개 서브 플롯을 만들 수 있게 해 주며, 각 서브 플롯에 별도의 그래프를 그릴 수 있음

#### • **subplots()**의 주요 파라미터

- `nrow()` : 행의 수(subplot 수)
- `ncols()` : 열의 수(subplot 수)
- `figsize` : 전체 그림의 크기
- `sharex`, `sharey` : X축 혹은 Y축 공유할 지 여부

### 예제

```
import matplotlib.pyplot as plt

2행 2열로 서브 플롯 만들기
fig, axes = plt.subplots(2, 2)

서브 플롯에 그래프 그리기
axes[0, 0].plot([1, 2, 3], [4, 5, 6]) # 첫 번째 서브 플롯
axes[0, 1].bar([1, 2, 3], [4, 5, 6]) # 두 번째 서브 플롯
axes[1, 0].scatter([1, 2, 3], [4, 5, 6]) # 세 번째 서브 플롯
axes[1, 1].hist([1, 2, 3, 4, 5, 6], bins=3) # 네 번째 서브 플롯

그래프 간의 간격을 자동으로 조정
plt.tight_layout()

그래프 출력
plt.show()
```

# 다중 그래프 그리기(subplots) – matplotlib

**subplots**를 이용하여 여러 그래프를 하나의 화면에 배치하고, 스타일 설정을 통해 그래프의 시각적 품질 향상

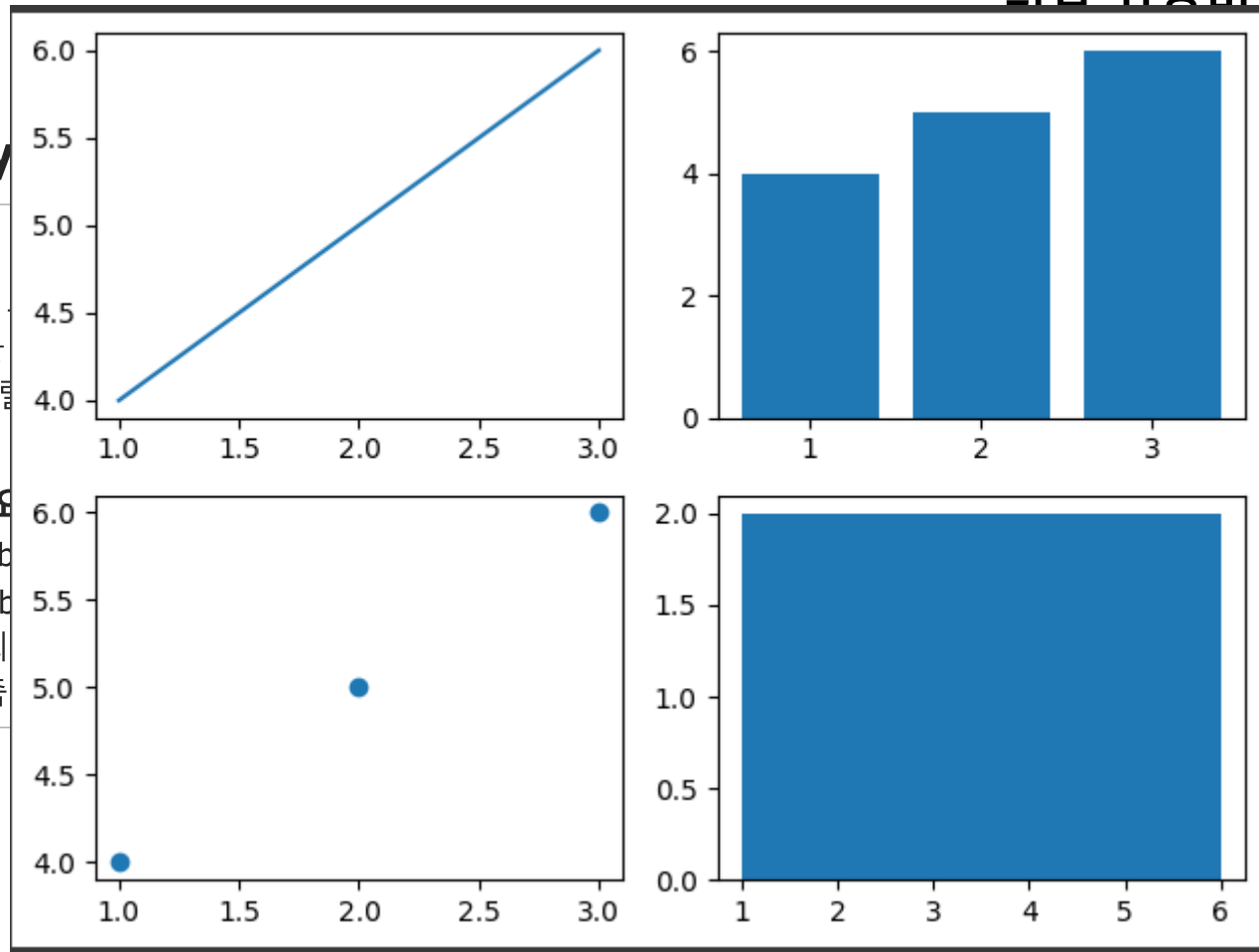
## matplotlib.pyplot

### • 개념

- 여러 개의 그래프를
- 여러 개 서브 플롯을  
별도의 그래프를 그림

### • **subplots()**의 주요

- `nrow()` : 행의 수(sub
- `ncols()` : 열의 수(sub
- `figsize` : 전체 그림의
- `sharex, sharey` : X축



첫 번째 서브 플롯  
두 번째 서브 플롯  
세 번째 서브 플롯  
네 번째 서브 플롯



# Seaborn

여러 변수 간의 관계를 한눈에 파악할 수 있는 강력한 다변량 분석 그래프들을 제공

## 【 pairplot() 】

- 데이터프레임의 모든 숫자형 변수 쌍에 대한 산점도와 각 변수의 분포(히스토그램 또는 커널 밀도 추정)를 한 번에 그려줌
- 데이터 탐색(EDA) 초기에 변수 간 관계를 파악하는 데 매우 유용

```
iris 데이터셋 로드
iris = sns.load_dataset('iris')

품종(species)에 따라 다른 색으로 pairplot 그리기
sns.pairplot(iris, hue='species', markers=["o", "s", "D"])
plt.show()
```

## 【 heatmap() 】

- 행렬 형태의 데이터를 색상을 이용해 시각화
- 변수 간의 상관관계수(correlation) 행렬이나 머신러닝 모델의 혼동 행렬(confusion matrix)을 시각화하는 데 탁월

```
iris 데이터의 상관관계수 계산
correlation_matrix = iris.drop(columns='species').corr()

히트맵으로 시각화
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True,
 cmap='coolwarm', linewidths=.5)
plt.title('Correlation Matrix of Iris Features')
plt.show()
```

## ✓ 1. 개념

- Seaborn은 Python의 데이터 시각화 라이브러리
- Matplotlib 위에서 동작
- 통계적 데이터 시각화에 최적화
- Matplotlib보다 더 간단하고 아름다운 그래프 생성 가능
- 다양한 내장 데이터셋과 고급 그래프 스타일을 제공
- 데이터 분석 용이

## 설치, 설정, 내장 데이터셋 활용

### 설치

```
pip install seaborn
```

### 기본 설정

```
import seaborn as sns
import matplotlib.pyplot as plt
```

### 내장 데이터셋 활용 (Google Colab에서 실행)

```
import seaborn as sns
import matplotlib.pyplot as plt

내장 데이터셋 로드
tips = sns.load_dataset('tips')
print(tips.head()) # 데이터 확인
```

# Seaborn의 주요 그래프 그리기

## 히스토그램 (Histogram)

- 데이터의 분포를 확인할 때 사용

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data=tips, x='total_bill', bins=10)
'total_bill' 분포
plt.title("Histogram of Total Bill")
plt.show()
```

## 상자 그림 (Boxplot)

- 데이터의 군집

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(data=tips, x='day', y='total_bill')
요일별 'total_bill' 분포
plt.title("Boxplot of Total Bill by Day")
plt.show()
```

# Seaborn의 주요 그래프 그리기

## 막대 그래프 (Barplot)

- 범주형 변수와 수치형 변수의 관계

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.barplot(x='day', y='total_bill', data=tips)
plt.title("Barplot of Total Bill by Day")
plt.show()
```

## 산점도 (Scatterplot)

- 두 변수 간의 관계를 시각화

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(data=tips, x='total_bill', y='tip',
 hue='time')
plt.title("Scatterplot of Total Bill vs Tip")
plt.show()
```

# Seaborn 고급 기능 – 스타일 설정, 색상/옵션 활용

## 스타일 적용

- 범주형 변수와 수치형 변수의 관계

```
sns.set_style('whitegrid') # 'darkgrid', 'whitegrid', 'dark',
'white', 'ticks' 중 선택 가능
```

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.histplot(data=tips, x='total_bill')
plt.title("Styled Histogram")
plt.show()
```

## 색상 및 옵션 활용

- 색상 팔레트 설정

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.barplot(x='day', y='total_bill', data=tips,
palette='Set2') # 색상 팔레트 적용

plt.title("Barplot with Custom Palette")

plt.show()
```

- hue 옵션 : 데이터를 그룹별로 나눠 색상을 다르게 표시

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.boxplot(data=tips, x='day', y='total_bill',
hue='sex') # 성별에 따른 분포 구분
plt.title("Boxplot with Hue")
plt.show()
```

# 다중 그래프 그리기(subplots) – seaborn

Seaborn은 Matplotlib의 기능을 확장하여 더 쉽고 아름다운 시각화를 제공하며 **seaborn**도 **matplotlib.pyplot.subplots()** 사용

## matplotlib.pyplot.subplots()

### • 개념

- 여러 개의 그래프를 그리드 형태로 배치
- 여러 개의 서브 플롯을 만들 수 있게 해 주며, 각 서브 플롯에 별도의 그래프를 그릴 수 있음

### • subplots()의 주요 파라미터

- nrow() : 행의 수(subplot 수)
- ncols() : 열의 수(subplot 수)
- figsize : 전체 그림의 크기
- sharex, sharey : X축 혹은 Y축 공유할 지 여부

```
import seaborn as sns
import matplotlib.pyplot as plt

예시 데이터
tips = sns.load_dataset("tips")
2행 2열로 서브 플롯 만들기
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

첫 번째 서브 플롯: 산점도 (scatterplot)
sns.scatterplot(x="total_bill", y="tip", data=tips, ax=axes[0, 0])
두 번째 서브 플롯: 박스 플롯 (boxplot)
sns.boxplot(x="day", y="total_bill", data=tips, ax=axes[0, 1])
세 번째 서브 플롯: 히트맵 (heatmap)
숫자형 데이터만 선택하여 상관 행렬 계산
corr = tips.select_dtypes(include=["float64", "int64"]).corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", ax=axes[1, 0])
네 번째 서브 플롯: 막대 그래프 (barplot)
sns.barplot(x="day", y="total_bill", data=tips, ax=axes[1, 1])

그래프 간의 간격 조정
plt.tight_layout()
그래프 출력
plt.show()
```

# Matplotlib — 기초부터 서브플롯까지

## 핵심 내용

- ✓ **Figure**  
: 전체 캔버스 / **Axes**: 개별 그래프
- ✓ 라인·막대·산점도·히스토그램 기본
- ✓ **subplot()**으로 여러 차트 나란히
- ✓ 색상·스타일·범례·제목·그리드 설정
- ✓ **savefig()**로 **PNG/PDF** 고해상도 저장

## 【 코드 예시 】

```
Matplotlib 실전
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(1,2,figsize=(12,5))

라인 차트
axes[0].plot(months, revenue, color='steelblue',
 lw=2, marker='o', label='매출')
axes[0].set_title('월별 매출 추이')
axes[0].set_xlabel('월'); axes[0].set_ylabel('원')
axes[0].legend(); axes[0].grid(alpha=0.3)

막대 차트
axes[1].bar(regions, values, color='teal', alpha=0.8)
axes[1].set_title('지역별 매출')
plt.tight_layout()
plt.savefig('report.png', dpi=150, bbox_inches='tight')
```

# Seaborn — 통계 시각화 특화 라이브러리

## 핵심 내용

- ✓ **Matplotlib** 기반의 **high-level** 인터페이스
- ✓ **hue·size·style**로 다차원 정보 인코딩
- ✓ **boxplot·violinplot**: 분포 비교
- ✓ **heatmap**: 상관행렬·결측치 패턴
- ✓ **pairplot**: 모든 변수 쌍 산점도 한 번에

## 【 코드 예시 】

```
Seaborn 통계 시각화
import seaborn as sns
import matplotlib.pyplot as plt

상관 히트맵
corr = df.select_dtypes('number').corr()
fig, axes = plt.subplots(2,2,figsize=(12,10))

sns.heatmap(corr, annot=True, cmap='coolwarm',
 fmt='.2f', ax=axes[0,0])

분포 비교 (boxplot + hue)
sns.boxplot(data=df, x='region', y='amount',
 hue='category', ax=axes[0,1])

히스토그램 + KDE
sns.histplot(data=df, x='amount',
 kde=True, ax=axes[1,0])

plt.tight_layout(); plt.show()
```



# Plotly Express — 인터랙티브 시각화

## 핵심 내용

- ✓ **px.scatter·bar·line·histogram·box**  
— 한 줄
- ✓ **color·size·hover\_name**: 다차원 인코딩
- ✓ **facet\_col/row**: 그룹별 분할 서브플롯
- ✓ **fig.write\_html()**: HTML로 공유 가능
- ✓ **update\_layout**: 제목·축·폰트 커스터마이징

## 【 코드 예시 】

```
Plotly Express 실전
import plotly.express as px

다차원 산점도
fig = px.scatter(df,
 x='gdp', y='happiness',
 color='region', size='population',
 hover_name='country',
 title='GDP vs 행복지수')

분할 막대 차트
fig2 = px.bar(monthly_df,
 x='month', y='revenue',
 color='category',
 facet_col='region',
 title='지역·카테고리별 월 매출')

HTML로 저장 (공유 가능)
fig.write_html('analysis.html')
```

# Altair 선언형 시각화 + 시각화 도구 선택 기준

## 핵심 내용

- ✓ **Grammar of Graphics** 기반 선언형 문법
- ✓ **mark\_point/bar/line + encode()** 체이닝
- ✓ **인터랙션 내장 (.interactive())**
- ✓ **EDA** 빠른 프로토타이핑에 최적
- ✓ **Streamlit**: 분석 결과 웹 공유 (소개 수준)

## 【 코드 예시 】

```
Altair + 도구 선택
import altair as alt

chart = (alt.Chart(df)
 .mark_point()
 .encode(
 x='gdp:Q',
 y='happiness:Q',
 color='region:N',
 tooltip=['country', 'gdp', 'happiness']
).interactive()
)

chart.save('chart.html')

도구 선택 기준
보고서·논문 → Matplotlib/Seaborn
인터랙티브 대시보드 → Plotly
빠른 EDA → Altair
웹 공유 → Streamlit (소개)
```

# 시각화 도구 선택 — 상황별 가이드

## Matplotlib: 커스터마이징 최강

- 학술 논문·보고서 품질 차트. 모든 요소를 픽셀 수준으로 제어
- 다른 라이브러리의 기반

## Seaborn: 통계 시각화 특화

- 분포·상관·그룹 비교에 최적
- hue 파라미터 하나로 다차원 정보 표현
- Matplotlib보다 코드 짧음

## Plotly: 인터랙티브 공유

- 줌·필터·호버를 HTML 파일 하나로 공유 가능
- 비전공자 발표에서 임팩트 최대

## Altair: 선언형 빠른 EDA

- 데이터 구조를 선언하면 차트가 완성
- 탐색 단계에서 수십 개 차트를 빠르게 만들 때

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

# 11. 기초 통계와 ML 파이프라인 연결

- 기술통계 · 상관분석 · CRISP-DM · sklearn Pipeline · joblib

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 기술통계 — 데이터를 숫자로 이해하기

## 핵심 내용

- ✓ 중심 경향: 평균·중앙값·최빈값 차이
- ✓ 산포도: 분산·표준편차·IQR·범위
- ✓ 분포 모양:  
왜도(skewness)·첨도(kurtosis)
- ✓ 상관계수:  $-1 \sim +1$ , 선형 관계 강도
- ✓ Pandas: describe() + corr() 실전

## 【 코드 예시 】

```
기술통계 실전
import numpy as np
from scipy import stats

기술통계
df.describe() # 자동 계산
df['amount'].skew() # 왜도 (0: 대칭)
df['amount'].kurt() # 첨도 (0: 정규)
df['amount'].median() # 중앙값

상관행렬
corr = df[['amount', 'visits', 'age']].corr()
양의 상관: 1에 가까울수록
음의 상관: -1에 가까울수록

분포 시각화
import seaborn as sns
sns.histplot(df['amount'], kde=True)

정규분포 검정
_, p = stats.shapiro(df['amount'].dropna())
print(f'정규분포 검정 p값: {p:.4f}')
```

# 가설 검정 기초 — t-test와 카이제곱

## 핵심 내용

- ✓ 귀무가설(H0) vs 대립가설(H1) 설정
- ✓ 유의수준  $\alpha = 0.05$ : p값 기준
- ✓ t-test: 두 그룹 평균 차이 검정
- ✓ 카이제곱: 범주형 변수 독립성 검정
- ✓  $p < 0.05 \rightarrow H0$  기각 (통계적 유의)

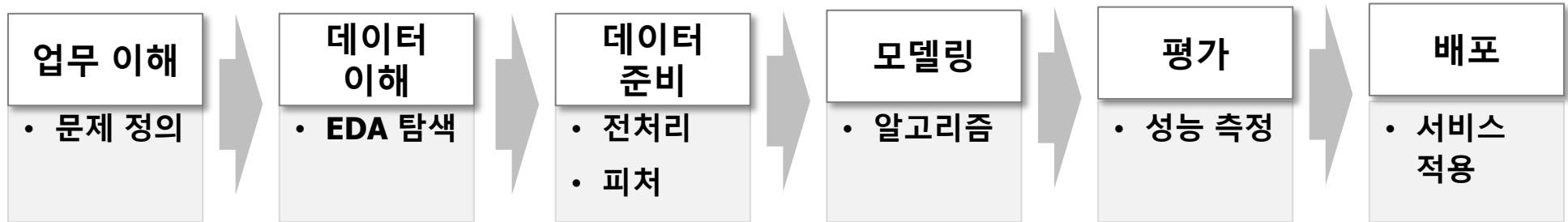
## 【 코드 예시 】

```
가설 검정 실전
from scipy import stats

t-test: A/B 그룹 매출 차이 검정
group_a = df[df['group']=='A']['amount']
group_b = df[df['group']=='B']['amount']
t, p = stats.ttest_ind(group_a, group_b)
print(f't={t:.3f}, p={p:.3f}')
if p < 0.05:
 print('통계적으로 유의미한 차이 있음')
else:
 print('차이 없음 (우연일 수 있음)')

카이제곱: 지역과 구매여부 독립성
from scipy.stats import chi2_contingency
ct = pd.crosstab(df['region'], df['purchased'])
chi2, p, dof, expected = chi2_contingency(ct)
print(f'카이제곱={chi2:.3f}, p={p:.3f}')
```

# CRISP-DM — 실무 데이터 분석 방법론



기술 중심이 아닌 문제 해결 중심 — 좋은 분석은 올바른 문제 정의에서 시작

# sklearn Pipeline — 전처리 + 모델 통합

## 핵심 내용

- ✓ **Pipeline**  
: 전처리 → 모델을 하나의 객체로
- ✓ **ColumnTransformer**  
: 수치·범주 컬럼 분리 처리
- ✓ **fit(): 학습**  
**predict(): 예측**  
**transform(): 변환**
- ✓ **joblib.dump/load**  
: 파이프라인 직렬화

## 【 코드 예시 】

```
sklearn Pipeline 실전
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler,
OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import Ridge

import joblib

preproc = ColumnTransformer([
 ('num', StandardScaler(), num_cols),
 ('cat', OneHotEncoder(), cat_cols)])

model = Pipeline([
 ('prep', preproc),
 ('reg', Ridge(alpha=1.0))])

model.fit(X_train, y_train)
print(f'R2: {model.score(X_test, y_test):.3f}')
joblib.dump(model, 'model.pkl') # 저장
loaded = joblib.load('model.pkl') # 로딩
```



# 통계와 ML 파이프라인이 연결되는 이유

## 통계 없이 ML은 블랙박스

- 평균·분산·상관계수를 모르면 피처 선택, 이상치 처리, 결과 해석이 불가능
- 모델보다 데이터 이해가 먼저

## CRISP-DM — 기술보다 프로세스

- 좋은 분석은 알고리즘이 아니라 올바른 문제 정의에서 시작
- 데이터 준비가 대부분 8~90% 차지

## 후속 과목 직접 연결

- Feature Engineering·머신러닝에서 대부분 sklearn Pipeline을 그대로 사용

## 재현 가능한 모델 관리

- joblib으로 Pipeline 전체 저장 → 배포 환경에서 동일한 전처리+예측 보장
- MLOps의 기본

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## [Practice 4]

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# [실습 4] 실습 3→4 연계

## 【 연계 Point 】

| 실습 3 산출물                   | 실습 4 활용               |
|----------------------------|-----------------------|
| IQR 이상치 제거된 DataFrame      | 시각화·통계 검정 입력 데이터로 사용  |
| region·category groupby 결과 | 카이제곱 분할표 기반 변수로 활용 가능 |
| sales_100k.csv             | Pipeline의 학습 데이터 원본   |

# [실습 4] 시각화 4종 · 통계 검정 · sklearn Pipeline

sales\_100k.csv – 실습 3 연계

## 1) EDA 시각화 4종 (2×2 서브플롯)

- fig, axes = plt.subplots(2,2)로  
히스토그램+KDE / 박스플롯 / 월별 라인 /  
상관 히트맵을 작성

## 2) 통계 검정 — t-test + 카이제곱

- 서울 vs 부산 평균 매출 차이를 t-test로,  
지역×카테고리 독립성을 카이제곱  
검정으로 확인

## 3) sklearn Pipeline 구성 + 저장

- ColumnTransformer + Pipeline을 완성하고  
훈련·평가·저장·재로딩을 순서대로 수행

## 4) Plotly 인터랙티브 차트 저장

- 지역·카테고리별 총매출을 Plotly Express  
막대 차트로 만들고 HTML로 저장

# [실습 4] 시각화 4종 · 통계 검정 · sklearn Pipeline

## Checkpoint

- 2×2 서브플롯으로 4종 차트 구성 — 히스토그램·박스플롯·바차트·히트맵 (또는 유사 조합) 한 figure에 출력
- 두 그룹(예: 성별·지역 등) 선택 후 `scipy.stats.ttest_ind` 적용, t통계량·p-value 출력 및 유의미 여부 해석 한 줄 출력
- 두 범주형 변수(예: category·payment\_method)로 분할표 생성 후 `chi2_contingency` 적용, p-value 출력 및 해석
- 전처리 + 모델 1개 이상을 Pipeline 객체로 구성, `fit·predict·score` 실행 후 `joblib`으로 모델 파일 저장
- Plotly 인터랙티브 차트 1개 이상 작성 후 .html 파일로 저장

## 감점 대상 (감점점수)

- 서브플롯 미사용 — 차트 4개를 개별 `plt.show()`로 따로 출력, `fig, axes`로 묶지 않은 경우 (-20)
- p-value 해석 누락 — t-test·카이제곱 결과에서 수치만 출력하고 유의미 여부( $p < 0.05$ ) 판단 코드·주석 없는 경우 (-20)
- Pipeline 미사용 — 전처리와 모델을 개별 단계로 분리 실행하고 Pipeline 객체로 묶지 않은 경우 (-20)
- 모델 저장 누락 — `joblib.dump()` 없이 학습만 하고 .pkl 또는 .joblib 파일 미생성 (-20)
- Plotly 차트 파일 미저장 — `fig.show()`로 화면 출력만 하고 `.write_html()` 미호출 (-20)

# 평가 기준

## 캠퍼스명\_반\_이름. py로 제출 / Practice Rule (총점 100점)

### Code의 Comm. (20점)

- Python Code를 통한 이해관계자와의 Comm.
  - 프로그램 전체 설명 및 변경내역(머리말)
  - 함수/기능 설명(중간)

### 코드 간결성 (35점)

- 불필요한 반복 지양

### 오류/예외 처리 (35점)

- 예외, 오류에 대한 코드 반영 여부  
(간단한 코드라도 오류/예외 처리 연습 목적)

### 납기(프로젝트 연습, 10점)

- 기한 내 납기
  - 1시간 내 제출 완료

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 12. 분석 자동화와 파이프라인 설계

- **schedule · Jinja2 리포트 · LLM API 개념 · 파이프라인 원칙**

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# 왜 분석을 자동화해야 하는가

## 반복 분석의 현실

- 매주 같은 매출 리포트를 수작업으로 만드는 분석가의 경우 자동화하면 수십 분 → 0분

## 데이터 최신성

- 어제 시황 및 뉴스, 산업 현황 수치 변화 등이 반영된 데이터로 만든 리포트는 오늘 오전 5시에 이미 완성돼 있어야 한다 (by 증권가 애널리스트)

## 오류 감소

- 수작업 복붙 오류 제거
- 동일한 코드가 동일한 결과를 항상 생성 → 신뢰성

## 확장성

- 10개 지역 리포트를 만드나 100개를 만드나 코드는 동일
- 규모 확장 비용 거의 없음



# schedule + cron — 반복 분석 자동화

## 핵심 내용

- ✓ **schedule**: Python 내 간단한 스케줄러
- ✓ 매일·매주·매시간 등 다양한 주기 설정
- ✓ 오류 발생 시 로깅 + 알림 패턴
- ✓ macOS launchd / cron으로 OS 레벨 스케줄
- ✓ **subprocess**로 외부 스크립트 실행 연동

## 【 코드 예시 】

```
schedule 자동화 실전
import schedule, time, logging

def run_daily_report():
 try:
 df = load_and_clean('sales.csv')
 stats = compute_stats(df)
 render_report(stats)
 logging.info('리포트 완료')
 except Exception as e:
 logging.error(f'실패: {e}')

schedule.every().day.at('08:00').do(run_daily_report)
schedule.every().monday.at('09:00').do(weekly_summary)
schedule.every(1).hours.do(check_new_data)

while True:
 schedule.run_pending()
 time.sleep(60)
```

# Jinja2 — 분석 리포트 자동 생성

## 핵심 내용

- ✓ Jinja2: Python 대표 템플릿 엔진
- ✓ {{ 변수 }}·{% for %}·{% if %} 문법
- ✓ 분석 결과·차트를 HTML 템플릿에 주입
- ✓ webbrowser.open()으로 바로 열기
- ✓ pdfkit 연동으로 PDF 변환 가능

## 【 코드 예시 】

```
Jinja2 리포트 실전
from jinja2 import Environment, FileSystemLoader
from pathlib import Path
from datetime import datetime

env = Environment(loader=FileSystemLoader('templates'))
tmpl = env.get_template('report.html')

html = tmpl.render(
 title='월간 판매 분석',
 generated=datetime.now().strftime('%Y-%m-%d'),
 summary=stats.to_dict(),
 chart_html=fig.to_html(full_html=False),
 top5=df.nlargest(5,'amount').to_dict('records')
)

out = Path('output/report.html')
out.write_text(html, encoding='utf-8')
import webbrowser
webbrowser.open(str(out.resolve()))
```

# LLM API 활용 개요 — 개념과 구조 중심

## 핵심 내용

- ✓ HTTP POST 방식  
: 모든 벤더 유사한 구조
- ✓ 요청: 메시지 배열 + 모델 파라미터
- ✓ 응답: content 배열 (text 타입)
- ✓ 토큰 기반 과금: 입력+출력 토큰 합산
- ✓ 비동기 활용: httpx로 대량 처리

## 【 코드 예시 】

```
LLM API 구조 이해
import httpx, os, asyncio

벤더 무관 HTTP 기반 구조
async def call_llm(text: str) -> str:
 async with httpx.AsyncClient() as c:
 r = await c.post(
 'https://api.{vendor}.com/v1/messages',
 headers={'Authorization': f'Bearer {os.getenv("API_KEY")}'},
 json={'model': 'model-name',
 'max_tokens': 500,
 'messages': [{'role': 'user', 'content': text}]},
 timeout=30
)
 return r.json()['content'][0]['text']

활용: 이상치 자동 설명
result = asyncio.run(call_llm(f'다음 데이터 이상치 설명: {outliers}'))
```

# 분석 파이프라인 설계 — ETL 구조 원칙

좋은 파이프라인: 각 단계 분리 + 오류 기록 + 재현 가능 + 테스트 가능

## 【 코드 예시 】

```
def run_pipeline(config: dict) -> dict:
 logger.info(f'파이프라인 시작: {config}')

 # E: Extract — 수집
 raw = extract(config['source']) # API / DB / 파일
 logger.info(f'수집 완료: {len(raw)}건')

 # V: Validate — 검증 (Pydantic)
 validated, errors = validate(raw, schema=SalesRecord)
 logger.warning(f'검증 오류: {len(errors)}건') if errors else None
```

→ 각 단계를 독립적으로 **pytest**로  
테스트 가능  
실패 시 어느 단계인지 즉시 파악

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## 13. 분석 코드 구조화와 공유

- Jupyter vs .py · 프로젝트 구조 · 모듈화 · GitHub

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

# Jupyter Notebook vs .py 스크립트 — 언제 무엇을?

## Jupyter(.ipynb)

- EDA·데이터 탐색 단계
- 시각화가 중심인 분석
- 한 번 실행으로 끝나는 분석
- 보고서로 변환해야 할 때 (nbconvert)
- 교육·튜토리얼 자료 작성

## Python(.py)

- 반복 실행 자동화 스크립트
- Git 버전 관리 + pytest 테스트
- 팀 협업·코드 리뷰 대상
- CI/CD 파이프라인 통합
- 라이브러리(src/) 코드

# 분석 프로젝트 폴더 구조(예시)

6개월 뒤 동료(또는 미래의 나)가 README만 읽고 바로 실행할 수 있어야 좋은 구조

## 【 코드 예시 】

```
data-project/
├── data/
│ ├── raw/ # 원본 데이터 (절대 수정 금지)
│ ├── processed/ # 전처리 완료 데이터
│ └── external/ # 외부 참조 데이터
├── notebooks/ # EDA·실험용 Jupyter 노트북
│ ├── 01_eda.ipynb
│ └── 02_feature_exploration.ipynb
├── src/ # 재사용 가능한 Python 모듈
│ ├── __init__.py
│ └── clean.py # 전처리 함수
```

# 모듈화 — 노트북 코드를 재사용 가능한 패키지로

## 핵심 내용

- ✓ `__init__.py`: 폴더를 Python 패키지로 만들기
- ✓ 공통 전처리 함수 → `src/clean.py`로 분리
- ✓ 시각화 함수 → `src/viz.py`로 분리
- ✓ 노트북에서 `sys.path`로 `src` 모듈 импорт
- ✓ 절대 импорт vs 상대 импорт 차이

## 【 코드 예시 】

```
모듈화 실전
src/clean.py
from typing import Optional
import pandas as pd

def clean_nulls(df: pd.DataFrame,
 cols: Optional[list] = None,
 strategy: str = 'median') -> pd.DataFrame:
 cols = cols or df.select_dtypes('number').columns.tolist()
 if strategy == 'median':
 return df.fillna(df[cols].median())
 elif strategy == 'drop':
 return df.dropna(subset=cols)
 return df

notebooks/01_eda.ipynb 에서 사용
import sys
sys.path.insert(0, '..')
from src.clean import clean_nulls
from src.viz import plot_distribution
```



# README.md 작성 + GitHub 공유

## 핵심 내용

- ✓ **README:** 프로젝트 목적·환경·실행 방법
- ✓ **배지(badge):** 테스트 통과·커버리지 표시
- ✓ **nbconvert:** 노트북 → HTML 변환·공유
- ✓ **Git add·commit·push** 기본 워크플로
- ✓ **GitHub Actions:** pytest 자동 실행

## 【 코드 예시 】

```
README + GitHub 실전
README.md 핵심 구성
프로젝트 개요
개발 환경 설정
```bash
git clone <url> && cd data-project
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```
실행 방법
pytest tests/
python src/run_pipeline.py

노트북 HTML 변환·공유
$ jupyter nbconvert --to html notebooks/01_eda.ipynb

GitHub Actions
.github/workflows/test.yml 에 pytest 자동화
```

# 분석 코드 구조화의 핵심 원칙

## 탐색과 재사용 분리

- 노트북은 탐색·실험용
- 검증된 함수는 src/로 이동해 재사용 가능한 모듈로 관리
- 두 역할을 섞지 말 것.

## 재현성 = 신뢰성

- requirements.txt + .env 예시 + README 실행 가이드
- 이 셋만 있으면 누구나 동일한 결과 재현 가능

## 데이터는 git에 올리지 않는다

- data/raw/는 .gitignore에 추가
- 대신 데이터 출처(URL·스크립트)를 README에 기록

## 점진적 개선

- 처음부터 완벽한 구조를 만들 필요 없음
- 분석이 반복되면 함수화하고, 함수가 쌓이면 모듈화

SKALA 2) AI의 서비스화데이터분석 및 AIOps – 1. 데이터 분석을 위한 Python 이해

## [Day 2] 종합 실습

- 실습 + 발표

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

## 사용 데이터셋

✓ 팀별로 아래 레코드셋 중 적용

- **NYC Yellow Taxi** : [https://d37ci6vzurychx.cloudfront.net/trip-data/yellow\\_tripdata\\_2026-05.parquet](https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2026-05.parquet)

- **Stack Overflow Survey 2024**

- 2024년 데이터 :

<https://github.com/StackExchange/Survey/raw/refs/heads/main/packages/archive/2024/results.csv>

- 전체 아카이브 폴더 (스키마 설명 등):

<https://github.com/StackExchange/Survey/tree/main/packages/archive/2024>

- 공식 안내 페이지 (연도별 링크 전부 정리) : <https://survey.stackoverflow.co/>

- **Adult Census Income**

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
cols = ["age", "workclass", "fnlwgt", "education", "education-num",
 "marital-status", "occupation", "relationship", "race", "sex",
 "capital-gain", "capital-loss", "hours-per-week", "native-country", "income"]
df = pd.read_csv(url, header=None, names=cols, na_values="?",
 skipinitialspace=True)
print(df.shape) # (32561, 15)
```

## 실습 내용

- ✓ 데이터 준비
  - 선택한 데이터셋을 **Pandas**와 **Polars** 양쪽으로 로딩하여 결과를 비교하고, 결측치·중복 처리 및 기본 **EDA** 수행
- ✓ 시각화 **Seaborn**으로 정적 차트, **Plotly**로 인터랙티브 차트 각 1개 이상 작성(분포·상관관계·그룹 비교 중 택일)
- ✓ 통계 분석기술통계(평균·표준편차·분위수) 산출, 변수 간 상관계수 계산, **scipy.stats.ttest\_ind**로 **t-test** 수행 및 **p-value** 해석
- ✓ **ML Pipelines****sklearn.pipeline.Pipeline**으로 전처리 + 모델 학습 구성, 평가 지표 출력, **joblib**으로 모델 저장
- ✓ 자동화·발표분석 결과를 **report.md**로 자동 생성 팀별 발표 (5분)

# 채점 기준 – 100점 (제출 기한 준수!)

## 데이터 준비 + 시각화 (35점)

- Pandas·Polars 모두 사용, 결측치 처리 및 EDA 결과 출력
- Seaborn 정적 차트·Plotly 인터랙티브 차트 각 1개 이상, 제목·축 레이블 포함

## 통계분석 + ML Pipeline (45점)

- 기술통계·상관계수 출력, t-test 결과 및 p-value 해석 포함
- Pipeline 객체로 전처리+모델 구성, 평가 지표(정확도·F1 등) 출력, 모델 파일 저장

## 자동화 + 발표 (20점)

- report.md 자동 생성
- 팀 발표 5분

## 완성도 (10점)

- 주석처리 누락 시 감점처리

# 제출물

## 코드

- 폴더 포함 전체 코드
- 캠퍼스명\_반\_이름\_실습명.zip으로 제출
  - 서울\_1반\_홍길동\_day2종합실습.zip
  - Github 연동 후 해당 폴더 구조 그대로 다운후 압축 제출

## 실행결과 정리 (PDF 제출)

- 실행결과 화면 Capture
- Code 분석 결과에 대한 본인 의견
  - 추가 의견  
(개선 사항, 코드 품질 개선 측면 등)

## 제출기한

- 기한 : 과정 종료 시까지 (~21:00 제출 엄수)
- 제출 기한 초과 시 감점 처리 예정



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.